

# 2025年度 コンピュータシステム 全説明スライド 画像圧縮版 前半

2025/9/26(金)

担当講師 富岡莉生

## 紙資料を持ち込み可能，問題数は多め

- 授業資料はページ数が膨大なため，必要な箇所を素早く参照できるように持ち込み資料を自作することを推奨
- 定義やアルゴリズムなどの暗記項目に対して，自身が場所を把握できる分量の資料を用意することを推奨
  1. 知識問題は，できるだけ参照せずに高速に解けるよう理解する
  2. 計算問題は，数式やアルゴリズムをすぐに見つけ出せるようにする
  3. 応用問題は，できるだけ計算特性を理解して不要な計算を避けるようにする（解放を思いつかなければ素直な手順で計算することでも回答可能）
- PowerPoint形式で資料を配布（※）するので，必要な箇所のみを抽出して，できるだけ少ない枚数に圧縮することを推奨
  - Wordや手書きノートなどに画像と説明をまとめる
  - スライド形式にまとめる場合は，6枚刷りなどにとすると枚数を約1/6にできる

※もし本講義の授業資料を授業外で使いたい場合は，本pptxファイルを使用してください。  
授業中に配布した資料には一部「教育目的の引用」および「個別の許可に基づく引用」が含まれます。

## コンピュータシステム

情報社会において**情報技術（IT）**を活用できるようになるために  
そのキーテクノロジーである**コンピュータの仕組み**を学ぶ

## 授業目的

1. コンピュータの仕組みを理解する
2. コンピュータの引き起こす現象をその仕組みから正しく把握できるようになる
3. ITシステムを仕事や生活で活用できるようになる

## 授業内容

- コンピュータ，データ・AIにできること
- ITシステムの依頼から，デザイン，実装されるまでの流れ
- コンピュータのハードウェア構成（CPU，メモリ，ストレージなど）
- コンピュータが様々な情報処理を行える仕組み（アルゴリズム，基数など）
- コンピュータの外でも役立つコンピュータを使うための思考（計算論的思考）

## 概要

この授業では、受講者が情報社会において求められる情報技術（IT）を活用できるようになることを目指す。  
そのために、キーテクノロジーであるコンピュータの仕組みを学ぶ。

## 目的

- ・まず、コンピュータの仕組みを理解して欲しい
- ・それにより、コンピュータの引き起こす現象をその仕組みから正しく把握できるようになって欲しい
- ・そして最終的に、ITシステムを仕事や生活で活用できるようになって欲しい

## 目標と内容

目標1：仕事のために、コンピュータ・ITシステムをデザインする

- ・コンピュータやデータ・AIにできることを知る
- ・ITシステムの依頼や開発をするために、依頼・開発・納品までどのように行われるかを知る
- ・コンピュータを選定できるように、ハードウェア構成（CPU、メモリ、ストレージ）を知る

目標2：既存のコンピュータ・ITシステムを利用する

- ・コンピュータの引き起こす現象を正しく理解するために、コンピュータが様々な情報処理を行える仕組み（アルゴリズム、基数など）を理解する

目標3：コンピュータ・ITシステムのためにを学んだことを、他の場面で活かす

- ・コンピュータを使うための思考が、生活や仕事の場面一般に応用できることを知る（計算論的思考）

これら目標と内容を実現するために、次とその次のスライドに示す流れで全15回の授業を構成する

1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて  
自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、  
目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる
3. ITシステムの構造や仕組みを学び、  
問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる
4. 計算機のハードウェア構成を知り、  
目的達成に必要な装置の見積もりや購入ができるようになる
5. 計算機に特有のデータ表現を学び、  
機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考  
を体験する

## I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機, インターネット, 情報システム

## II. 計算機やデータ, AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

## III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

## IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習 (データ構造とアルゴリズム)

## V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

## VI. 計算機特有の数値表現に慣れる

12. 演習 (基数と論理演算)

## VII. 計算論的思考

13. 問題解決のためのモデル: PERT図
14. 問題解決のためのモデル: 決定表
15. まとめ + 演習 (PERT図, 決定表)

# I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機, インターネット, 情報システム

## I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機，インターネット，情報システム

## II. 計算機やデータ，AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

## III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

## IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習（データ構造とアルゴリズム）

## VI. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

## VII. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

## VIII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習（PERT図，決定表）

## コンピュータシステム

情報社会において情報技術（IT）を活用できるようになるために  
そのキーテクノロジーであるコンピュータの仕組みを学ぶ

内容に入る前に  
なぜこのような授業が必要なのか考える

## コンピュータとプログラム

コンピュータは自動で“計算”を行う装置

→ 計算の仕方を書いておけば情報処理を自動化できる

プログラムは計算手順を予め書いたもの

## 情報通信技術と社会の情報化

インターネット・IoTは物理・仮想世界で膨大な情報を集積可能にする技術

→ 情報を分析・活用する技術（データ科学やAI）が発展する

→ 現実問題を「情報世界の問題」に変換すれば、情報技術の力を借りて解決できる

Industry4.0・Society5.0は情報を集積・分析して産業・社会を最適化する構想

→ 産業・社会での情報技術活用を推進するため、国民全員が情報を学ぶ方針になる

## 計算機やインターネットの仕組み

計算機内部の情報表現はデジタル（数字式）の二進表現（0 or 1）

- 物理量（アナログ値）を数字式に近似すると記録・伝達しやすくなる
- 数値を0と1のみで表現すると、単純な“スイッチ”（半導体）で実現できる

ネットワークは網目状に結ばれた計算機同士の通信経路

- インターネット（世界中のネットワーク同士が相互に通信できる仕組み）が登場する
- 情報システム（基本的には情報を操作する仕組み）が普及する
- セキュリティ（攻撃者や災害などの外的要因から情報を守る仕組み）が重要になる

# 2025年度 コンピュータシステム

## 第1回：計算機とプログラムによる自動化

## 1. コンピュータとは何か

- 自動で計算する道具（計算機）
- 計算機が様々な情報処理を行う仕組み

## 2. 計算の自動化

- 計算機とプログラム
- 計算の組み合わせで作られる機能・サービス

## 3. 情報通信技術と社会の情報化

- インターネット・IoTによる情報の集積
- 情報を分析・活用する技術の発達
- 情報社会の問題解決

## 1. コンピュータとは何か

- 自動で計算する道具（計算機）
- 計算機が様々な情報処理を行う仕組み

## 2. 計算の自動化

- 計算機とプログラム
- 計算の組み合わせで作られる機能・サービス

## 3. 情報通信技術と社会の情報化

- インターネット・IoTによる情報の集積
- 情報を分析・活用する技術の発達
- 情報社会の問題解決



パソコン



スパコン・データセンター



ゲーム機



スマートフォン



通信機器

## コンピュータ（計算機）

Compute + er : 計算する人・もの

## 計算

情報を決められた方法で処理すること(演算), その結果を求めること(算出)

例1) 四則演算

- $3+5=8$ ,  $10-3=7$ ,  $4\times 8=32$ ,  $7\div 2=3.5$  (3あまり1)

例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

## コンピュータ（計算機）

- Compute + er : 計算する人・もの

## 計算

- 情報を決められた方法で処理すること(演算), その結果を求めること(算出)

例1) 四則演算

- $3+5=8$ ,  $10-3=7$ ,  $4\times 8=32$ ,  $7\div 2=3.5$  (3あまり1)

例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

## “コンピュータ”のイメージとの乖離

- Wordでレポートを作る
- Amazonで買い物をする
- YouTubeで動画を見る



## 素朴な“計算”のイメージ

例1) 四則演算

$$3+5=8, \quad 10-3=7, \quad 4\times 8=32, \quad 7\div 2=3.5 \quad (3\text{あまり}1)$$

例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$



## 素朴な“コンピュータ”のイメージ

Wordでレポートを作る  
Amazonで買い物をする  
YouTubeで動画を見る

## 素朴な“計算”のイメージ

### 例1) 四則演算

$$3+5=8, \quad 10-3=7, \quad 4\times 8=32, \quad 7\div 2=3.5 \quad (3\text{あまり}1)$$

### 例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$



## 素朴な“コンピュータ”のイメージ

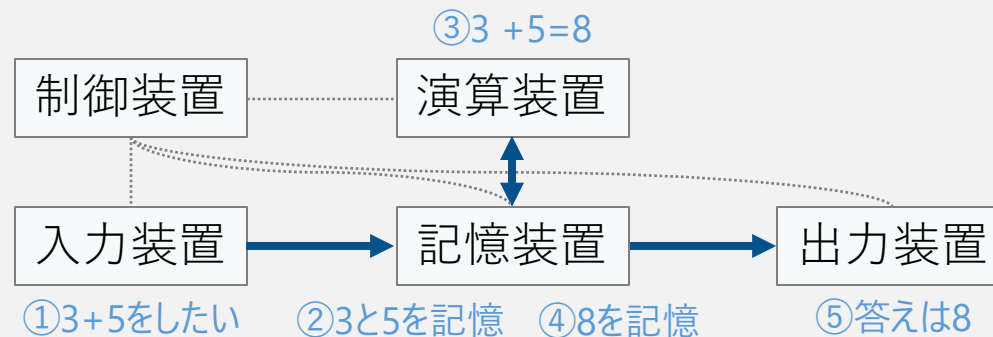
Wordでレポートを作る  
Amazonで買い物をする  
YouTubeで動画を見る

文字や画像などあらゆる情報を数値として表すことで  
様々な機能を計算問題に表現できる

計算機の仕組みを理解すれば、この説明の意味が理解できる

自動で計算をする装置は次の仕組みで実現可能

- 入力装置：数値や計算方法を入力する装置
- 出力装置：計算結果を出力する装置
- 演算装置：足し算や掛け算などを行う回路
- 制御装置：計算を行うために装置同士を連携させる装置



この装置があれば、様々な仕事を自動化できる

自動で計算をする装置は図に示すようなハードウェア構成で実現できる

まず、計算をするは入力装置が必要。例えば、キーボードのようなもの

そして、結果を出力する装置が必要。例えば、ディスプレイのようなもの

当然、数値同士の足し算や掛け算など、演算を行う装置が必要。

また、筆算をするときに書いておかないと分からなくなるように、計算過程の数値を覚えておくためのメモのような記憶装置が必要

最後に、これらがバラバラにあっても自動で計算ができることにはならないので、これら4つの装置を連携させて動作させるための制御装置が必要

これら5つの装置があれば、「計算」を自動実行する機械を作ることができる

例えば、 $3+5$ という演算を考える

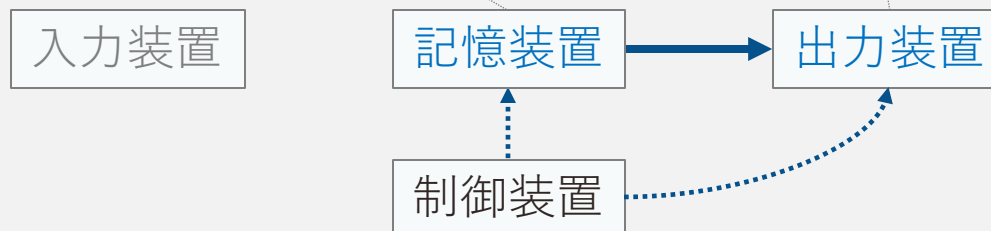
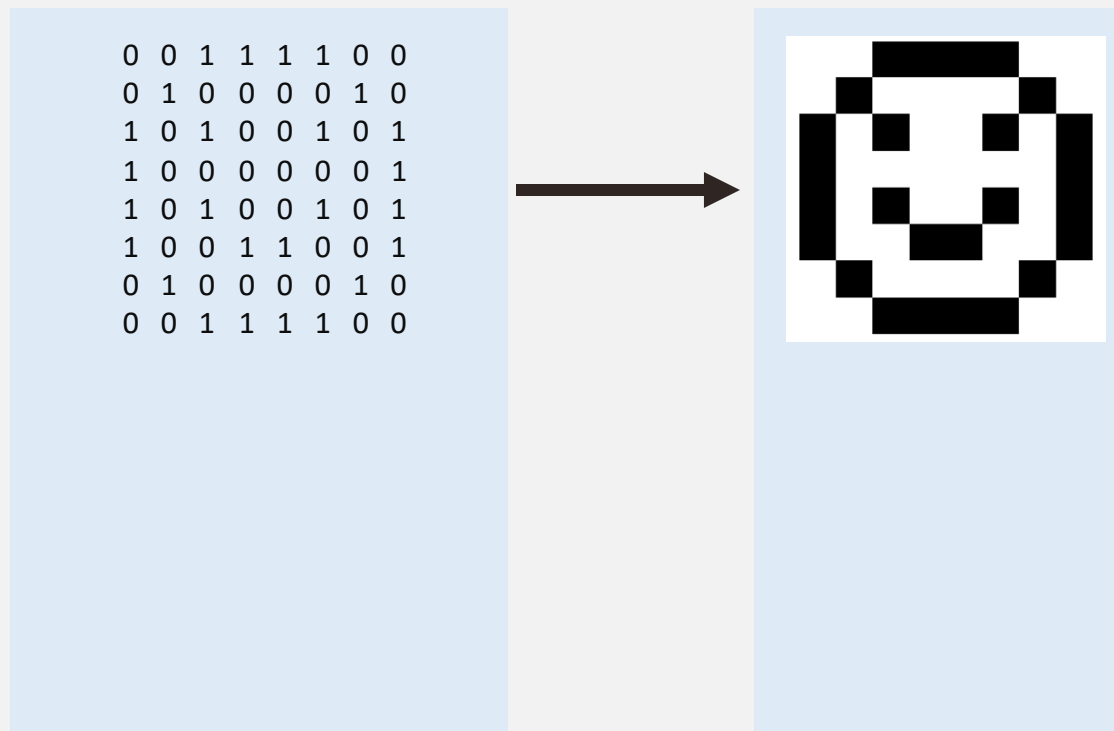
1. 入力装置で数値と行いたい演算を入力する
2. 記憶装置に入力情報が記憶される
3. 記憶した情報に基づいて、演算装置で足し算されて、その結果が導出される
4. 導出された結果は記憶装置に記憶される
5. 最後に、その結果がディスプレイから出力される

制御装置はこれら一連の動作を自動で取り持ってくれる

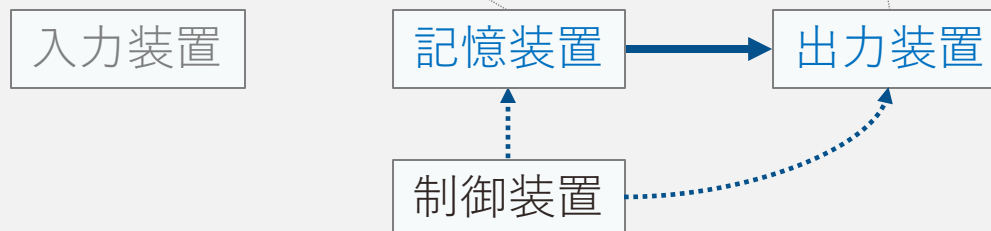
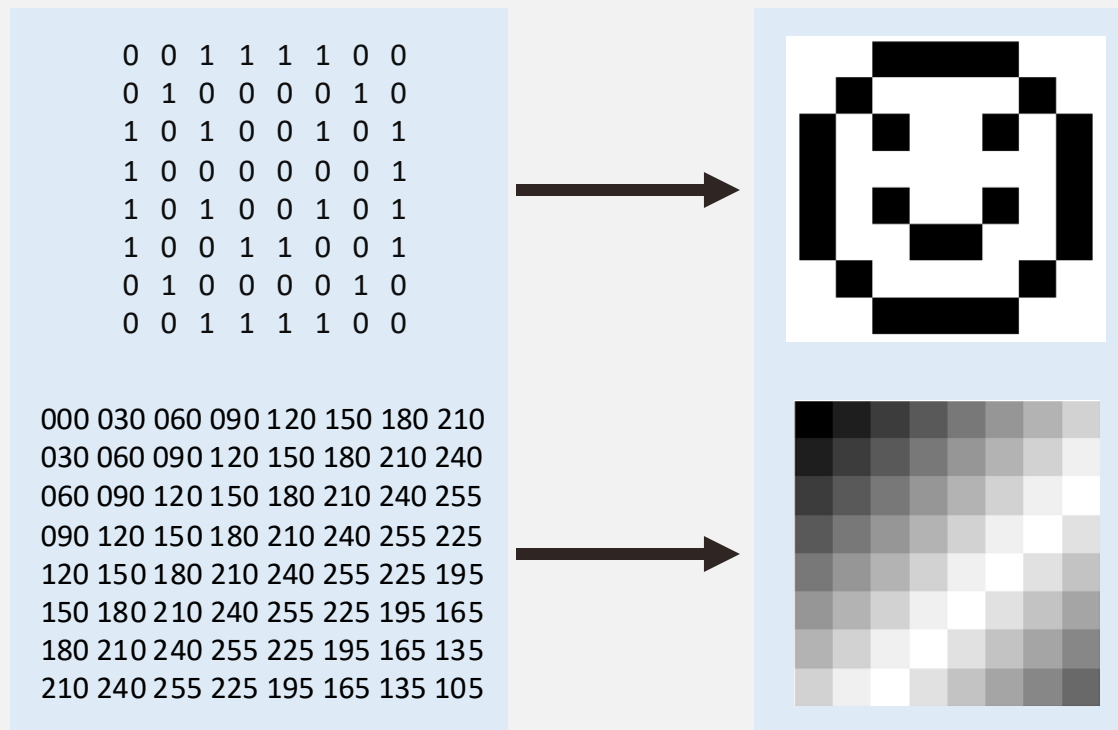
このように、5つの装置があれば計算は人間が介在することなく自動で実行される。

これまでをまとめると、計算機とはこのようなもので、「自動で計算する機械」である。そして、これがあれば色々な仕事ができる

画像をたくさんの数値が縦横に並んだ情報として出力する



画像をたくさんの数値が縦横に並んだ情報として出力する



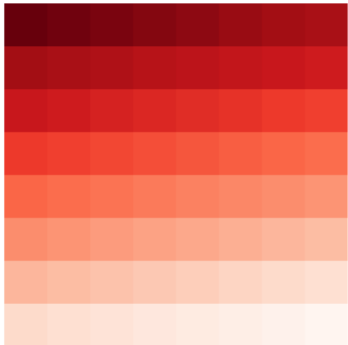
一つの画素に赤緑青の3種類の明るさ情報を持たせる

- 赤青緑を混ぜ合わせると、その割合により様々な色ができる

255 250 245 240 235 230 225 220  
225 220 215 210 205 200 195 190  
195 190 185 180 175 170 165 160  
165 160 155 150 145 140 135 130  
135 130 125 120 115 110 105 100  
105 100 095 090 085 080 075 070  
075 070 065 060 055 050 045 040  
045 040 035 030 025 020 015 010



Red



000 000 000 000 000 000 000 000  
036 036 036 036 036 036 036 036  
072 072 072 072 072 072 072 072  
109 109 109 109 109 109 109 109  
145 145 145 145 145 145 145 145  
182 182 182 182 182 182 182 182  
218 218 218 218 218 218 218 218  
255 255 255 255 255 255 255 255



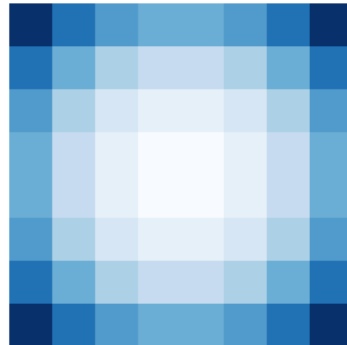
Green



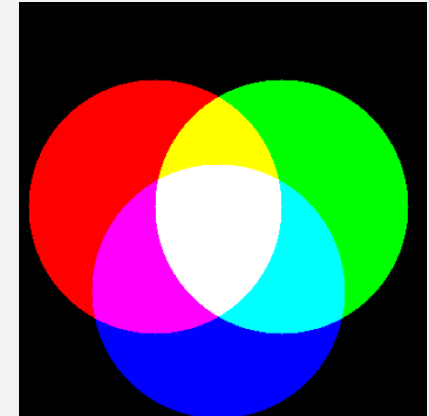
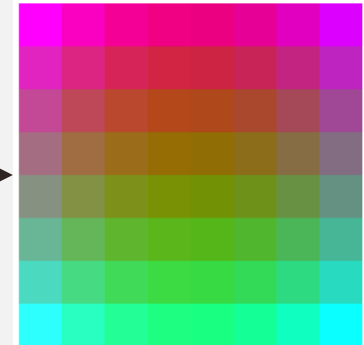
255 192 150 130 130 150 192 255  
192 130 088 067 067 088 130 192  
150 088 046 026 026 046 088 150  
130 067 026 005 005 026 067 130  
130 067 026 005 005 026 067 130  
150 088 046 026 026 046 088 150  
192 130 088 067 067 088 130 192  
255 192 150 130 130 150 192 255



Blue

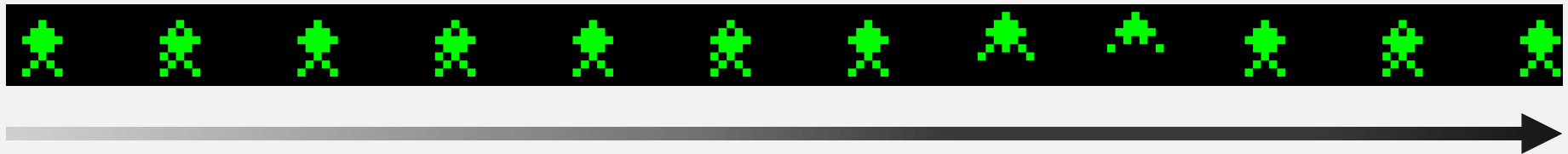
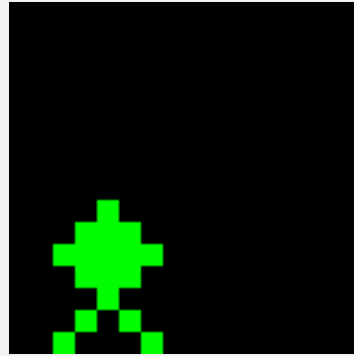


Color



光の三原色

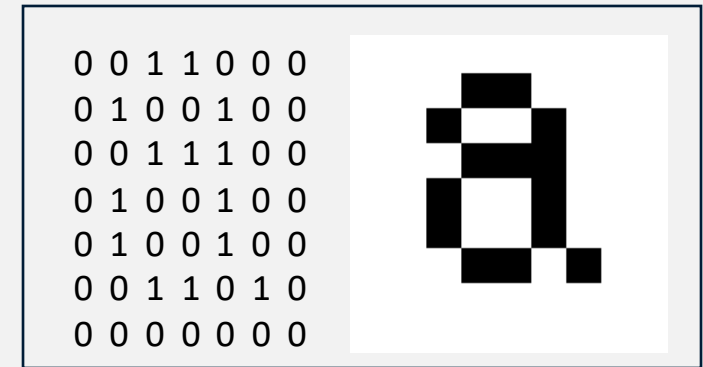
画像を素早く切り替えて出力する



## 文字を数値として記憶して，画像として出力する

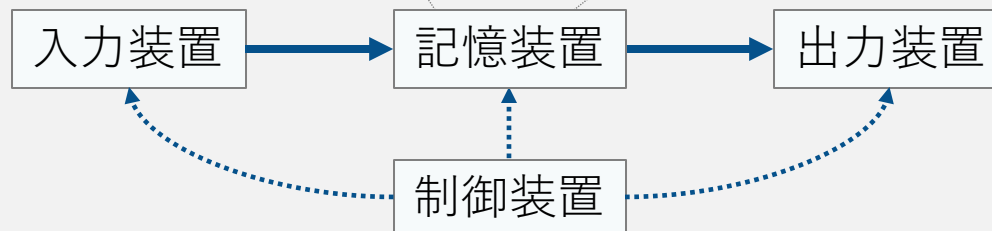
| 文字 | コード | 文字 | コード | 文字 | コード | 文字 | コード |
|----|-----|----|-----|----|-----|----|-----|
| a  | 97  | b  | 98  | c  | 99  | d  | 100 |
| e  | 101 | f  | 102 | g  | 103 | h  | 104 |
| i  | 105 | j  | 106 | k  | 107 | l  | 108 |
| m  | 109 | n  | 110 | o  | 111 | p  | 112 |
| q  | 113 | r  | 114 | s  | 115 | t  | 116 |
| u  | 117 | v  | 118 | w  | 119 | x  | 120 |
| y  | 121 | z  | 122 |    |     |    |     |

ASCIIコード



フォント  
(ビットマップフォント)

97

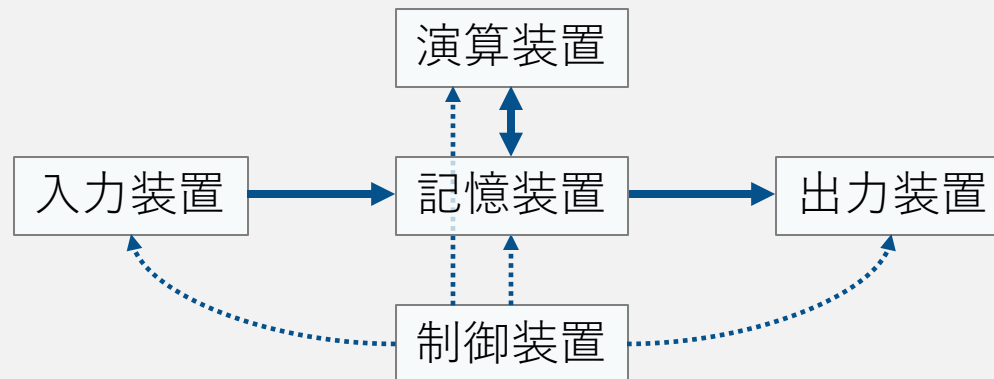


入力を数値として，処理を計算として，出力を画像として扱う

1. コントローラからキー入力を受け取る
2. キー入力に沿った物理演算を計算

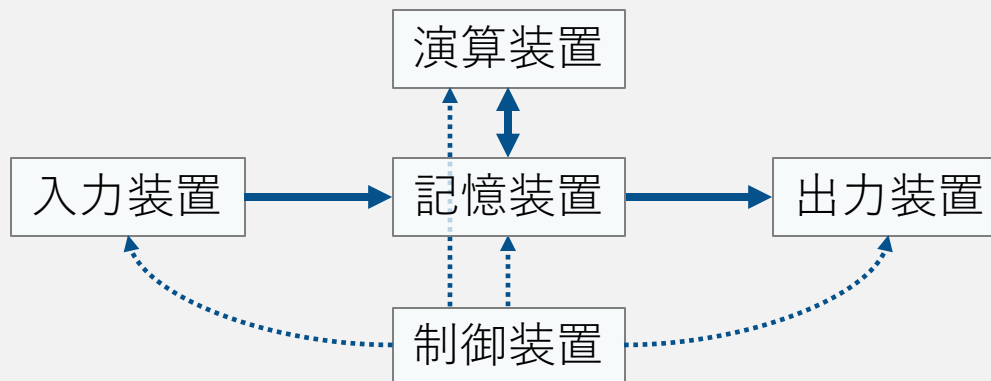
例：ジャンプコマンドの計算

1. キャラクターに上向きの力を加えて，ジャンプアニメーションを動かす
2. 重力に従った計算をする： $y(t) = y_o + v_o t - \frac{1}{2} g t^2$
3. キャラクターの位置を $y(t)$ に合わせて変える
4. 映像をディスプレイに出力する



## 帳簿情報を表形式で記録，取り出し，編集する

| 日付         | 摘要       | 借方科目 | 借方金額   | 貸方科目 | 貸方金額   |
|------------|----------|------|--------|------|--------|
| 2025/03/01 | 商品を仕入れた  | 仕入   | 50,000 | 現金   | 50,000 |
| 2025/03/02 | 商品を現金で販売 | 現金   | 80,000 | 売上   | 80,000 |
| 2025/03/03 | 事務用品を購入  | 消耗品費 | 5,000  | 現金   | 5,000  |
| 2025/03/04 | 家賃を支払った  | 地代家賃 | 60,000 | 現金   | 60,000 |



## ◎ 数値予報とは

スーパーコンピュータを用いて、未来の大気状態をシミュレーションすることを**数値予報**と呼ぶんだ。  
たとえば、天気予報には、数値予報による未来の大気状態の予測結果が使われているんだよ。



### 天気予報ができるまで

#### ①観測データを収集



#### ③天気予報を作成

予報を検討、  
発表する予報官



#### ②スーパーコンピュータで未来の気象を予測

#### 数値予報



予報官は、数値予報による未来の大気状態の予測結果などをもとに、きめ細やかな天気予報を発表しているんだ。  
この他にも、航空機や船舶の運航など、様々な分野において、数値予報の予測結果が役立てられているよ。



毎日の天気予報は最新の科学技術に支えられているのね！

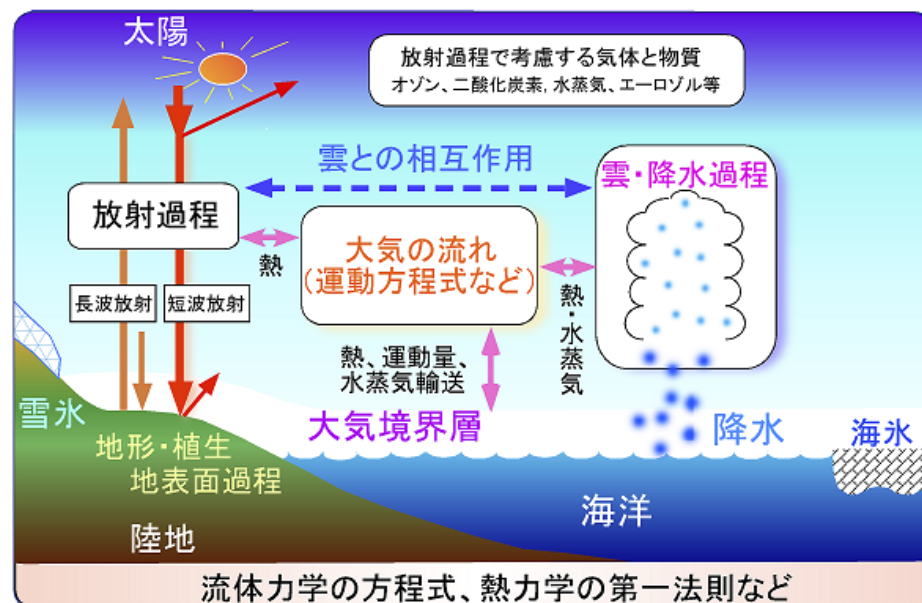
### コラム① 数値予報の歴史

数値予報の歴史は、コンピュータ実用化以前の1920年頃に、大気状態を記述する方程式を人の手で計算する試みが行なわれたことにはじまります。気象庁では、1959年にわが国の官公庁として初めて大型コンピュータを導入して数値予報業務を開始し、今日では予報業務の根幹を担っています。

写真：数値予報業務開始当時の大型コンピュータ



数値予報モデルでは、大気の流れ（風）をはじめ、水蒸気が凝結して雨が降ること、地面が太陽に暖められたり冷やされたりすることなど、さまざまな現象が考慮されています。



## 気象予報の計算

1. 「気象の時間変化」を計算式に記述
2. 「現在の気象」として式に観測データを代入
3. 「未来の気象状況」を計算
4. 雨量予測図など人間に分かりやすい形で出力

出典：気象庁ホームページ

(公共データ利用規約 (第1.0版) に従って引用)

左：<https://www.jma.go.jp/jma/kishou/books/nwpleaflet/index.html>

右：<https://www.jma.go.jp/jma/kishou/know/whitep/1-3-1.html>

## 素朴な“計算”のイメージ

### 例1) 四則演算

$$3+5=8, \quad 10-3=7, \quad 4\times 8=32, \quad 7\div 2=3.5 \quad (3\text{あまり}1)$$

### 例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$



## 素朴な“コンピュータ”のイメージ

Wordでレポートを作る  
Amazonで買い物をする  
YouTubeで動画を見る

文字や画像などあらゆる情報を数値として表すことで  
様々な機能を計算問題に表現できる

## 1. コンピュータとは何か

- 自動で計算する道具（計算機）
- 計算機が様々な情報処理を行う仕組み

## 2. 計算の自動化

- 計算機とプログラム
- 計算の組み合わせで作られる機能・サービス

## 3. 情報通信技術と社会の情報化

- インターネット・IoTによる情報の集積
- 情報を分析・活用する技術の発達
- 情報社会の問題解決

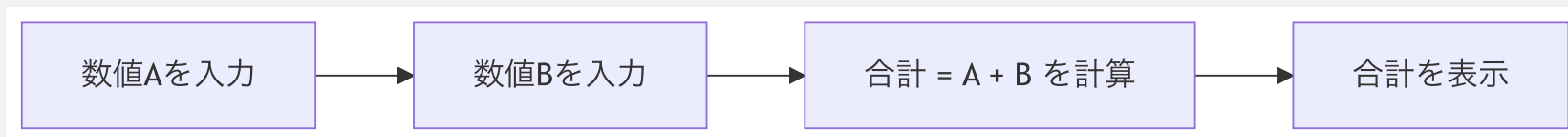
## 計算機は計算を自動で行う機械

→ 計算の手順として書かれていれば様々な情報処理を自動化できる

## プログラム (program)

- Pro + gram : あらかじめ + 書かれたもの
  - 例1) 四則演算
  - 例2) フローチャート
  - 例3) アルゴリズム, プログラミング言語で書かれたソースコード

```
a = int(input("数値Aを入力: "))  
b = int(input("数値Bを入力: "))  
total = a + b  
print("合計は", total)
```



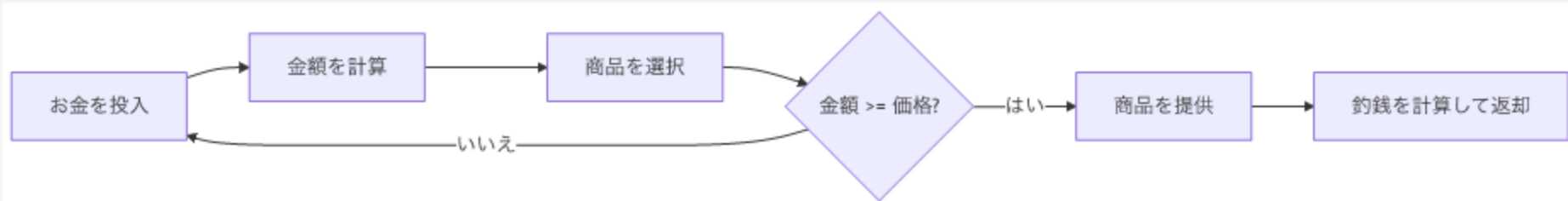
## 例1：Word

1. キーボードで入力された文字を数列として記憶
2. カーソルの場所（何行目の何文字目）を記憶
3. 新しい文字が入力されたら、その位置に「文字を表す数値」を挿入
4. 「文字を表す数値の列」を「対応するフォント」に置き換えて表示

## 例2：YouTube

1. 動画が投稿されたら識別番号（ID）を割り振り、動画データを紐付ける
2. アプリは指定された動画に対応するIDをサーバーにリクエスト
3. サーバーはIDに対応するデータをアプリに送信
4. アプリはデータを動画としてディスプレイに表示

## 例3：自動販売機



## コンピュータとプログラム

コンピュータは自動で“計算”を行う装置

→ 計算の仕方を書いておけば情報処理を自動化できる

プログラムは計算手順を予め書いたもの

## 情報通信技術と社会の情報化

インターネット・IoTは物理・仮想世界で膨大な情報を集積可能にする技術

→ 情報を分析・活用する技術（データ科学やAI）が発展する

→ 現実問題を「情報世界の問題」に変換すれば、情報技術の力を借りて解決できる

Industry4.0・Society5.0は情報を集積・分析して産業・社会を最適化する構想

→ 産業・社会での情報技術活用を推進するため、国民全員が情報を学ぶ方針になる

## 1. コンピュータとは何か

- 自動で計算する道具（計算機）
- 計算機が様々な情報処理を行う仕組み

## 2. 計算の自動化

- 計算機とプログラム
- 計算の組み合わせで作られる機能・サービス

## 3. 情報通信技術と社会の情報化

- インターネット・IoTによる情報の集積
- 情報を分析・活用する技術の発達
- 情報社会の問題解決

## 通信：情報をやり取りする仕組み

例) 狼煙, 音声会話, 手紙, モールス信号機, 電話, インターネット

## インターネット：世界中の計算機が互いに情報をやり取りできる仕組み

インターネットを基盤として様々なサービスが発達

例) SNS, 動画配信, 通販, 銀行, データ管理, ビデオ会議

## IoT (Internet of Things)：もののインターネット

インターネットを介して相互に情報をやり取りする物理的な機器や装置

例) スマホ, スマート家電, 自動運転車, 残量通知陳列棚

インターネットにより仮想世界+現実世界の  
膨大なデータが収集可能になった

## 膨大なデータを前提とした技術・学問の発展

### データ科学：大量のデータから有益な知見を導き出す科学的な手法

- 数学，統計学，計算機科学，情報学など様々な知識を活用して，

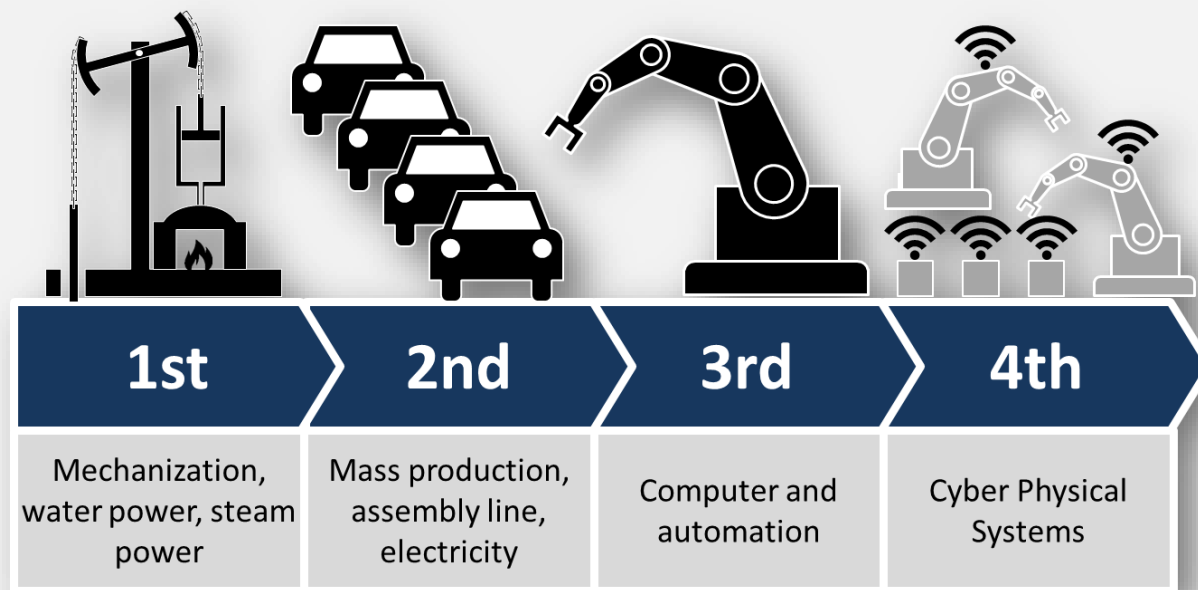
### 人工知能：知的な振る舞いを計算機で再現する技術

- 第一次AIブーム：論理的な推論に基づくゲームや迷路の攻略
- 第二次AIブーム：整備された専門知識に基づくクイズやカスタマーサポート
- 第三次AIブーム：ビッグデータによる最適化に基づく画像認識や対話

膨大なデータを分析・整理する方法が整備され  
社会への活用が進められている

## データ・コンピュータによる製造工程全体の最適化

- 水力・蒸気機関による**動力の獲得**（1.0）石油・電力による**大量生産**（2.0）  
計算機・情報技術を活用した**機械操作の自動化**（3.0）に続く製造業の変革
- **仮想世界での分析により，生産・流通を含めた全工程を自動的に調整**



ChristophRoser. Please credit "Christoph Roser at AllAboutLean.com". - 投稿者自身による著作物,  
CC 表示-継承 4.0, <https://commons.wikimedia.org/w/index.php?curid=47640595> による

参考資料：総務省 | 平成30年版 情報通信白書 | インダストリー4.0とは

<https://www.soumu.go.jp/johotsusintokei/whitepaper/ja/h30/html/nd135210.html> , 2025/04/01アクセス

## TOTO滋賀工場の歩留まり改善サイクル

1. 衛生陶器工場の設備自動化に際して**データ自動取得の仕組み**を導入
  - ・ 製造過程の温度、湿度、圧力、原材料の状態、運搬ルート、時刻など、多岐にわたるデータを自動的に取得
2. データ分析により**不良の出やすい条件**や**安定する条件**を解明
3. 分析結果に基づいて状況を**リアルタイムに取得・可視化**して、生産技術や現場へ**即座にフィードバック**する仕組みを構築
4. 成功を受けて他の工場へ横展開
  - ・ 工場を視察して少しでも多くのデータが取れる状況づくりを思案
    - ・ 紙面データや扱いにくいデータを手動で整理
    - ・ 紙入力をタブレット入力に変更
    - ・ センサーの取り付け など

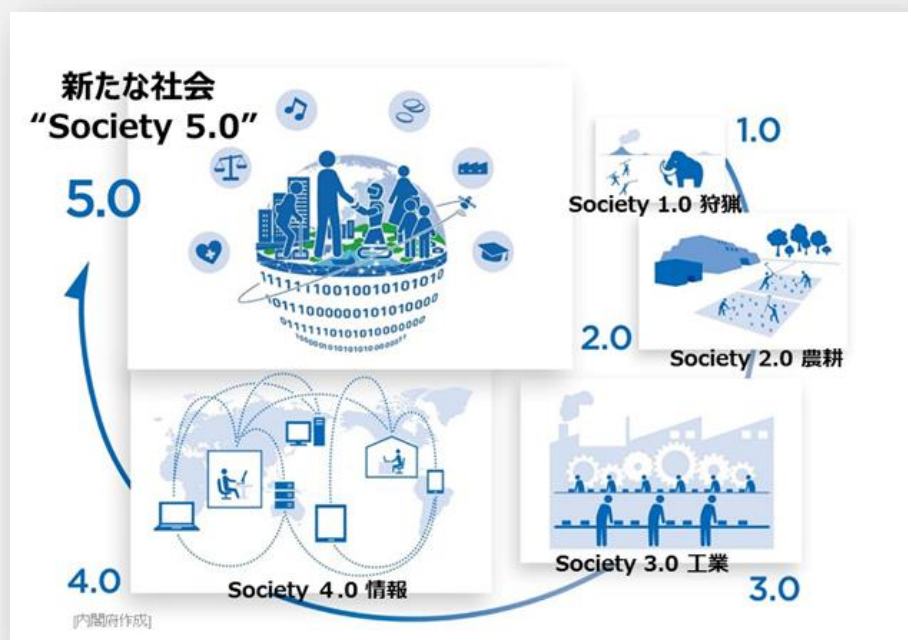
### 情報源：

- [1] 成功のカギは「良品と均質」を追求する理念の浸透 創立107年のTOTOが挑むデジタル活用 | Koto Online  
[https://www.cct-inc.co.jp/koto-online/archives/412?utm\\_source=chatgpt.com](https://www.cct-inc.co.jp/koto-online/archives/412?utm_source=chatgpt.com) 2025/04/18 アクセス
- [2] グリーンファクトリー滋賀工場 | サステナビリティ | 会社情報 | TOTO株式会社  
[https://jp.toto.com/company/csr/environment/greenfactory/?utm\\_source=chatgpt.com](https://jp.toto.com/company/csr/environment/greenfactory/?utm_source=chatgpt.com) 4/18アクセス

## 内閣府が掲げる「我が国の目指すべき未来社会の姿」

### 第四次産業革命を「製造業」から「社会全体」に拡張したようなもの

- 社会全体を**仮想世界で分析して**，社会制度やビジネスデザイン，都市や地域の整備などを再構成することで，**現実社会を変革**していく
- 技術革新や人材育成，研究開発などの**政策を通じて実現**が目指される



参考資料より「公共データ利用規約（第1.0版）」に基づいて引用

参考資料：

Society 5.0 - 科学技術政策 - 内閣府 [https://www8.cao.go.jp/cstp/society5\\_0/](https://www8.cao.go.jp/cstp/society5_0/)， 2025/04/01アクセス

## 複数社にまたがるデータ分析体制を作り，社会全体を改善した事例

### 複数の会社・研究室が協力して行った実証実験

- ・ 佐川急便，JDSC，東京大学大学院越塚登研究室・田中謙司研究室，横須賀市，グリッドデータバンク・ラボ有限責任事業組合

### 150世帯に配布したスマートメーター（遠隔で情報取得できる計器）を利用

1. スマートメータが消費電力を記録
2. ネットワークを経由してサーバにアップロード
3. 消費電力のパターンを元に在宅か不在かを推定
4. 在宅確率が高い時間帯に訪問するように，再送先経路をリアルタイムに変更

結果：不在率が20%改善

これらに共通している方向性としては、情報世界を介して問題を解決するという発想である。

この考え方が重要である理由は、計算機という「自動で計算する装置」の存在に由来する。

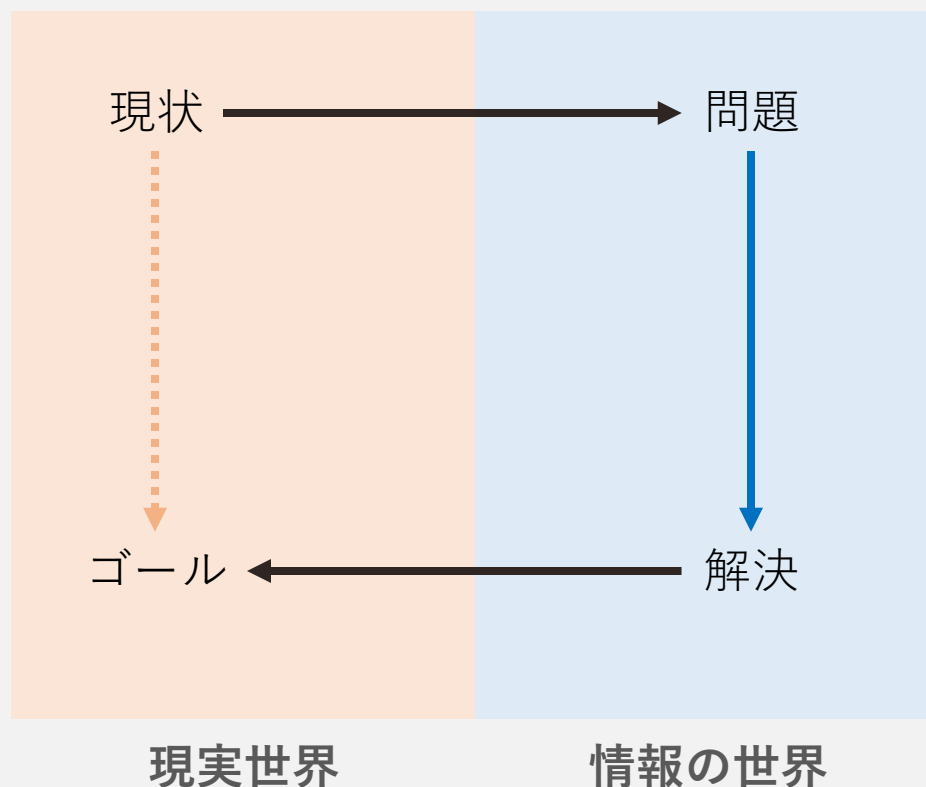
「現実世界の仕事」を「情報世界の問題解決」に整理してしまえば、人間が手を煩わせることなく、エネルギーと機械により自動的に遂行されるようになる。

これは、石油 + エンジン + 重機が発明されたことで、「力仕事の一部が人の手から離れた」というのと類似したイメージ（アナロジー）を考えると分かりやすいかもしれない

生活者の視点に立つなら、電気+家電が発明されたことで、「家事が形を変えながら人の手から離れた」というのと類似したアナロジーで考えると想像しやすいかもしれない。

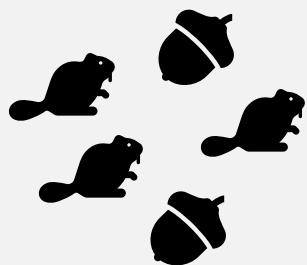
## 情報世界を介して現実世界の問題を解決する

1. 現実世界の問題を**情報世界の問題**に変換
2. 情報処理の**知識や道具を活用**して情報世界で問題を解決
3. 情報世界で解決した結果を**現実世界の結果**に変換

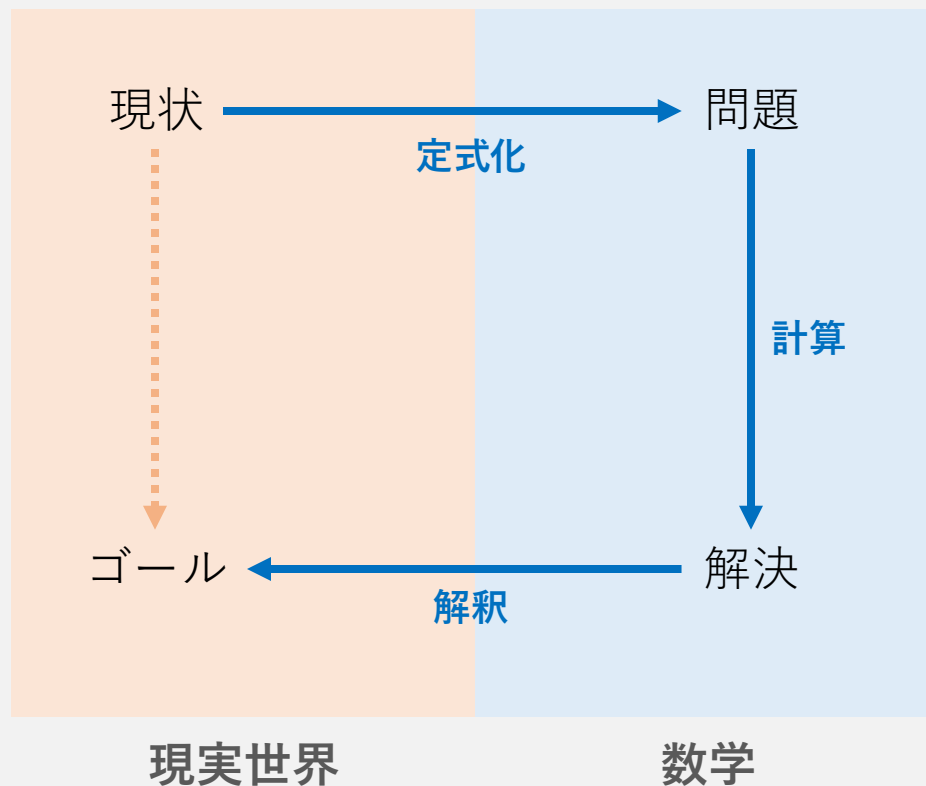


## 数学世界を介して現実世界の問題を解決する

1. 現実世界の問題を**数学の問題**に変換
2. 数学の**知識や道具**を活用して情報世界で問題を解決
3. 数学で得られた結果を**現実世界の結果**に変換



どんぐりは  
1つ足りない



3-2=?

回答：1

イメージが難しければ、皆さんの多くも「数学」で似たようなことをしているので、そのアナロジーが分かりやすいかもしれない

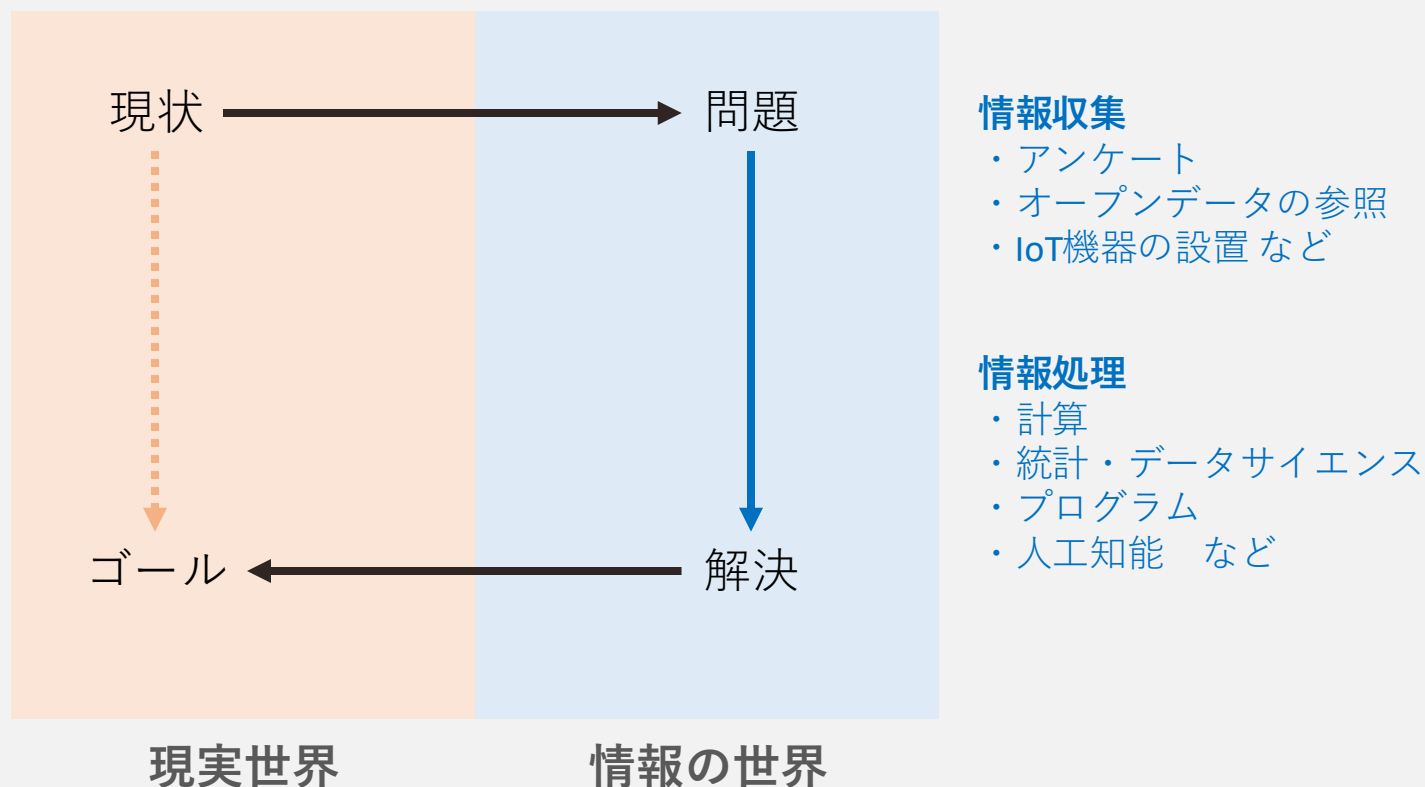
例えば、買い物をするときに、物の価格と手持ちの金額を計算するとき、当然のように足し算をすると思う。

これは、現実世界の問題（手持ちのお金を、商品の金額の関係と比較する）を「数学世界の問題」に変換して、数学の道具を使って解決していると説明できる

これと同様に、「情報世界の問題解決」では、「現実世界の問題」を「情報処理の問題」に変換して、情報処理のための道具（理論的なものから、計算機のような装置を含む）を使って問題を解決する

## 情報世界を介して現実世界の問題を解決する

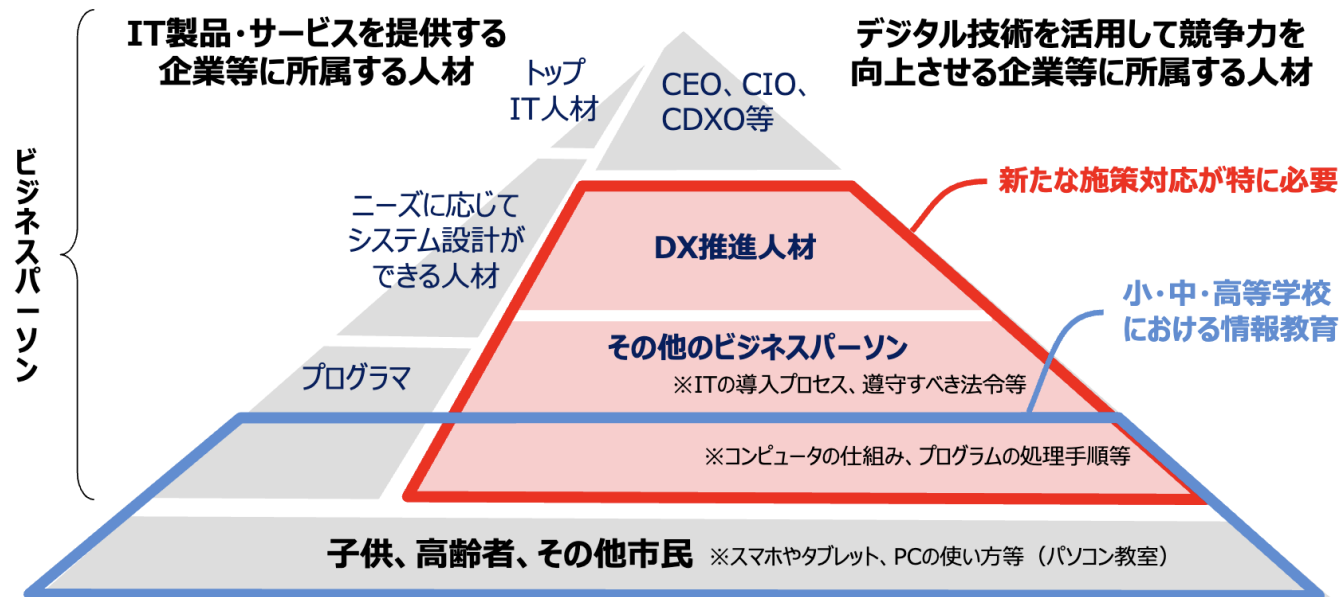
1. 現実世界の問題を**情報世界の問題**に変換
2. 情報処理の**知識や道具を活用**して情報世界で問題を解決
3. 情報世界で解決した結果を**現実世界の結果**に変換



「コンピュータの仕組み、プログラムの処理手順等」は  
「**小・中・高等学校**」と「**その他のビジネスパーソン**」の重複領域に存在

## デジタル社会における人材像

- デジタル社会においては、全ての国民が、役割に応じた相応のデジタル知識・能力を習得する必要がある。
- 若年層は、小・中・高等学校の情報教育を通じて一定レベルの知識を習得する。現役のビジネスパーソンの学び直し（＝リスキリング）が重要。



11

## ITパスポート試験

- ・ IT化された社会で働くすべての人に必要な基礎知識の証明

| 分野      | 大分類 |              | 中分類 |                |
|---------|-----|--------------|-----|----------------|
| ストラテジ系  | 1   | 企業と法務        | 1   | 企業活動           |
|         |     |              | 2   | 法務             |
|         | 2   | 経営戦略         | 3   | 経営戦略マネジメント     |
|         |     |              | 4   | 技術戦略マネジメント     |
|         |     |              | 5   | ビジネスインダストリ     |
|         | 3   | システム戦略       | 6   | システム戦略         |
|         |     |              | 7   | システム企画         |
| マネジメント系 | 4   | 開発技術         | 8   | システム開発技術       |
|         |     |              | 9   | ソフトウェア開発管理技術   |
|         | 5   | プロジェクトマネジメント | 10  | プロジェクトマネジメント   |
|         | 6   | サービスマネジメント   | 11  | サービスマネジメント     |
|         |     |              | 12  | システム監査         |
| テクノロジー系 | 7   | 基礎理論         | 13  | 基礎理論           |
|         |     |              | 14  | アルゴリズムとプログラミング |
|         | 8   | コンピュータシステム   | 15  | コンピュータ構成要素     |
|         |     |              | 16  | システム構成要素       |
|         |     |              | 17  | ソフトウェア         |
|         |     |              | 18  | ハードウェア         |
|         | 9   | 技術要素         | 19  | 情報デザイン         |
|         |     |              | 20  | 情報メディア         |
|         |     |              | 21  | データベース         |
|         |     |              | 22  | ネットワーク         |
|         |     |              | 23  | セキュリティ         |

出典：【ITパスポート試験】試験内容・出題範囲  
<https://www3.jitec.ipa.go.jp/JitesCbt/html/about/range.html>

## ITパスポート試験

- ・ IT化された社会で働くすべての人に必要な基礎知識の証明

## 基本情報技術者試験（FE），応用情報技術者試験（AP）

- ・ FE：ITエンジニアとしてのキャリアを始めるにあたり基礎的な内容
- ・ AP：ITシステム開発に重要な技術から管理，経営までの幅広い知識

### 3. 期待する技術水準

---

ITを活用した戦略の立案、システムの企画・要件定義、設計・開発・運用に関し、担当する活動に応じて次の知識・技能が要求される。

1. IT全般に関する基本的な事項を理解し、担当する活動に活用できる。
2. 上位者の指導の下に、IT戦略に関する予測・分析・評価に参加できる。
3. 上位者の指導の下に、システム又はサービスの提案活動に参加できる。
4. 上位者の指導の下に、システムの企画・要件定義に参加できる。
5. 上位者の指導の下に、情報セキュリティの確保を考慮して、システムの設計・開発・運用ができる。
6. 上位者の指導の下に、ソフトウェアを設計できる。
7. 上位者の方針を理解し、自らプログラムを作成できる。

出典：基本情報技術者試験 | 試験情報 | IPA 独立行政法人 情報処理推進機構

<https://www.ipa.go.jp/shiken/kubun/fe.html>

## コンピュータとプログラム

コンピュータは自動で“計算”を行う装置

→ 計算の仕方を書いておけば情報処理を自動化できる

プログラムは計算手順を予め書いたもの

## 情報通信技術と社会の情報化

インターネット・IoTは物理・仮想世界で膨大な情報を集積可能にする技術

→ 情報を分析・活用する技術（データ科学やAI）が発展する

→ 現実問題を「情報世界の問題」に変換すれば、情報技術の力を借りて解決できる

Industry4.0・Society5.0は情報を集積・分析して産業・社会を最適化する構想

→ 産業・社会での情報技術活用を推進するため、国民全員が情報を学ぶ方針になる

# 2025年度 コンピュータシステム

## 第2回：計算機，インターネット，情報システム

## 計算機やインターネットの仕組み

計算機内部の情報表現はデジタル（数字式）の二進表現（0 or 1）

- 物理量（アナログ値）を数字式に近似すると記録・伝達しやすくなる
- 数値を0と1のみで表現すると、単純な“スイッチ”（半導体）で実現できる

ネットワークは網目状に結ばれた計算機同士の通信経路

- インターネット（世界中のネットワーク同士が相互に通信できる仕組み）が登場する
- 情報システム（基本的には情報を操作する仕組み）が普及する
- セキュリティ（攻撃者や災害などの外的要因から情報を守る仕組み）が重要になる

## 1. 計算機の仕組み

- 計算機内の情報表現
- 計算機のハードウェア
- 人間が扱うための仕組み

## 2. ネットワークの仕組み

- 計算機同士の情報交換
- インターネット

## 3. 情報システムの仕組み

- 情報システムとデータベース
- 情報セキュリティ

## 1. 計算機の仕組み

- 計算機内の情報表現
- 計算機のハードウェア
- 人間が扱うための仕組み

## 2. ネットワークの仕組み

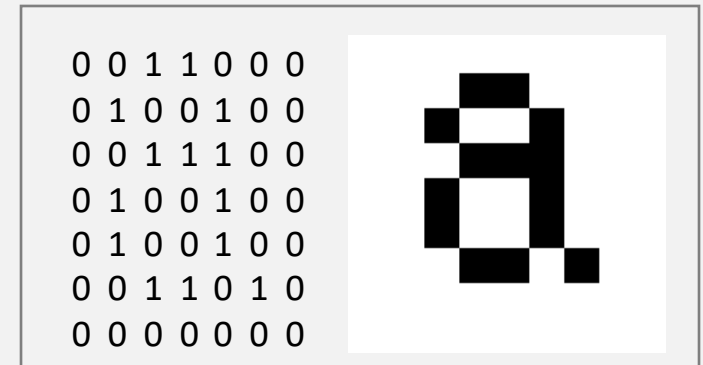
- 計算機同士の情報交換
- インターネット

## 3. 情報システムの仕組み

- 情報システムとデータベース
- 情報セキュリティ

文字，画像，音声など様々な情報はすべて数値として表現される

| 文字 | コード | 文字 | コード | 文字 | コード | 文字 | コード |
|----|-----|----|-----|----|-----|----|-----|
| a  | 97  | b  | 98  | c  | 99  | d  | 100 |
| e  | 101 | f  | 102 | g  | 103 | h  | 104 |
| i  | 105 | j  | 106 | k  | 107 | l  | 108 |
| m  | 109 | n  | 110 | o  | 111 | p  | 112 |
| q  | 113 | r  | 114 | s  | 115 | t  | 116 |
| u  | 117 | v  | 118 | w  | 119 | x  | 120 |
| y  | 121 | z  | 122 |    |     |    |     |

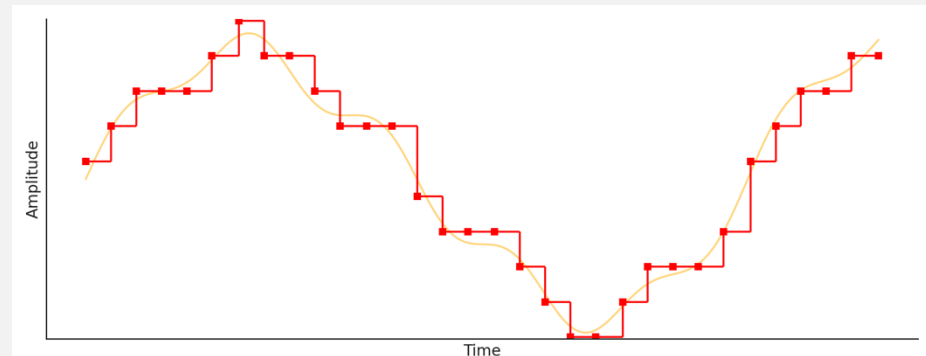


文字コードとフォント

000 030 060 090 120 150 180 210  
030 060 090 120 150 180 210 240  
060 090 120 150 180 210 240 255  
090 120 150 180 210 240 255 225  
120 150 180 210 240 255 225 195  
150 180 210 240 255 225 195 165  
180 210 240 255 225 195 165 135  
210 240 255 225 195 165 135 105



画像データ



音声波形の数値化  
(後で説明)

## ハードウェアの中で物理的に表現される

計算機的设计によって色々な表現が可能

例：ENIACは10進数，アナログ時計は12進数と60進数の併用



そろばん：二・五進法

(+5で一段階進み，2段階進むと桁が上がる)



スイッチ：二進数

(0と1の二つの状態のみ表せる)

現在の電子計算機では、2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで記述される



## ハードウェアの中で物理的に表現される

(計算機的设计によって色々な表現が可能)

例：ENIACは10進数，アナログ時計は12進数と60進数の併用

現在の電子計算機では，2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで表現される



デジタルとは何か？

2進数とは何か？

パソコンやスマホ，ゲーム機，電卓などは全て「デジタル」な計算機

## デジタル：数字式

- digit + al：数字の+方式
- 物理量を**数字**で記述
  - 結び目の数，デジタル温度計，チャットなど

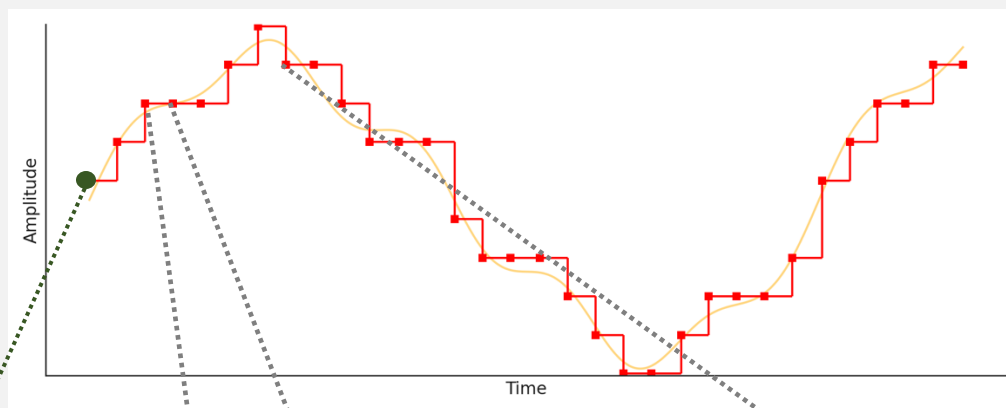
## アナログ：相似式

- analog：語源はanalogy（類似・相似）
- 物理量を別の**物理量との対応**で表現
  - 紐の長さ，水銀温度計，糸電話など

**デジタルは情報を完全に表現できないが，ノイズに強くて伝達しやすい**

- 量を正しく表すには無限桁の数値が必要
- 数値は量よりも“伝達ミス”が起きにくい
  - 例1) 水銀温度計の結果を“正しく”伝えることは難しいが，数値を伝えるのは簡単
  - 例2) 紐の長さを正確に伝えることは難しいが，結び目の数を伝えるのは簡単

縦横に適当な間隔で区切り，棒状にすることで波形を数列に変換



| Sample 1 | Sample 2 | Sample 3 | Sample 4 | Sample 5 | Sample 6 | Sample 7 | Sample 8 | Sample 9 |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 0.143    | 0.429    | 0.714    | 0.714    | 0.714    | 1        | 1.286    | 1        | 1        |

## ハードウェアの中で物理的に表現される

(計算機的设计によって色々な表現が可能)

例：ENIACは10進数，アナログ時計は12進数と60進数の併用

現在の電子計算機では，2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで表現される



デジタル：情報を数値で表現すること

2進数とは何か？

通常の計算機は情報（=数値）を"0"と"1"で表現する

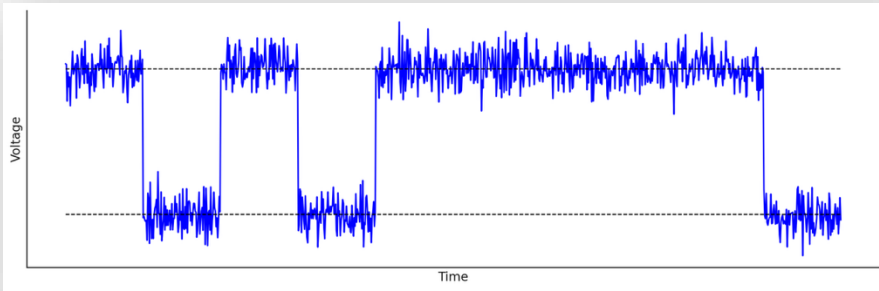
10進数：10種類の数字を用いて、10ごとに桁が上がる

2進数：2種類の数字を用いて、2ごとに桁が上がる

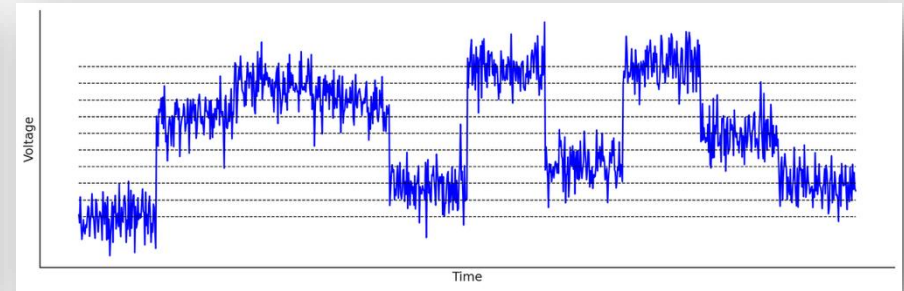
|      |   |   |    |    |     |     |     |     |      |      |      |      |
|------|---|---|----|----|-----|-----|-----|-----|------|------|------|------|
| 10進数 | 0 | 1 | 2  | 3  | 4   | 5   | 6   | 7   | 8    | 9    | 10   | 11   |
| 2進数  | 0 | 1 | 10 | 11 | 100 | 101 | 110 | 111 | 1000 | 1001 | 1010 | 1011 |

2進数は機械の仕組みとして表現しやすい

- スイッチ：オン(1), オフ(0)
- 電圧：高い(1), 低い(0)



電圧による数値表現のイメージ  
(2進数)



電圧による数値表現のイメージ  
(10進数)

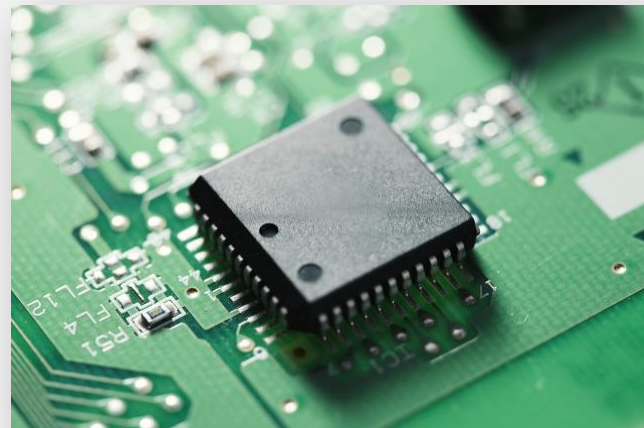
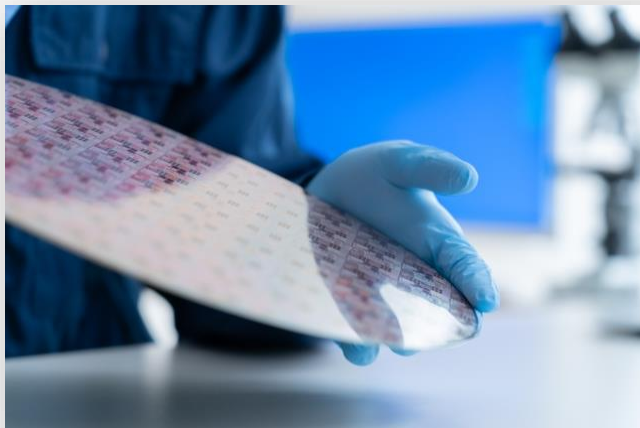
## 微細な電圧スイッチで0と1を表現

**半導体：導体（電気を通す）と絶縁体（通さない）の中間的な物質**

- 電気抵抗が導体と絶縁体の中間程度の物質
  - 例：シリコン，ゲルマニウム
- 不純物や外部刺激の有無によって，電気を通したり通さなかったりする  
→ スイッチとして0/1を表現可能

**ICチップ：大量のスイッチを並べた装置（集積回路，Integrated Circuit）**

- 微細な半導体装置を大量に並べたデバイス
- 各半導体装置は電気信号の「遮断 or 通過」で「0 or 1」を表現



左：シリコン単結晶のインゴット

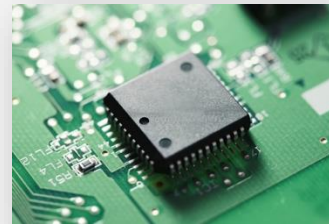
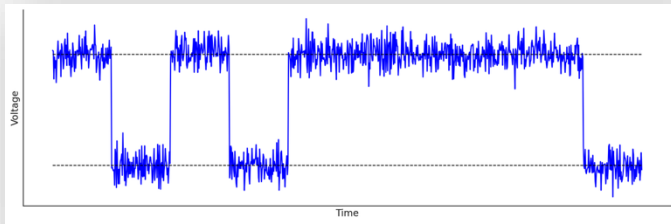
現在の電子計算機では、2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで記述される

デジタル：情報を数値で表現する方法，保持・伝達しやすい

2進数：2で桁の上がる数値表現，機械で表現しやすい

|      |   |   |    |    |     |     |     |     |      |      |      |      |
|------|---|---|----|----|-----|-----|-----|-----|------|------|------|------|
| 10進数 | 0 | 1 | 2  | 3  | 4   | 5   | 6   | 7   | 8    | 9    | 10   | 11   |
| 2進数  | 0 | 1 | 10 | 11 | 100 | 101 | 110 | 111 | 1000 | 1001 | 1010 | 1011 |



## 1. 計算機の仕組み

- 計算機内の情報表現
- 計算機のハードウェア
- 人間が扱うための仕組み

## 2. ネットワークの仕組み

- 計算機同士の情報交換
- インターネット

## 3. 情報システムの仕組み

- 情報システムとデータベース
- 情報セキュリティ

## 「2進数表現にしたときの情報の大きさ」を分かりやすく示す単位

ビット：2進数の桁数を表す単位

例：0→1bit, 1→1bit, 101→3bit, 10010010→8bit

バイト：2進数の8桁※を“1つ”にまとめたとき単位（8bit = 1Byte）

例：10010010 → 1Byte (=8bit), 1101101010000011 → 2Byte (=16bit)

## ビット数やバイト数が増えると、内部で扱える情報が大きくなる

- ファミコンのCPUは8bit, 通常のパソコンのCPUは64bit
- iPhone 16のメモリは8GB, ストレージは128GB  
(計算機では通常,  $K = 2^3$ ,  $M = 2^6$ ,  $G = 2^9$ を表す)

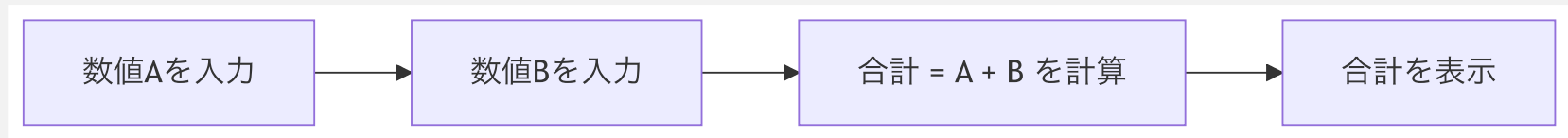
| 情報量          | 表せる情報の種類              |
|--------------|-----------------------|
| 1bit         | 2種類：0か1のどちらか          |
| 2bit         | 4種類：00, 01, 10, 11    |
| 3bit         | 8種類：0～ $2^3$ , つまり0～8 |
| ...          |                       |
| 8bit (1Byte) | 256種類：0～ $2^8$        |

※ “8”は $2^3$ , すなわち, 2進数表現における“1000”というキリのいい数字

## 人間が計算機に命令を出すための言語

「0と1を記憶・伝達するスイッチ群」を直接操作してITシステムを作るのは難しい

- 四則演算や文字入力・表示など**基本的な命令を簡単に書ける**言語
- 用途や好みに応じて様々な言語が開発されている
  - 例：Python, java, C++, R, Go, Rust, ...

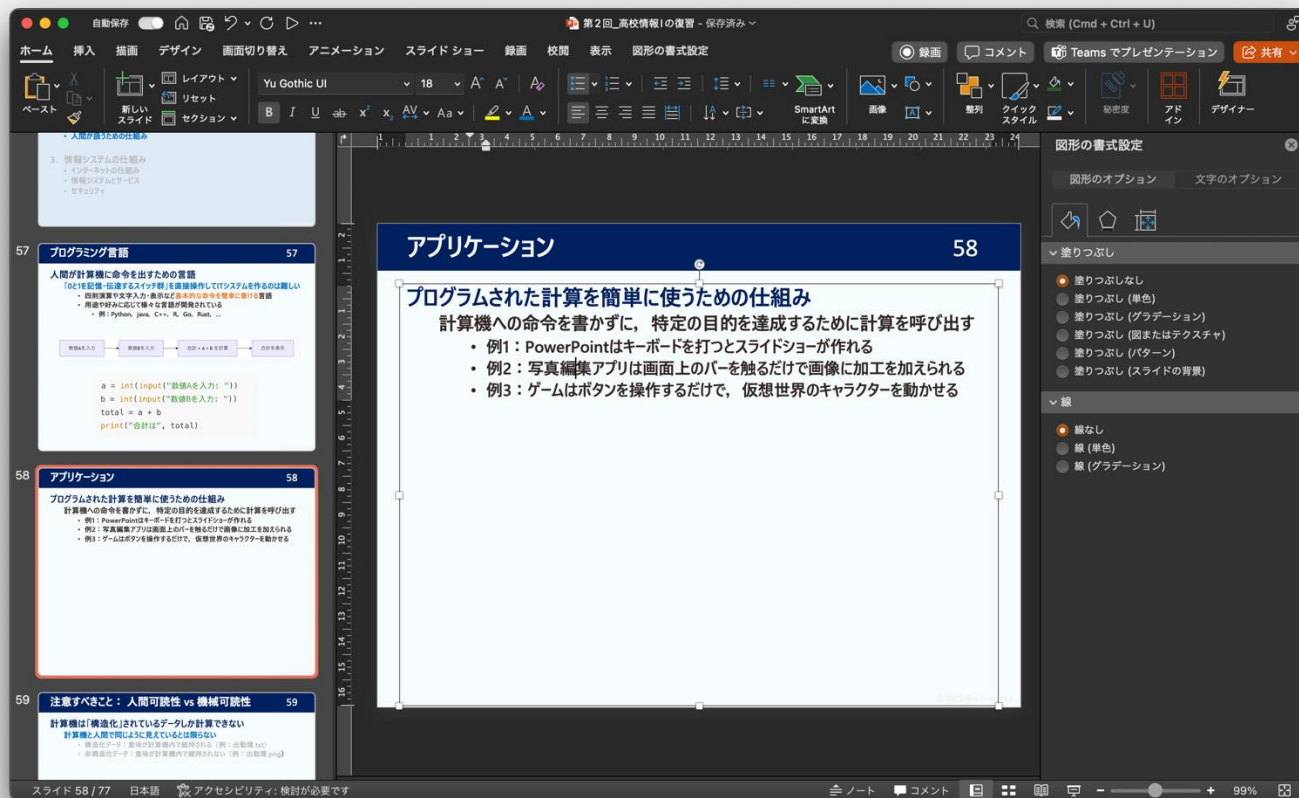


```
a = int(input("数値Aを入力: "))
b = int(input("数値Bを入力: "))
total = a + b
print("合計は", total)
```

## プログラムされた計算を簡単に使うための仕組み

計算機への命令を書かずに、特定の目的を達成するために計算を呼び出す

- 例1：PowerPointはキーボードを打つとスライドショーが作れる
- 例2：写真編集アプリは画面上のバーを触るだけで画像に加工を加えられる
- 例3：ゲームはボタンを操作するだけで、仮想世界のキャラクターを動かせる



## 計算機は「構造化」されているデータしか計算できない

計算機と人間で同じように見えているとは限らない

- ・ 構造化データ：意味が計算機内で維持される（例：出勤簿.txt）
- ・ 非構造化データ：意味が計算機内で維持されない（例：出勤簿.png）

「人間に読みやすいデータ」が計算機にも読みやすいとは限らない

「10,000」→ 31302c303030（Shift\_JISコード）

「10000」→ 10000（整数型）

| 修正前                                    |         |         |        | 修正後                 |       |      |        |
|----------------------------------------|---------|---------|--------|---------------------|-------|------|--------|
| 「円」、「▲（マイナス表記）」、「,（カンマ）」が文字列として入力されている |         |         |        | 数値データを数値属性として入力した状態 |       |      |        |
|                                        | 単価      | 前回差分    | 生産台数   |                     | 単価    | 前回差分 | 生産台数   |
| サンプル1                                  | 10,000円 | 130     | 12,000 | サンプル1               | 10030 | 130  | 12000  |
| サンプル2                                  | 9,100円  | ▲200    | 29,000 | サンプル2               | 9100  | -200 | 29000  |
| サンプル3                                  | 8,020円  | ▲350    | 37,000 | サンプル3               | 8020  | -350 | 37000  |
| サンプル4                                  | 7,500円  | 500     | 43,000 | サンプル4               | 7500  | 500  | 43000  |
| SUM関数                                  | 0       | 630     | 0      | SUM関数               | 34650 | 80   | 121000 |
| +(加算演算)                                | #VALUE! | #VALUE! | 121000 | +(加算演算)             | 34650 | 80   | 121000 |
| ↑黄色の行は関数で合計を表示した例                      |         |         |        | ↑黄色の行は関数で合計を表示した例   |       |      |        |

| 文字 | コード | 文字 | コード |
|----|-----|----|-----|
| a  | 97  | b  | 98  |
| e  | 101 | f  | 102 |
| i  | 105 | j  | 106 |
| m  | 109 | n  | 110 |
| q  | 113 | r  | 114 |
| u  | 117 | v  | 118 |
| y  | 121 | z  | 122 |

## 1. 計算機の仕組み

- 計算機内の情報表現
- 計算機のハードウェア
- 人間が扱うための仕組み

## 2. ネットワークの仕組み

- 計算機同士の情報交換
- インターネット

## 3. 情報システムの仕組み

- 情報システムとデータベース
- 情報セキュリティ

## 計算機同士の通信経路を網目上に結んだもの

### LAN : Local Area Network

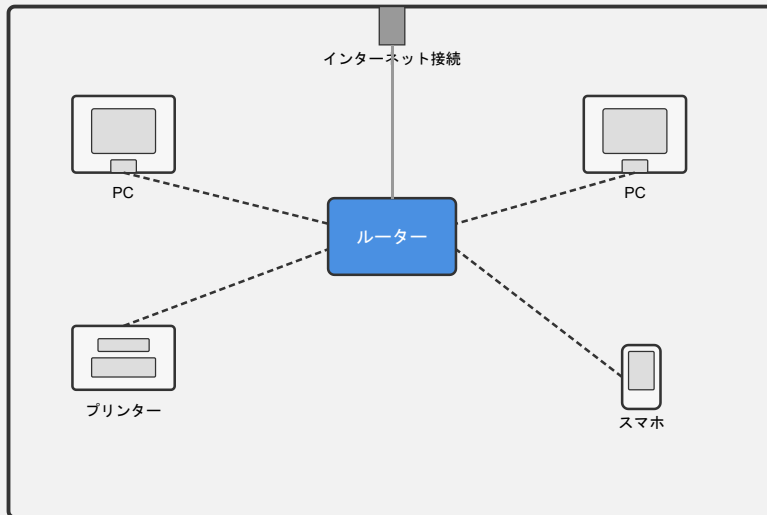
限定された場所のネットワーク

例) 大学の無線LAN, 社内LAN, (LANケーブル)

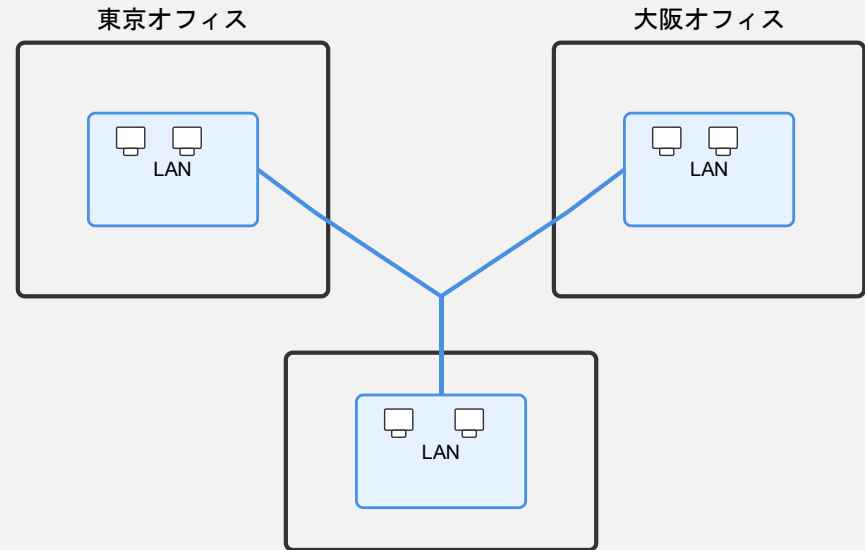
### WAN : Wide Area Network

遠く離れた地点を結ぶネットワーク

例) 電話回線, 企業の複数拠点で通信するためのネットワーク



LAN



福岡オフィス

WAN

## 世界中のネットワークを相互に接続可能にする通信経路

### インターネット

Inter + net : 中間の/相互の + ネットワーク

→ ネットワークとネットワークを接続したもの

世界中のネットワーク同士の通信経路を網状に結んだもの

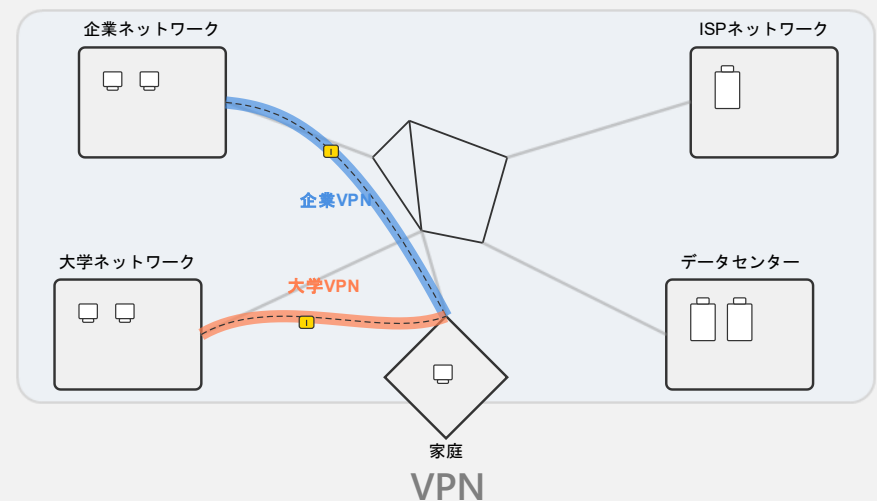
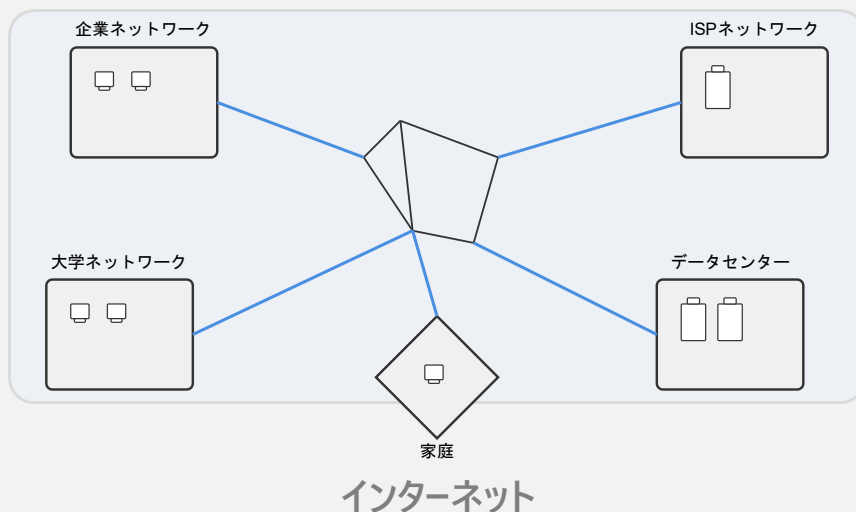
→ 接続されたネットワークの中にいれば、別のネットワークの端末にもアクセス可能

### VPN : Virtual Private Network

インターネット上に作られた仮想的なネットワーク

→ VPNで結ばれたネットワーク内の端末同士でのみアクセス可能

例) 在宅ワーク用VPN, 大学VPN, 海外VPN



## 1. 計算機の仕組み

- 計算機内の情報表現
- 計算機のハードウェア
- 人間が扱うための仕組み

## 2. ネットワークの仕組み

- 計算機同士の情報交換
- インターネット

## 3. 情報システムの仕組み

- 情報システムとデータベース
- 情報セキュリティ

## 計算機同士の通信経路を網目上に結んだもの

### LAN : Local Area Network

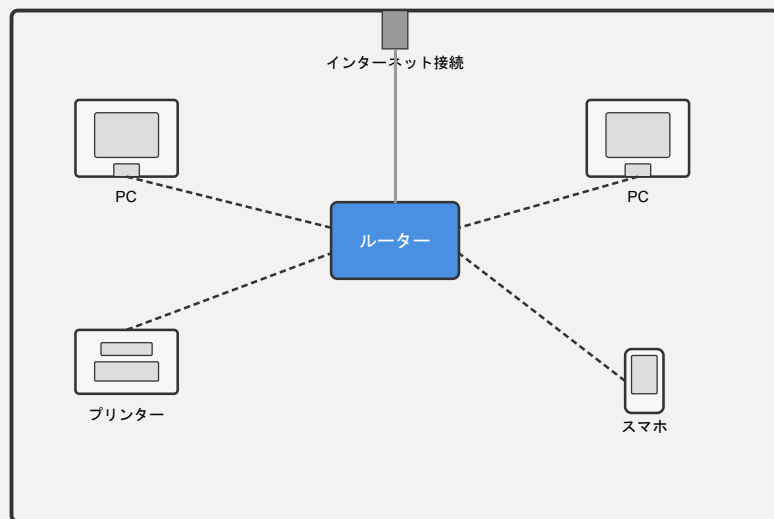
限定された場所のネットワーク

例) 大学の無線LAN, 社内LAN, (LANケーブル)

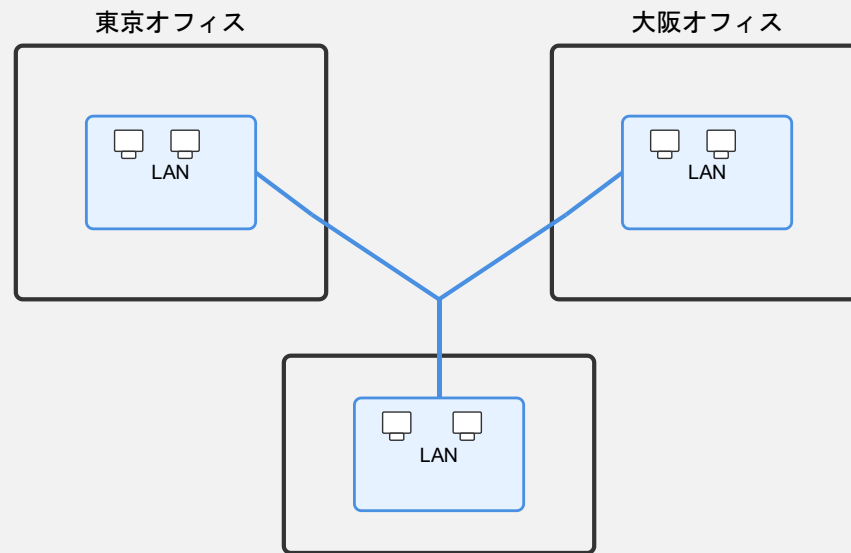
### WAN : Wide Area Network

遠く離れた地点を結ぶネットワーク

例) 電話回線, 企業の複数拠点で通信するためのネットワーク



LAN



福岡オフィス

WAN

## 世界中のネットワークを相互に接続可能にする通信経路

### インターネット

Inter + net : 中間の/相互の + ネットワーク

→ ネットワークとネットワークを接続したもの

世界中のネットワーク同士の通信経路を網状に結んだもの

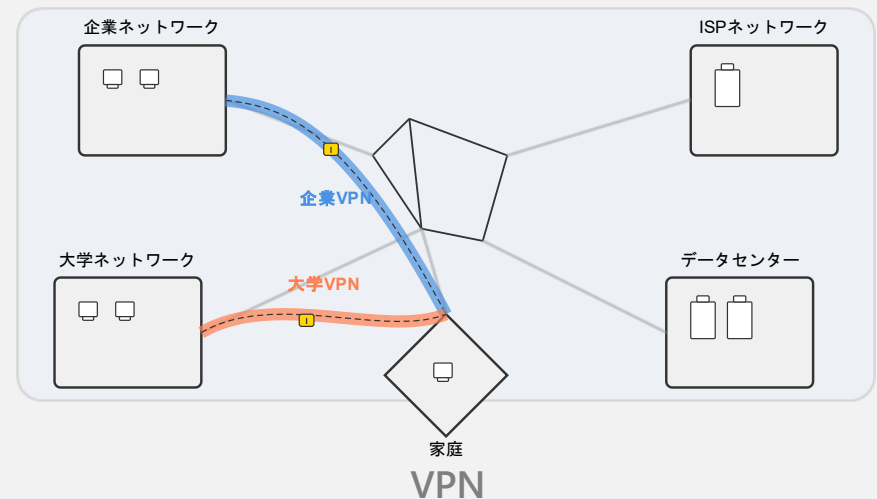
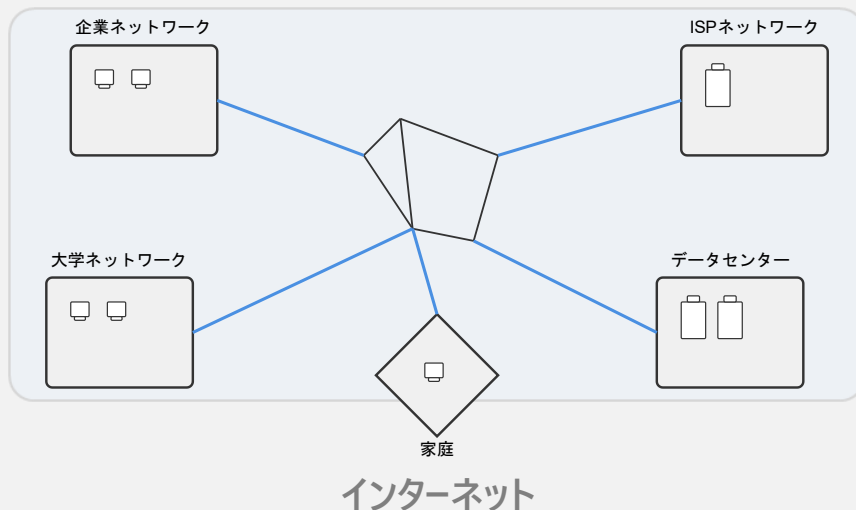
→ 接続されたネットワークの中にいれば、別のネットワークの端末にもアクセス可能

### VPN : Virtual Private Network

インターネット上に作られた仮想的なネットワーク

→ VPNで結ばれたネットワーク内の端末同士でのみアクセス可能

例) 在宅ワーク用VPN, 大学VPN, 海外VPN



## 多くはデータを管理する「データベース」が中核にある

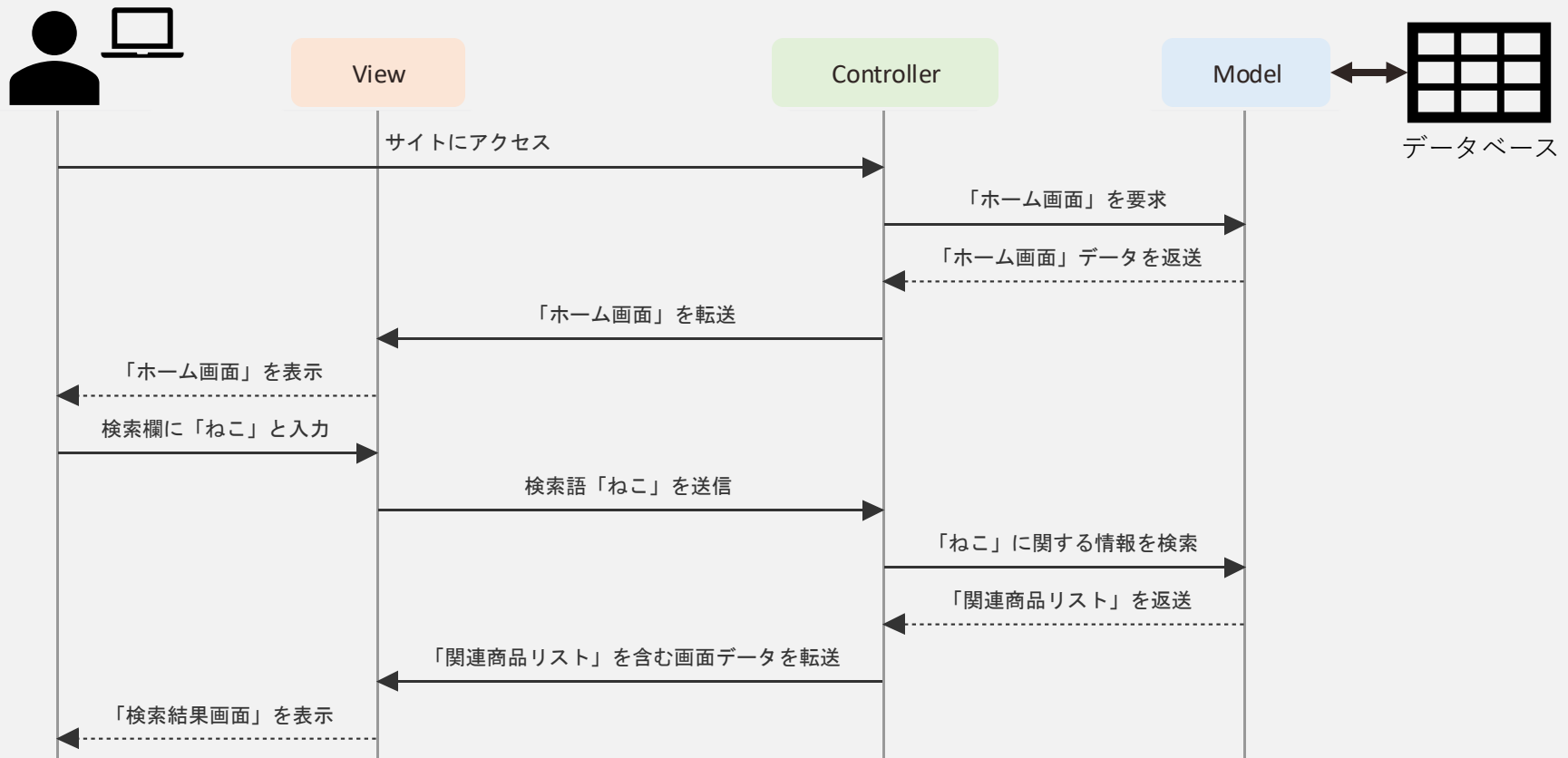
### 例) MVCモデル

**Model** : データベースを操作するプログラム

**View** : ユーザーに情報を閲覧・入力させるプログラム

例) ボタン, 検索欄, 入力ウィンドウ, マウスでクリックして動かせる図形

**Controller** : 情報を**View**に渡す・データの操作を**Model**に指示するプログラム



## 情報漏洩や改竄，喪失などを防ぐための安全対策

### 情報セキュリティの3要素

#### 機密性：許可された人だけアクセス可能

- ・ 脅威例) 不正アクセスによる情報の漏洩・破壊
- ・ 対策例) パスワード管理，外部からの不正通信を遮断（ファイアウォール）

#### 完全性：情報を正確に維持

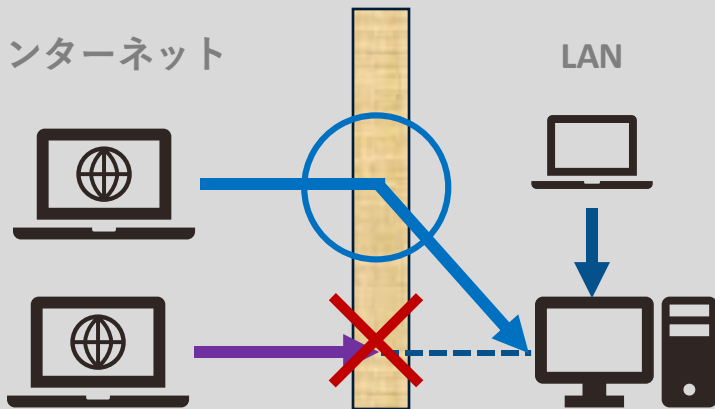
- ・ 脅威例) データの改竄，ウイルス感染
- ・ 対策例) セキュリティソフト，OS更新

#### 可用性：許可された人は必要なときいつでも利用可能

- ・ 脅威例) 自然災害，サービス妨害攻撃
- ・ 対策例) バックアップ，無停電電源装置

インターネット

LAN



ファイアウォール：正常な通信のみ認める



成績情報

教員：○  
学生：本人は○，他人は×



授業資料

教員：○  
学生：○



期末試験

教員：○  
学生：×

アクセス制御：データごとにアクセス権限を設定する

## 計算機やインターネットの仕組み

計算機内部の情報表現はデジタル（数字式）の二進表現（0 or 1）

- 物理量（アナログ値）を数字式に近似すると記録・伝達しやすくなる
- 数値を0と1のみで表現すると、単純な“スイッチ”（半導体）で実現できる

ネットワークは網目状に結ばれた計算機同士の通信経路

- インターネット（世界中のネットワーク同士が相互に通信できる仕組み）が登場する
- 情報システム（基本的には情報を操作する仕組み）が普及する
- セキュリティ（攻撃者や災害などの外的要因から情報を守る仕組み）が重要になる

## II. 計算機やデータ， AIにできることを知る

3. 計算機の基本原理

4. 計算機と統計

## I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機，インターネット，情報システム

## II. 計算機やデータ，AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

## III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

## IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習（データ構造とアルゴリズム）

## V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

## VI. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

## VII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習（PERT図，決定表）

1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、  
**目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる**
3. ITシステムの構造や仕組みを学び、  
問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる
4. 計算機のハードウェア構成を知り、  
目的達成に必要な装置の見積もりや購入ができるようになる
5. 計算機に特有のデータ表現を学び、  
機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考を体験する

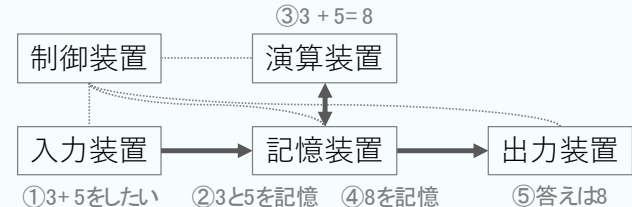
# 2025年度 コンピュータシステム

## 第3回：計算機とプログラムによる自動化

## 計算機の持つ機能は少ないが，組み合わせて色々な機能を作れる

### 1. 計算機の構造と自動計算の原理

- 5種類の装置で構成される
- 装置が連携して計算を行う

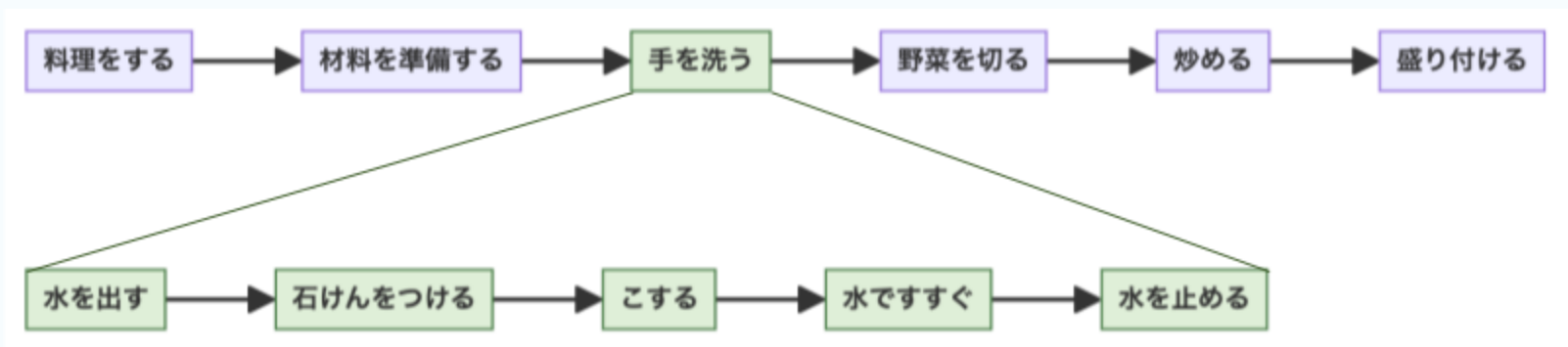


### 2. ハードウェア実装される演算は数種類のみ

- 機械語（2進数），アセンブリ言語（機械語直訳），プログラミング言語

### 3. 演算の組み合わせ（アルゴリズム）で多様な演算を実現

- 命令を逐次処理，分岐，繰り返しに並べてアルゴリズムを作る
- アルゴリズムを組み合わせでアルゴリズムをつくる



第3回の内容は、端的に述べると次のようになる。

まず、「計算機」というハードウェアに実装されている演算の種類はとても少ない。

しかし、演算を使って「計算手順（アルゴリズム）」を考えることで様々な機能を実現できる

そして、そうして作ったアルゴリズムを、別のアルゴリズムを考えると「一つの演算」として使うことができる。

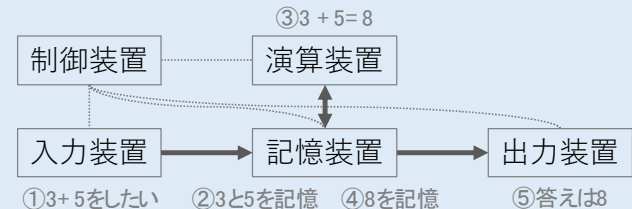
このように、アルゴリズムを組み合わせてアルゴリズムを作ることで、より複雑なアルゴリズムを考えられるようになる

エンジニアやプログラマは、アルゴリズムを利用してアルゴリズムを作ることで、色々な仕事を計算機で自動化している

計算機の持つ機能は少ないが，組み合わせて色々な機能を作れる

## 1. 計算機の構造と自動計算の原理

- 5種類の装置で構成される
- 装置が連携して計算を行う



## 2. ハードウェア実装される演算は数種類のみ

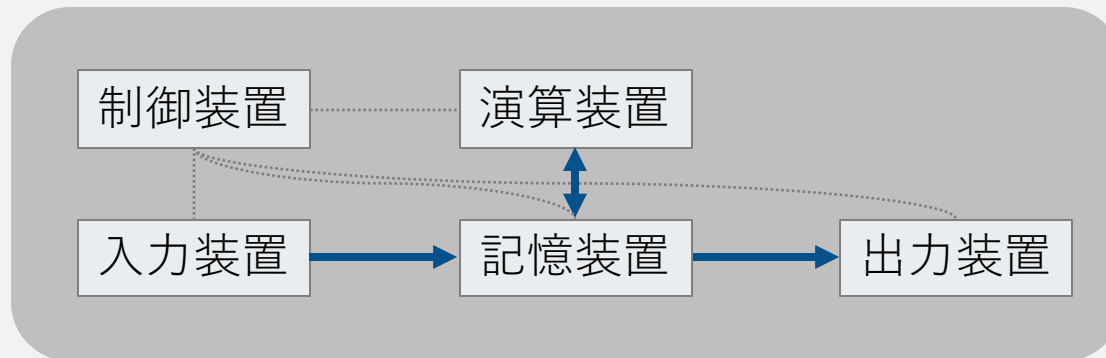
- 機械語（2進数），アセンブリ言語（機械語直訳），プログラミング言語

## 3. 演算の組み合わせ（アルゴリズム）で多様な演算を実現

- 命令を逐次処理，分岐，繰り返しに並べてアルゴリズムを作る
- アルゴリズムを組み合わせでアルゴリズムをつくる

## 計算機が計算をする仕組みの簡単な説明

1. 演算装置：足し算や掛け算など、演算を行う回路
2. 記憶装置：計算途中の数値などを記憶する装置
3. 入力装置：数値や計算方法を入力する装置
4. 制御装置：計算を行うために装置同士を連携させる装置
5. 出力装置：計算結果を出力する装置

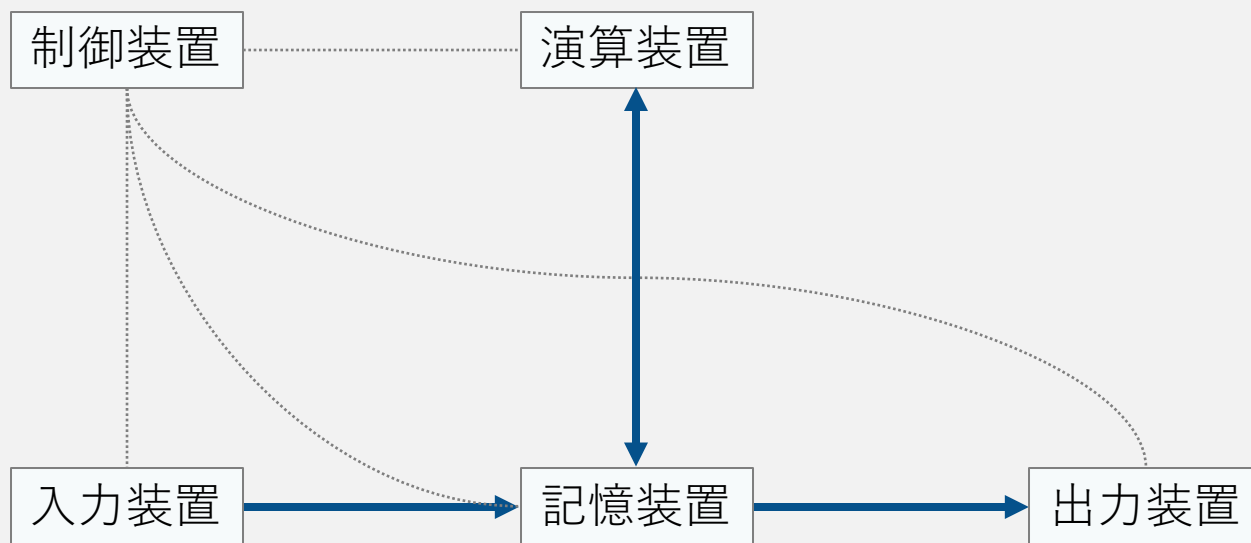


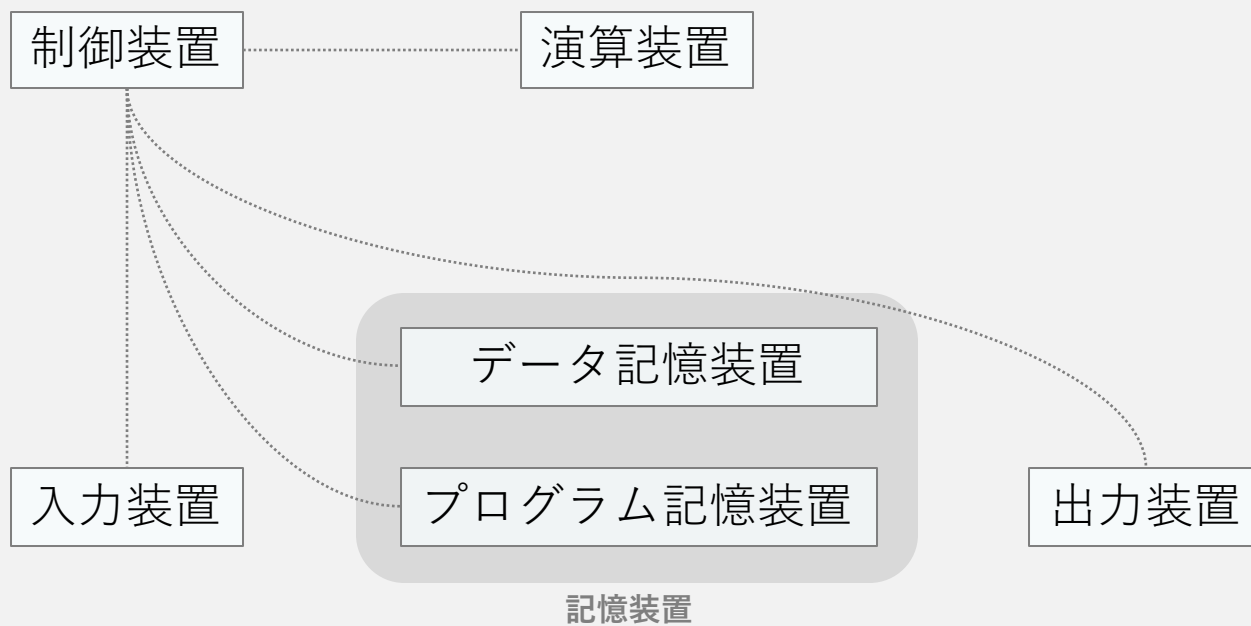
PCやスマホを選ぶとき、スペック表はどう見たらいいのか？

CPU, メモリ, HDD/SSD, ...

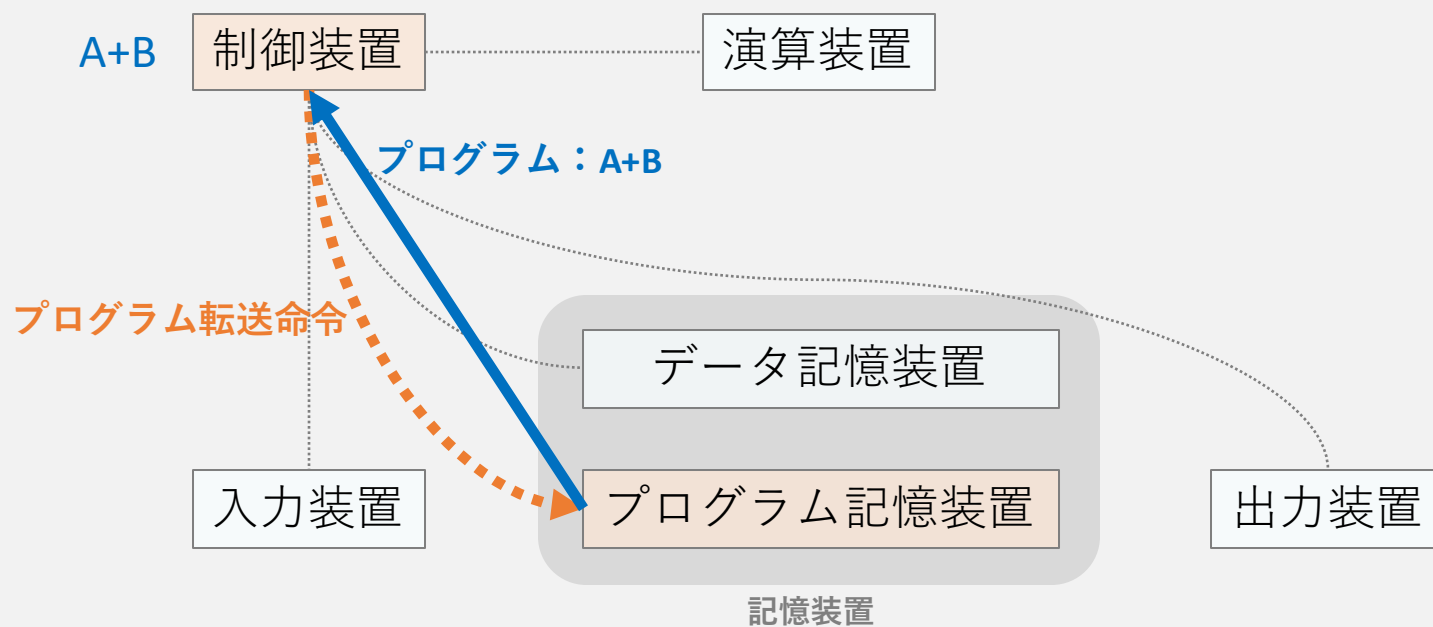
結局、情報技術は何に使えるのか？どう使えばいいのか？

より詳細に踏み込んで理解する必要がある

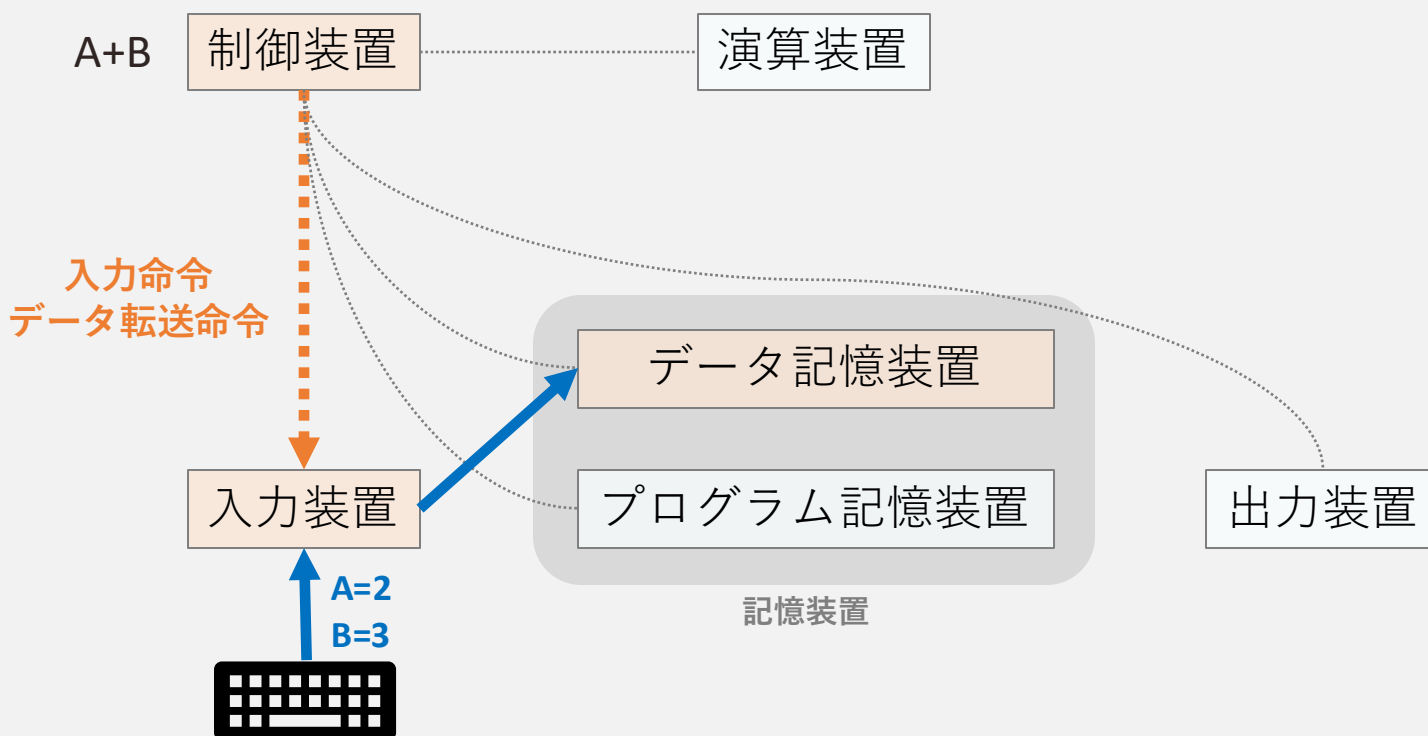




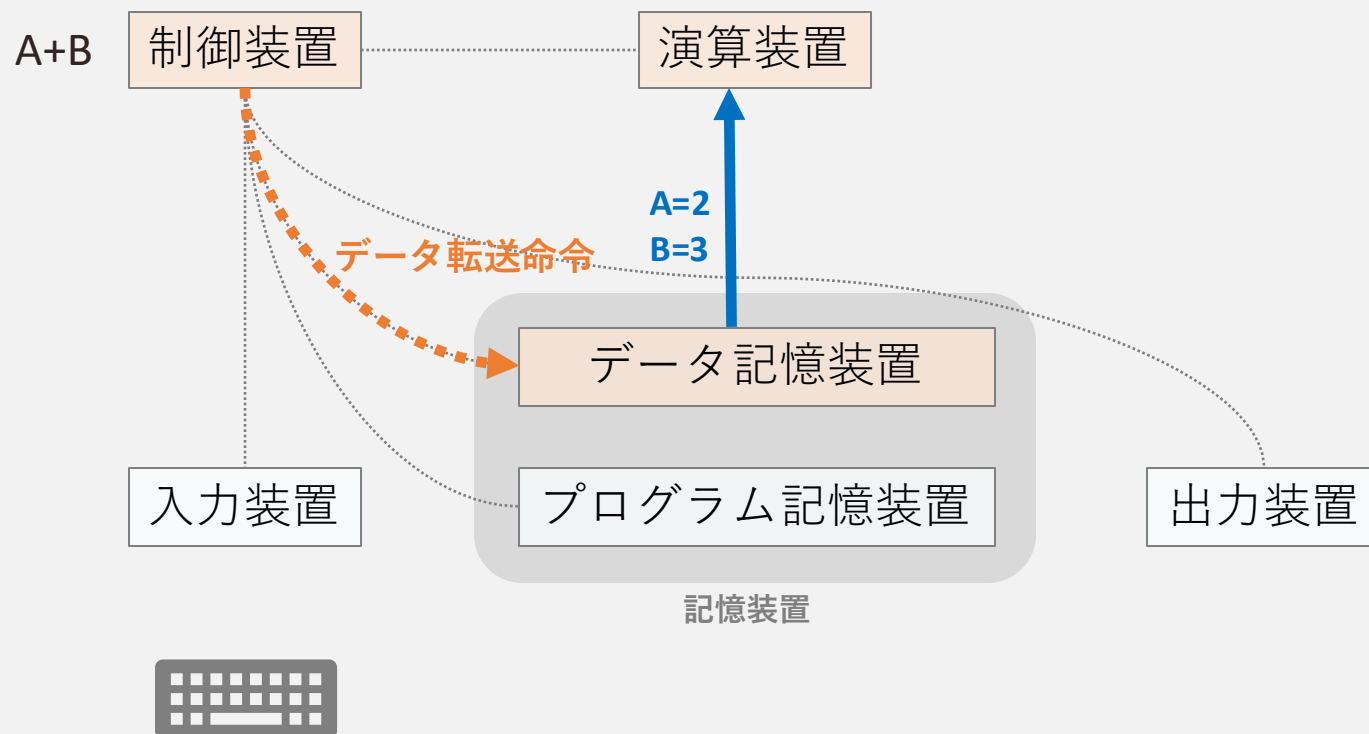
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



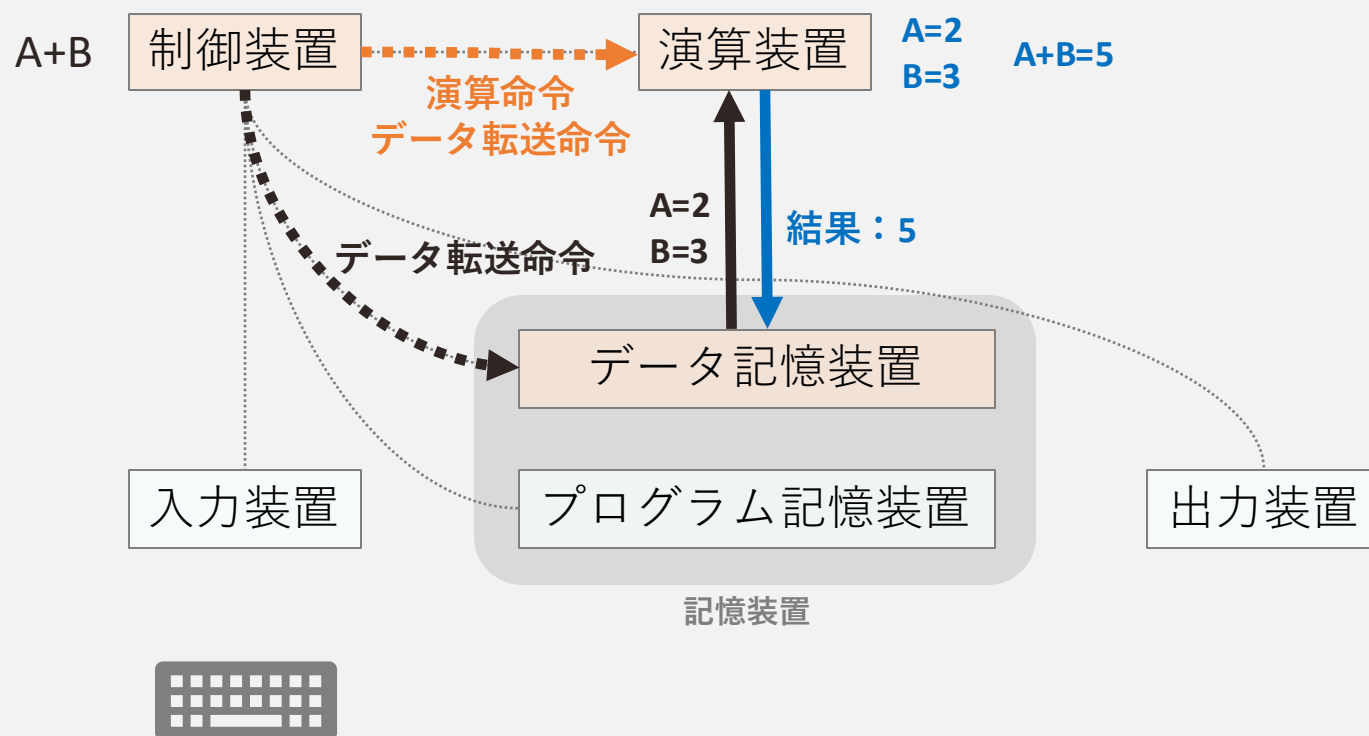
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



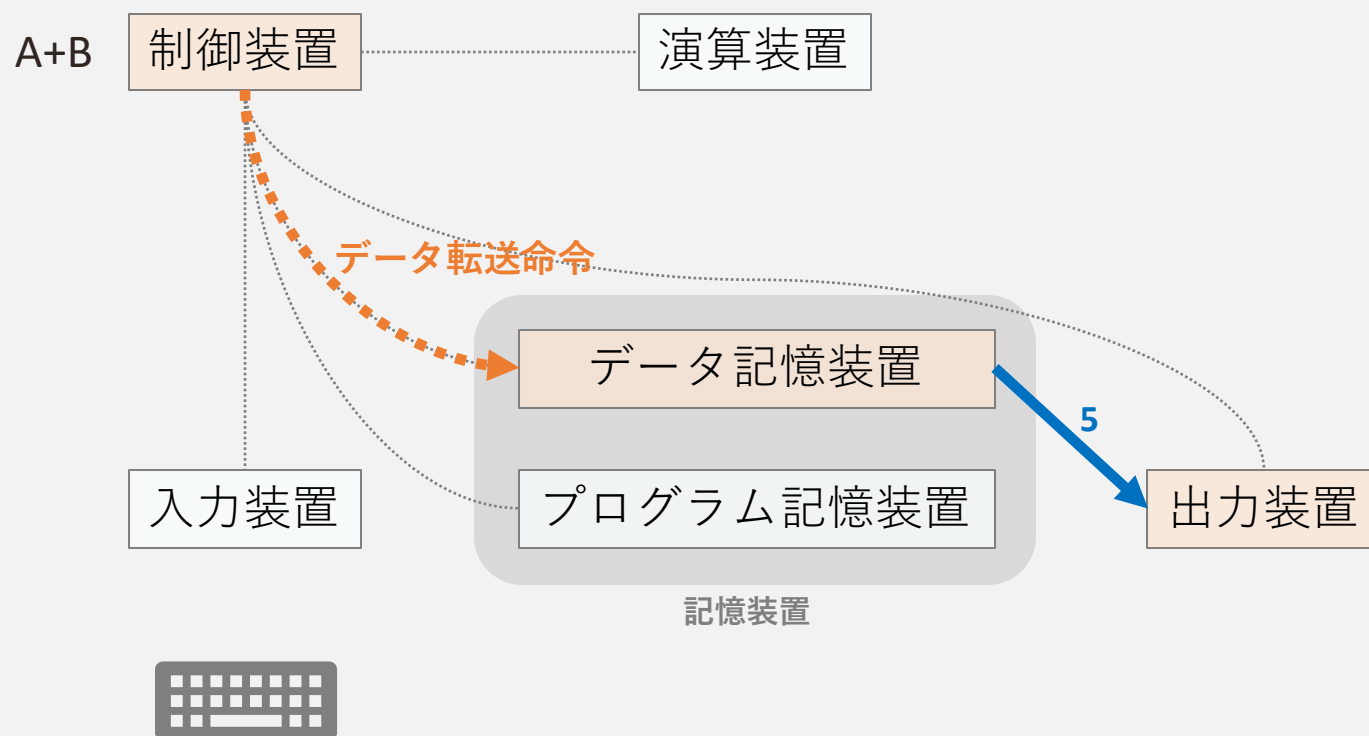
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



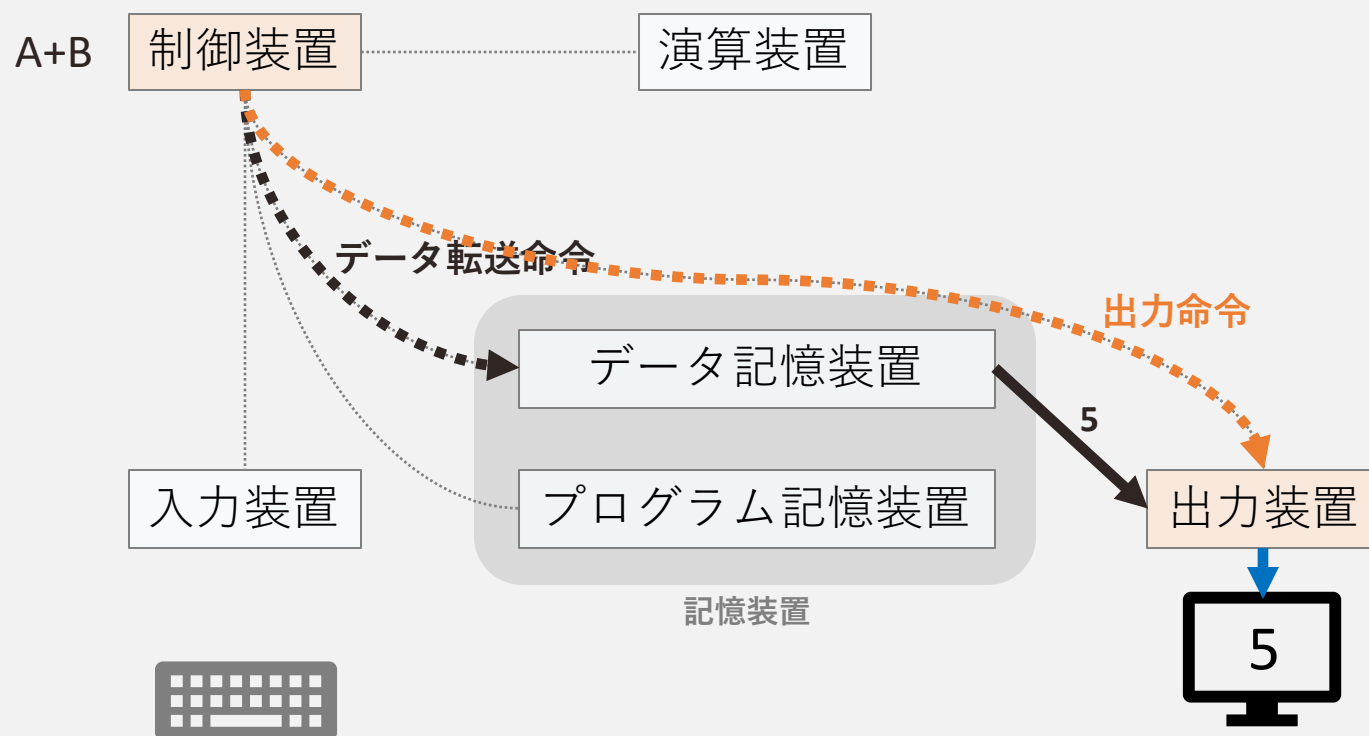
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



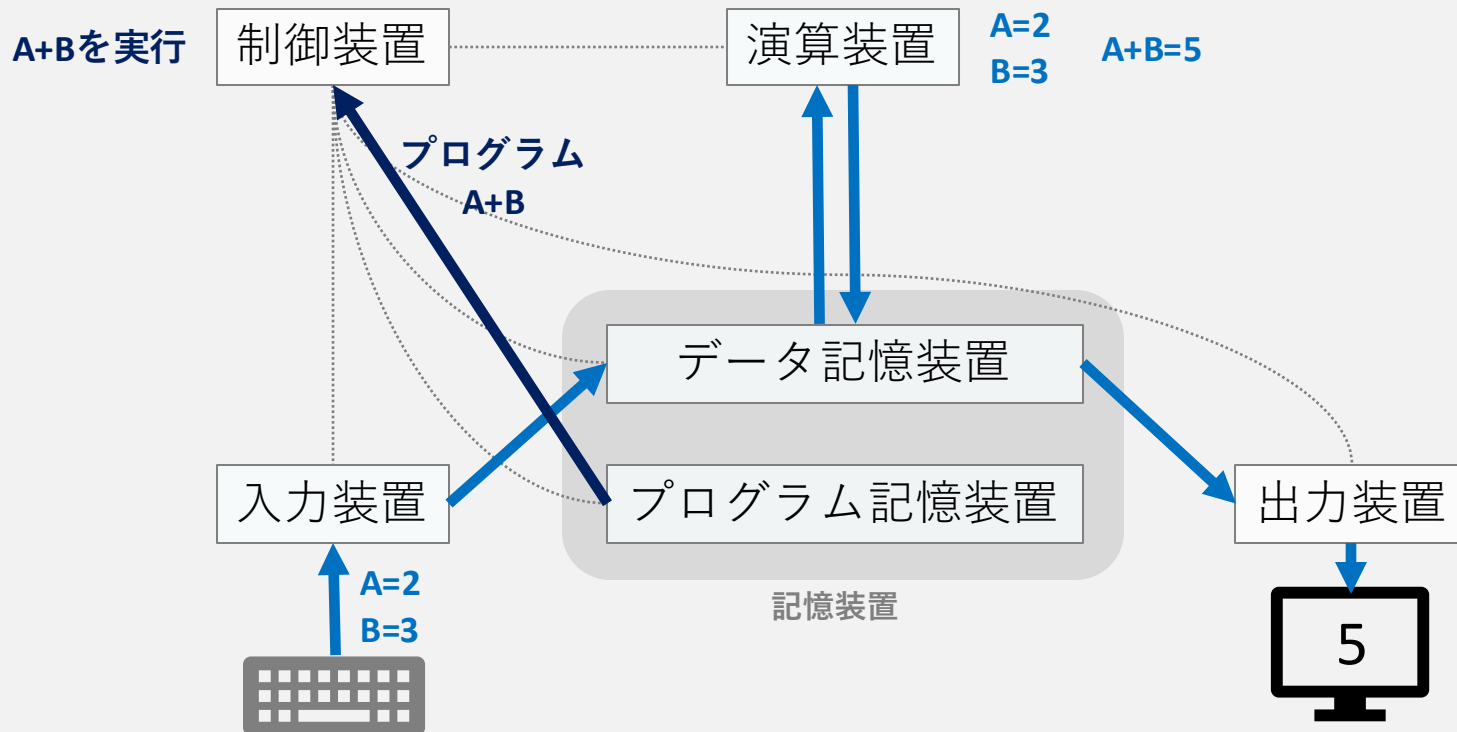
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令

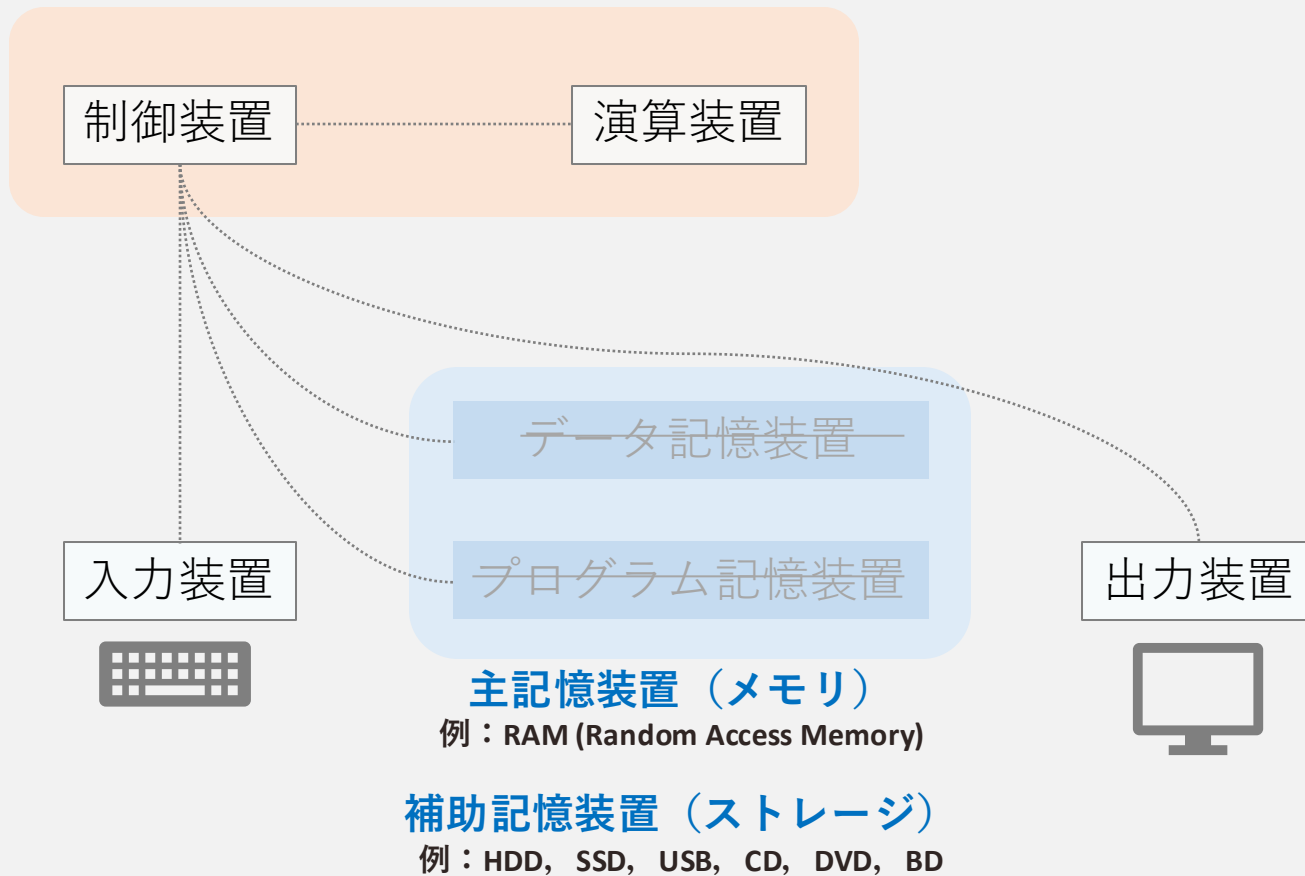


- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



## 中央処理装置（CPU, Central Processing Unit）

例：Intel Core i5, Apple M1, Ryzen5 2500U, Snapdragon S2

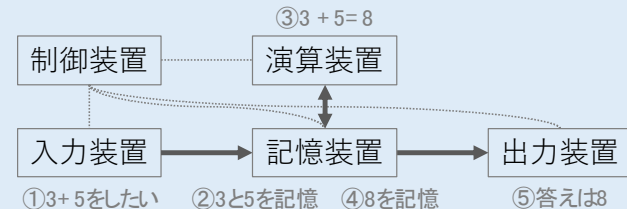


ハードウェアの詳細は第5回授業で取り扱う

計算機の持つ機能は少ないが，組み合わせて色々な機能を作れる

## 1. 計算機の構造と自動計算の原理

- 5種類の装置で構成される
- 装置が連携して計算を行う



## 2. ハードウェア実装される演算は数種類のみ

- 機械語（2進数），アセンブリ言語（機械語直訳），プログラミング言語

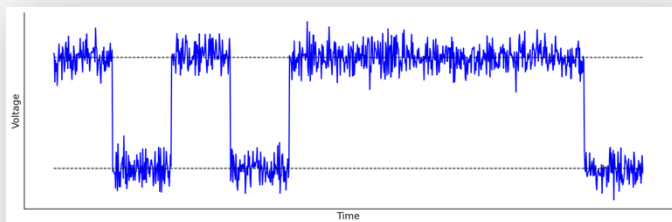
## 3. 演算の組み合わせ（アルゴリズム）で多様な演算を実現

- 命令を逐次処理，分岐，繰り返しに並べてアルゴリズムを作る
- アルゴリズムを組み合わせでアルゴリズムをつくる

現在の電子計算機では、2進数のデジタル表現が標準的

- あらゆる情報はデジタル化されて有限桁の数字に近似される
- 数字化された情報は2進数で表現されて0と1のみで記述される

|      |   |   |    |    |     |     |     |     |      |      |      |      |
|------|---|---|----|----|-----|-----|-----|-----|------|------|------|------|
| 10進数 | 0 | 1 | 2  | 3  | 4   | 5   | 6   | 7   | 8    | 9    | 10   | 11   |
| 2進数  | 0 | 1 | 10 | 11 | 100 | 101 | 110 | 111 | 1000 | 1001 | 1010 | 1011 |



デジタル（数字式）：情報を保持・伝達しやすい  
2進数（0と1のみ）：機械で表現しやすい

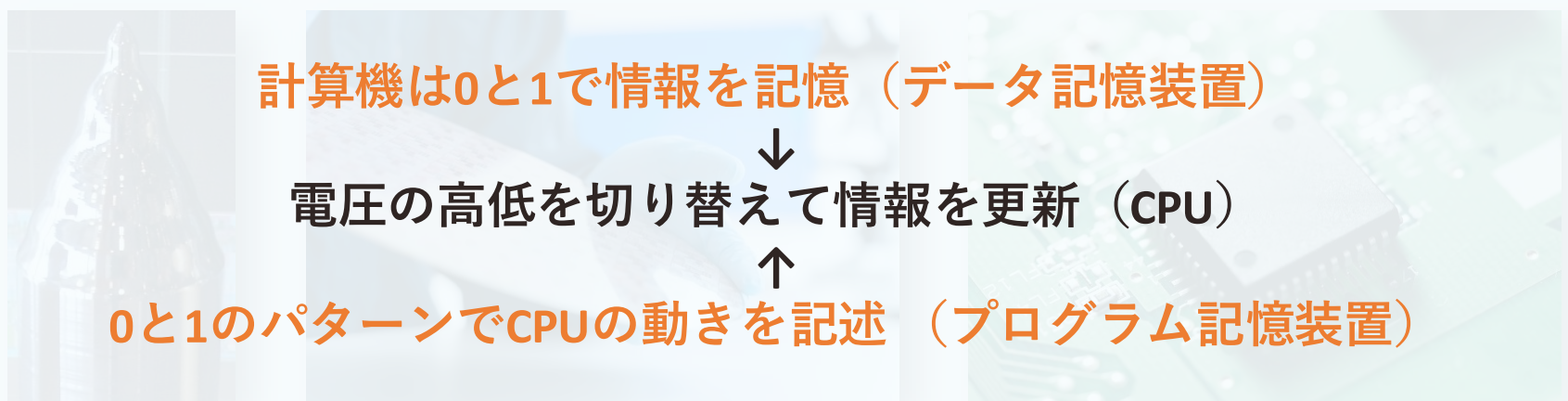
## 微細な電圧スイッチで0と1を表現

半導体：導体（電気を通す）と絶縁体（通さない）の中間的な物質

- ・ 電気抵抗が導体と絶縁体の中間程度の物質
  - ・ 例：シリコン，ゲルマニウム
- ・ 不純物や外部刺激の有無によって，電気を通したり通さなかったりする  
→ スイッチとして0/1を表現可能

ICチップ：大量のスイッチを並べた装置（集積回路，Integrated Circuit）

- ・ 微細な半導体装置を大量に並べたデバイス
- ・ 各半導体装置は電気信号の「遮断 or 通過」で「0 or 1」を表現



計算機は0と1で情報を記憶（データ記憶装置）

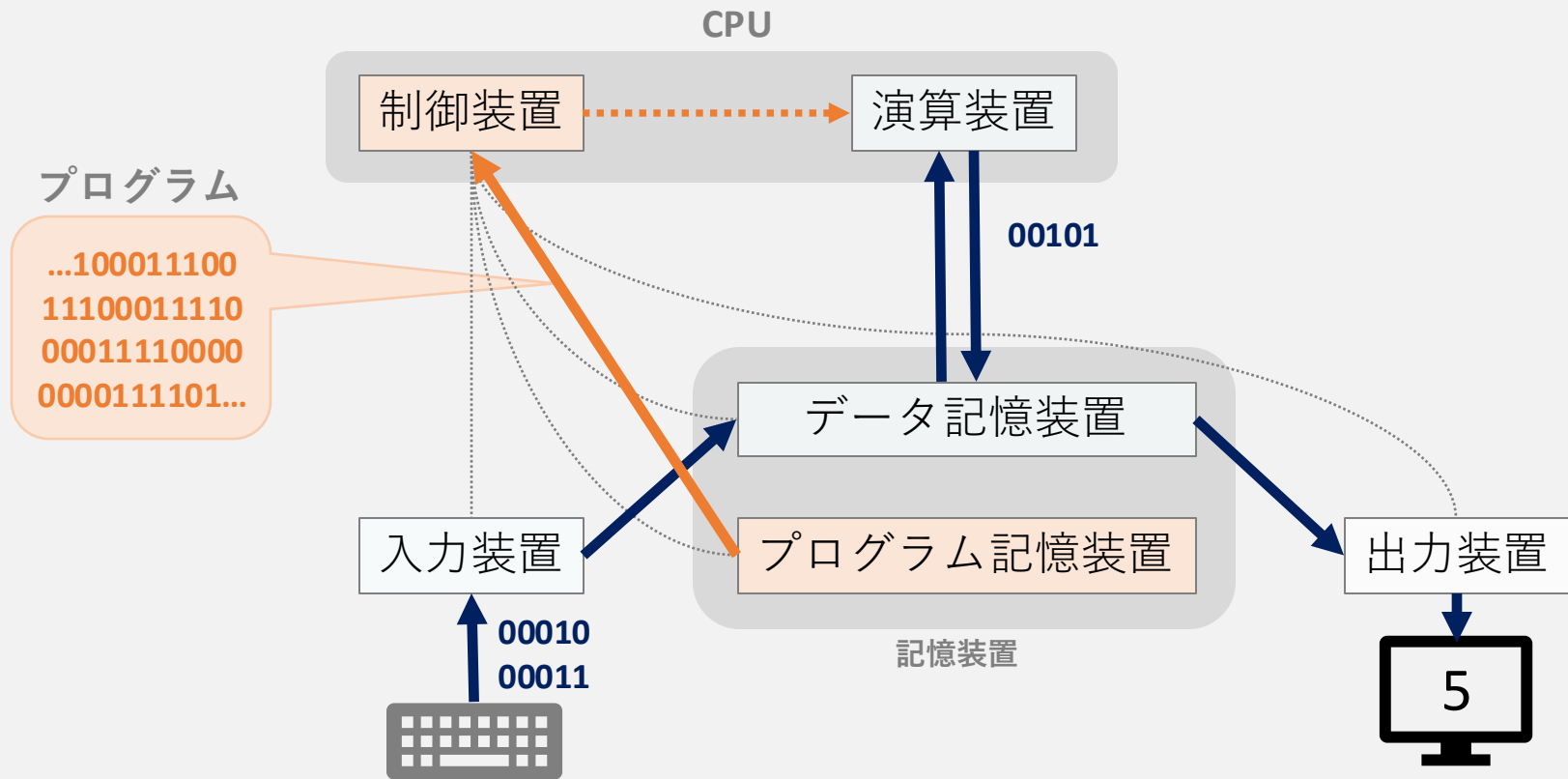
↓  
電圧の高低を切り替えて情報を更新（CPU）

↑  
0と1のパターンでCPUの動きを記述（プログラム記憶装置）

左：シリコン単結晶のインゴット

CC 表示-継承 3.0, <https://commons.wikimedia.org/w/index.php?curid=314282>

## 0と1のパターンでCPUの動きを制御



## 機械語を人間に分かりやすく直訳した表現

### 2進数の意味を命令と記号で記述

例1) 10110000 10101011

↓

MOV AL, abh : 記憶装置ALに”abh”という情報を記録せよ

例2) 01001001 10000001 11000101 11101110 11111111 11000000 00000000

↓

ADD r13, 0xc0ffee : 記憶装置r13の情報に 0xc0ffee (12648430) を加算せよ

### アセンブリ言語で書ける命令 (=CPUの動作) は大きく分けると4種類

|        |                                 |
|--------|---------------------------------|
| 演算     | 記憶した情報同士の操作 (加算, 比較, and, orなど) |
| 逐次実行   | 命令を上から順に実行                      |
| 条件ジャンプ | 命令の実行順序を変更 (真下でない命令にジャンプ)       |
| メモリ入出力 | メモリの読み出し, 書き込み                  |

参考: 高校生だけじゃもったいない 仕事に役立つ新・必修科目「情報Ⅰ」

アセンブリ言語の命令を組み合わせて「人間に分かりやすい命令」を簡単に書けるようにした人工言語

例1) `print("こんにちは")`

意味: "こんにちは"と出力する

例2) `If (number>0) then print("自然数"), else print("0以下")`

意味: numberが1以上なら"自然数"と出力, そうでないなら"0以下"と出力

## アセンブリ言語

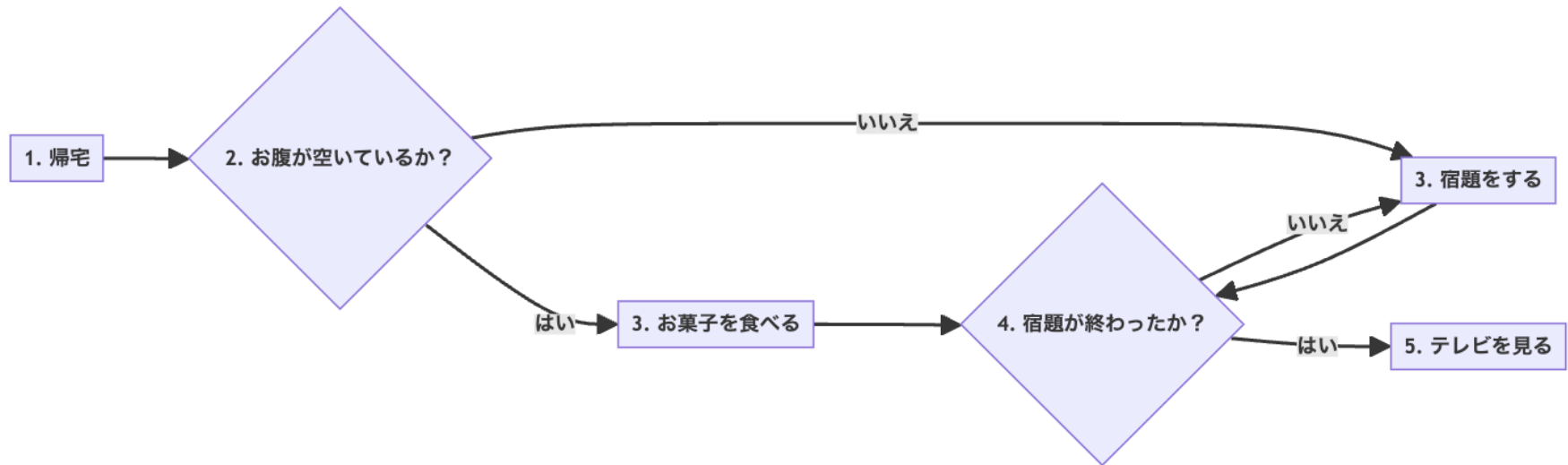
- 
- 1: 変数numberの値をメモリから得る
  - 2: メモリから得た値と0を比較する
  - 3: 0以下であれば6行目にジャンプ
  - 4: 別の場所にかかれたプログラム「print」に情報"0以下"を与えて実行して戻ってくる
  - 5: 7行目にジャンプ
  - 6: 別の場所にかかれたプログラム「print」に情報"自然数"を与えて実行して戻ってくる
  - 7:
  - ...

演算, 逐次実行, 条件ジャンプ, メモリ入出力 (=CPU動作) の組み合わせを簡単に使える

命令をどのように組み立てたらいいのか?

## 命令を逐次実行，条件分岐，繰り返しで並べる

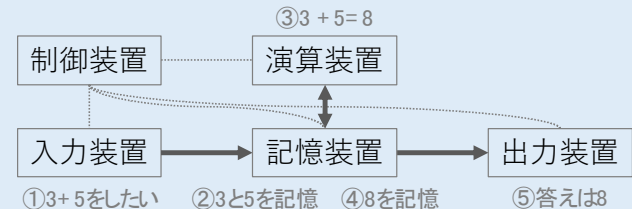
- ・ 逐次実行：順番に処理を実施する（■ → ■）
- ・ 条件分岐：条件に応じて処理を分岐する（◆）
- ・ 繰り返し：条件を満たすまで処理を繰り返す（◆で矢印をループさせる）



計算機の持つ機能は少ないが，組み合わせて色々な機能を作れる

## 1. 計算機の構造と自動計算の原理

- 5種類の装置で構成される
- 装置が連携して計算を行う

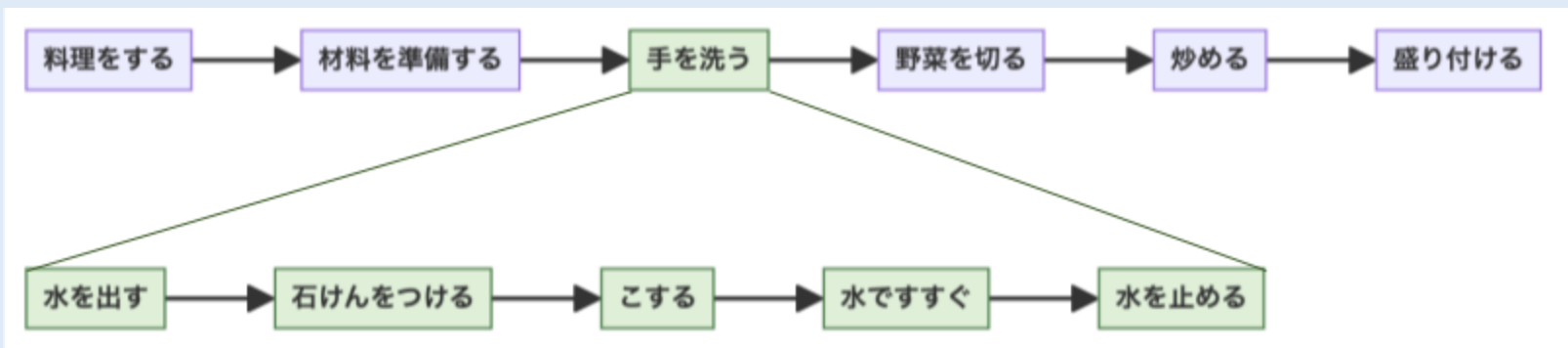


## 2. ハードウェア実装される演算は数種類のみ

- 機械語（2進数），アセンブリ言語（機械語直訳），プログラミング言語

## 3. 演算の組み合わせ（アルゴリズム）で多様な演算を実現

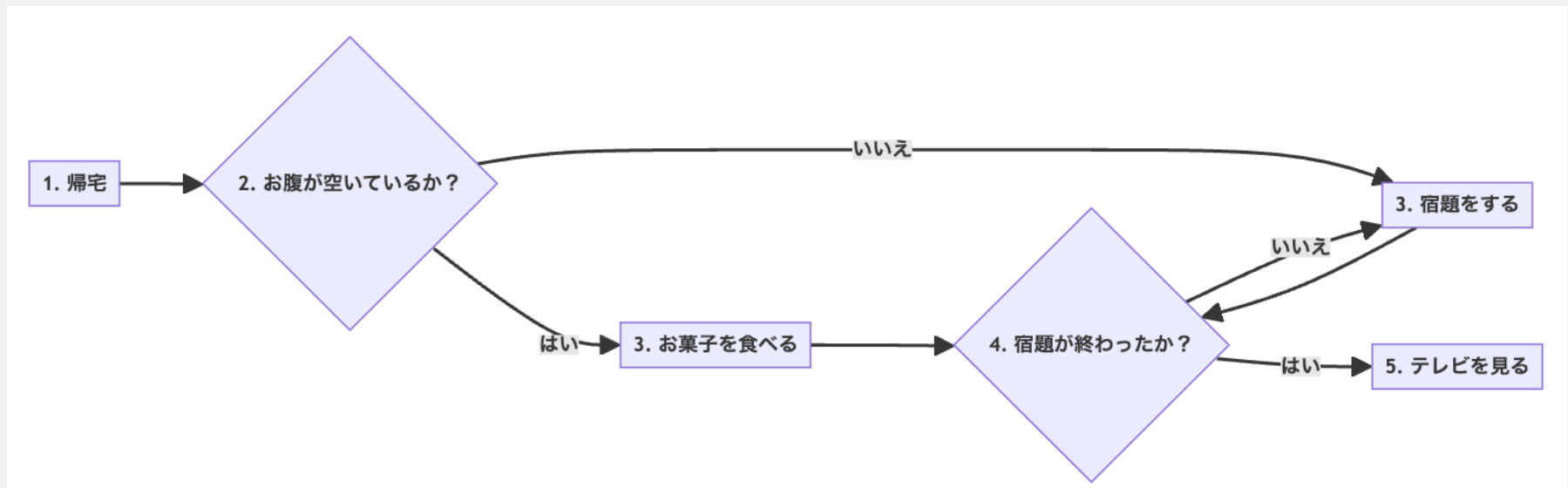
- 命令を逐次処理，分岐，繰り返しに並べてアルゴリズムを作る
- アルゴリズムを組み合わせでアルゴリズムをつくる



## 問題を解決するための具体的な手順 (計算方法)

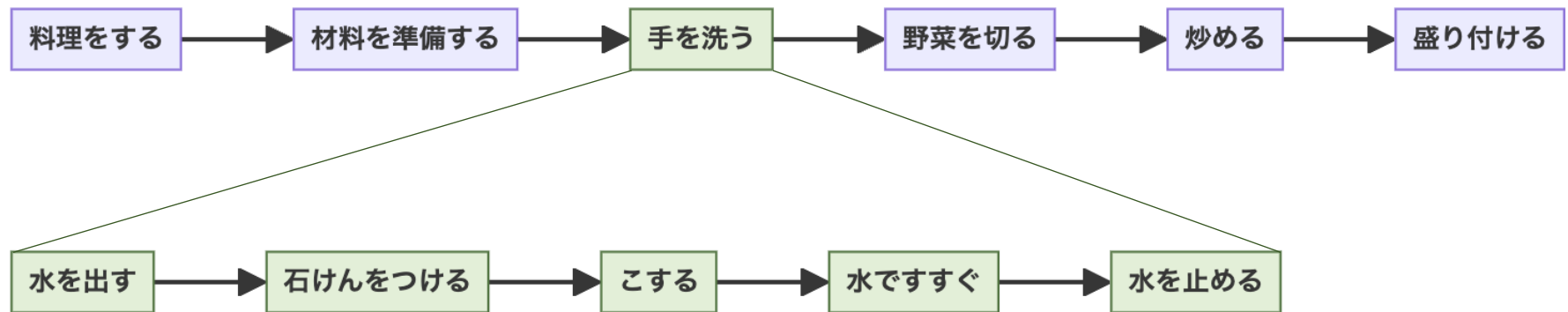
語源：昔に計算方法に関する本を書いた数学者 (al-Khwarizmi氏)

**命令**を逐次実行, 条件分岐, 繰り返しで並べることで作る



CPUに用意されている命令は少ないが  
どのようにして複雑な問題解決を記述するのか？

- アルゴリズムは「命令」を組み合わせた計算手順
- アルゴリズムの中で、別のアルゴリズムを1つの「命令」として使える
- これを繰り返すと、欲しい機能をCPUの動作まで還元せずに記述できる



# 2025年度 コンピュータシステム

## 第4回：計算機と統計

限られた機能しかないCPUで遂行可能な手順書を作ること

CPUの演算は基本的に2つの入力情報のみが対象で、全体を見れない

例) +, -, =, >, <, and, or, ...

人間は全体をみて「なんとなく」問題が解けてしまう

例) 店長は「このくらいの気候なら牛乳が売れそうだ」と直感的に仕入れできる

→ 具体的な計算手順がわからない

1. 来週の天気を調べて影響しそうな情報を探す
2. 過去の似た天気のときの売り上げデータを調べる
3. 取り出した売り上げデータで特に売れている商品を見極める

小さい範囲の情報しか注目できないCPUを使って

全体を考慮した複雑な処理を行わせる方法を考えなければならない

人間が「なんとなく」行う思考を計算手順として書くのに役立つ技術

**統計：人間の直感的な判断を再現するような数式**

- データの傾向を要約する（統計量）
- 未来の結果を予測する（回帰分析）
- 仮説の妥当性を確認する（検定）

**機械学習：経験則的に良い行動を発見する計算**

- 人の作ったデータから計算手順を逆算する（教師あり学習）
- データから未知のパターン・特徴を発見する（教師なし学習）
- 試行錯誤の中でうまい振る舞い方を発見する（強化学習）

**大規模言語モデル：言語を用いた思考を再現する計算**

- 言葉を自然に連ねることで情報を整理する（翻訳・対話・推論）
- 計算手順の中に曖昧な言語処理を組み込む（ワークフロー）
- 検索や計算機を駆使して現実世界で行動する（エージェント）

人間が「なんとなく」行う思考を計算手順として書くのに役立つ技術

**統計：人間の直感的な判断を再現するような数式**

- データの傾向を要約する（統計量）
- 未来の結果を予測する（回帰分析）
- 仮説の妥当性を確認する（検定）

**機械学習：経験則的に良い行動を発見する計算**

- 人の作ったデータから計算手順を逆算する（教師あり学習）
- データから未知のパターン・特徴を発見する（教師なし学習）
- 試行錯誤の中でうまく振る舞い方を発見する（強化学習）

**大規模言語モデル：言語を用いた思考を再現する計算**

- 言葉を自然に連ねることで情報を整理する（翻訳・対話・推論）
- 計算手順の中に曖昧な言語処理を組み込む（ワークフロー）
- 検索や計算機を駆使して現実世界で行動する（エージェント）

## 人間の直感的な判断を再現するような数式

**統計量**：データの傾向を要約した数値

例) 平均値, 偏差値, 画像特徴量

**回帰分析**：データを近似する関数を作り, データのない箇所を予測する方法

例) 気温 (x軸) とアイスの売れ行き (y軸) の関係を示す直線

**検定**：観測データをとって仮説の妥当性を確率的に判定する枠組み

例) 薬が病に有効かどうかの確認

## データの傾向を要約する

例1) 平均値：データの**中間**的な値

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} \quad (\text{算術平均})$$

例2) 分散：データの**ばらつき**を表すための指標

平均値から離れたデータが多いほど大きく、近いデータが多いほど小さくなる

$$s_x^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

例3) 標準偏差：分散の二乗を打ち消して直感的に把握しやすくした値

$$s = \sqrt{s^2}$$

例4) 共分散・相関係数：2つの変数がどの程度連動しているかを表す値

$$s_{xy} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad r = \frac{s_{xy}}{s_x s_y}$$

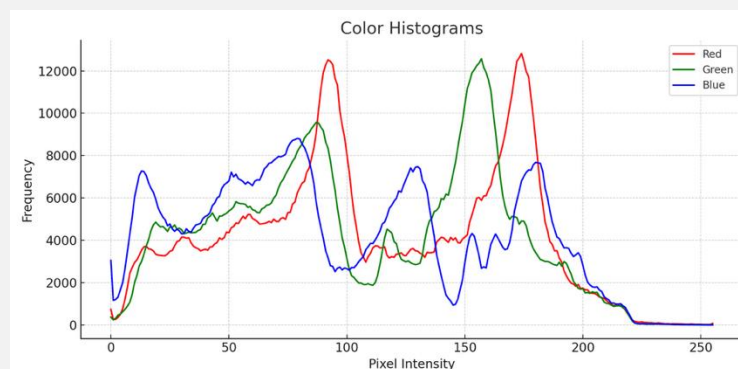
## 例1) 偏差値

- 集団の中での位置を示すのに役立つ標準的な指標
- 平均点を中心に綺麗な山状の分布であることを前提として、平均が50、標準偏差が10となるようにデータの分布を調整して導出される

$$\text{偏差値} = 10 \times \frac{\text{点数} - \text{標準偏差}}{\text{標準偏差}} + 50$$

## 例2) 画像特徴量

- 画像から意味のある情報を数値として抽出したもの
- 明るさの分布形状、全体の乱雑さなど



|       | Mean   | Std Dev | Entropy |
|-------|--------|---------|---------|
| Red   | 111.87 | 54.37   | 5.23    |
| Green | 107.26 | 53.57   | 5.22    |
| Blue  | 98.93  | 58.24   | 5.29    |

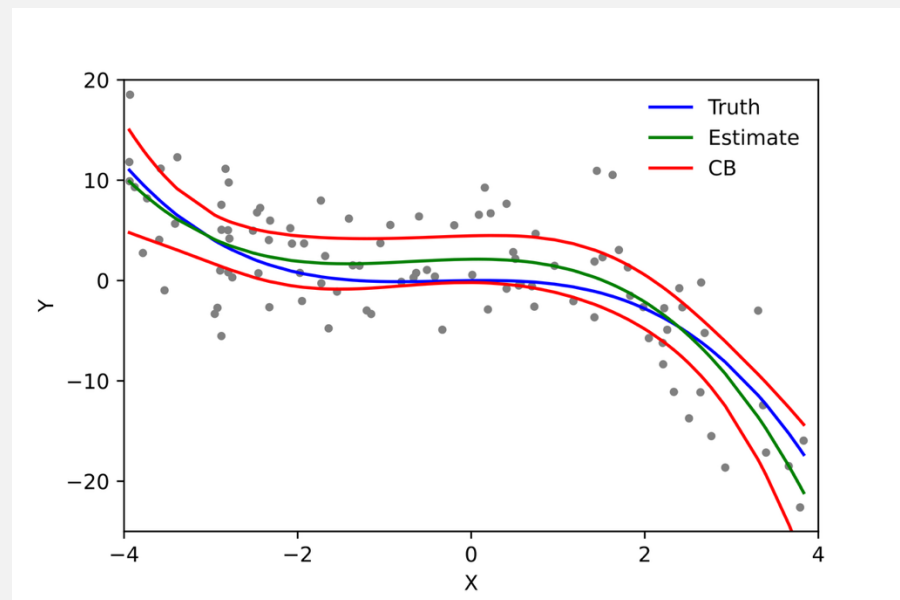
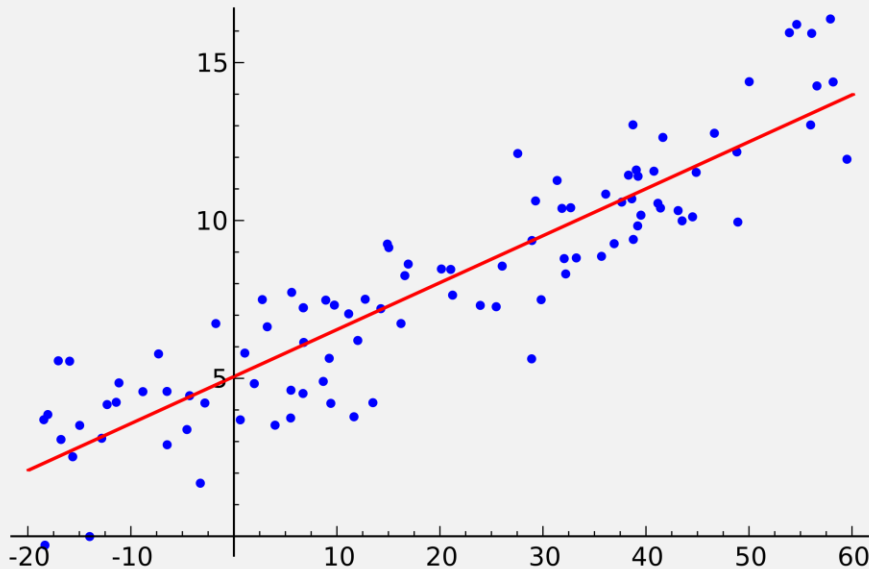
データを近似する関数を作り，データの無い箇所を予測する

例1) 線形回帰：データの特徴を直線で近似する

- $y = Ax + B$

例2) 多項式回帰：データの特徴を多項式で近似する

- $y = a + bx + cx^2 + \dots + kx^n$



## 観測データをとって仮説の妥当性を確率的に判定する枠組み

ある仮説が正しいと仮定した状態で観測データをとって、  
その結果が異常な外れ値を示したら、最初の仮説が疑わしいと判断する  
→「薬が効かない」ことが疑わしいデータがあるなら、「薬は効く」と考えるのが妥当

### 具体例) 薬の有効性

1. 示したい仮説（薬が病に効く）の否定（薬が病に効かない）を用意する
2. 治験で薬を投与された群/されなかった群の症状変化をデータにとる
3. 仮説の否定を前提とすると、めったに生じない違いが現れているならば、「仮説の否定は疑わしい」と判断する
4. 仮説の否定が疑わしいので、仮説を支持するのが妥当と判断する

限られた機能しかないCPUで遂行可能な手順書を作ることは難しい

CPUの演算は基本的に2つの入力情報のみが対象で、全体を見れない

例)  $+$ ,  $-$ ,  $=$ ,  $>$ ,  $<$ , and, or, ...

人間は全体をみて「なんとなく」問題が解けてしまう

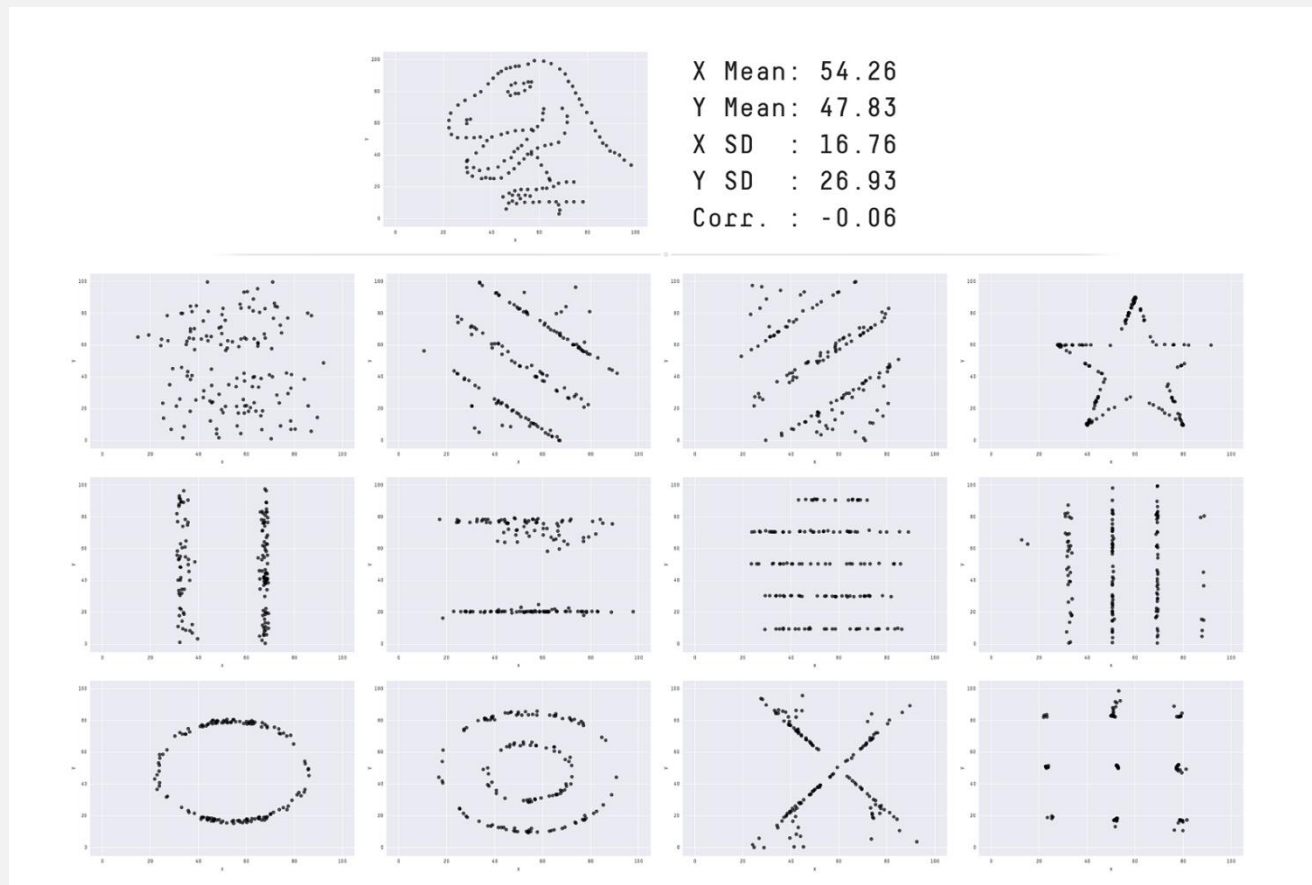
例) 店長は「このくらいの気候なら牛乳が売れそうだ」と直感的に仕入れできる

→ 具体的な計算手順がわからない

1. 来週の天気を調べて影響しそうな情報を探す → 例：相関
2. 過去の似た天気のときの売り上げデータを調べる → 例：分散の大きい変数を抽出
3. 取り出した売り上げデータで特に売れている商品を見極める → 例：回帰分析

## 統計だけを見て，データを見ないと誤った判断をすることがある

- 統計は全体の情報を小さな情報に変換して，計算をしやすくする
- しかし，数個の統計的特性のみを見ると，全体の特性を見誤ることがある



ライセンス未確認  
教育目的の引用

平均値，分散，相関の等しい異なるグラフ

<https://www.research.autodesk.com/publications/same-stats-different-graphs/> より引用

人間が「なんとなく」行う思考を計算手順として書くのに役立つ技術

統計：人間の直感的な判断を再現するような数式

- データの傾向を要約する（統計量）
- 未来の結果を予測する（回帰分析）
- 仮説の妥当性を確認する（検定）

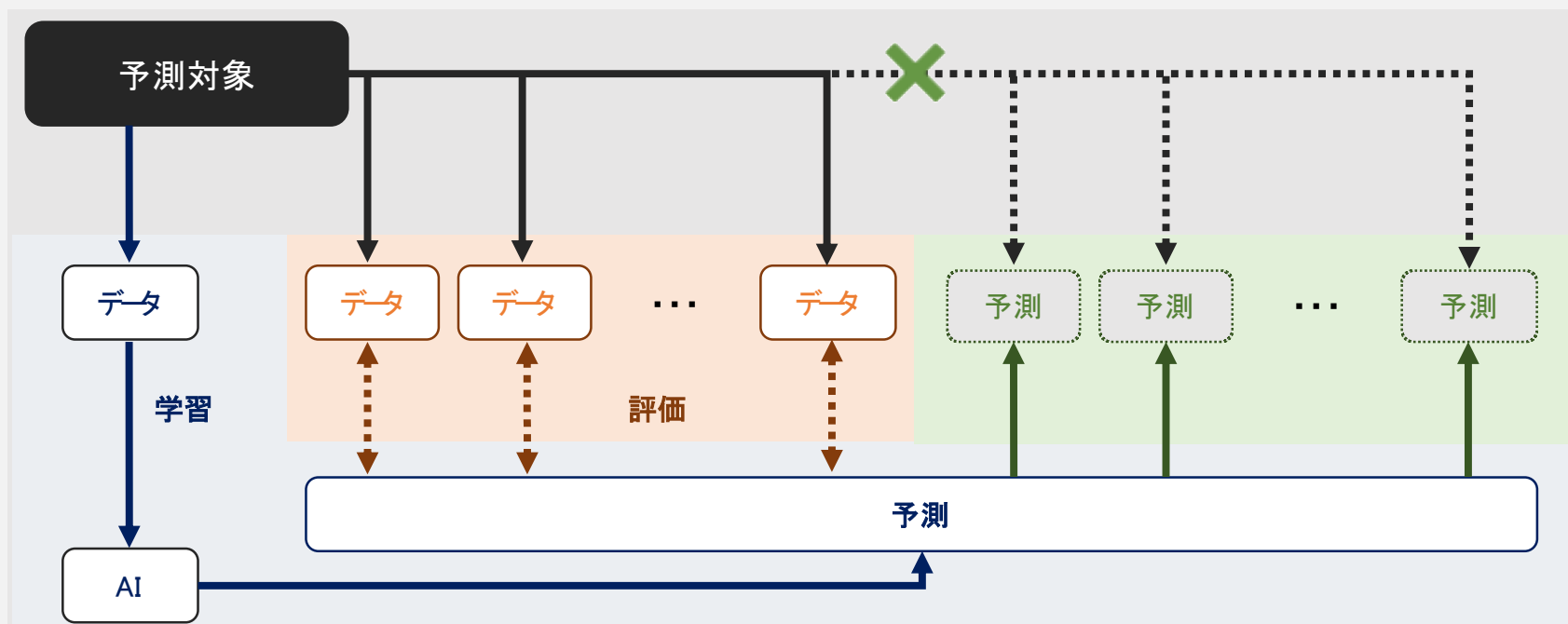
機械学習：経験則的に良い行動を発見する計算

- 人の作ったデータから計算手順を逆算する（教師あり学習）
- データから未知のパターン・特徴を発見する（教師なし学習）
- 試行錯誤の中でうまく振る舞い方を発見する（強化学習）

大規模言語モデル：言語を用いた思考を再現する計算

- 言葉を自然に連ねることで情報を整理する（翻訳・対話・推論）
- 計算手順の中に曖昧な言語処理を組み込む（ワークフロー）
- 検索や計算機を駆使して現実世界で行動する（エージェント）

1. 過去のデータに現れるパターンを自動抽出する
2. 抽出したパターンが未知のデータにも有効か確認する
3. 抽出したパターンを活用して，未知の事象を予測する



I. 学習段階  
(データからパターンを学習)

II. テスト段階  
(獲得パターンが学習外の事象にも有効か確認)

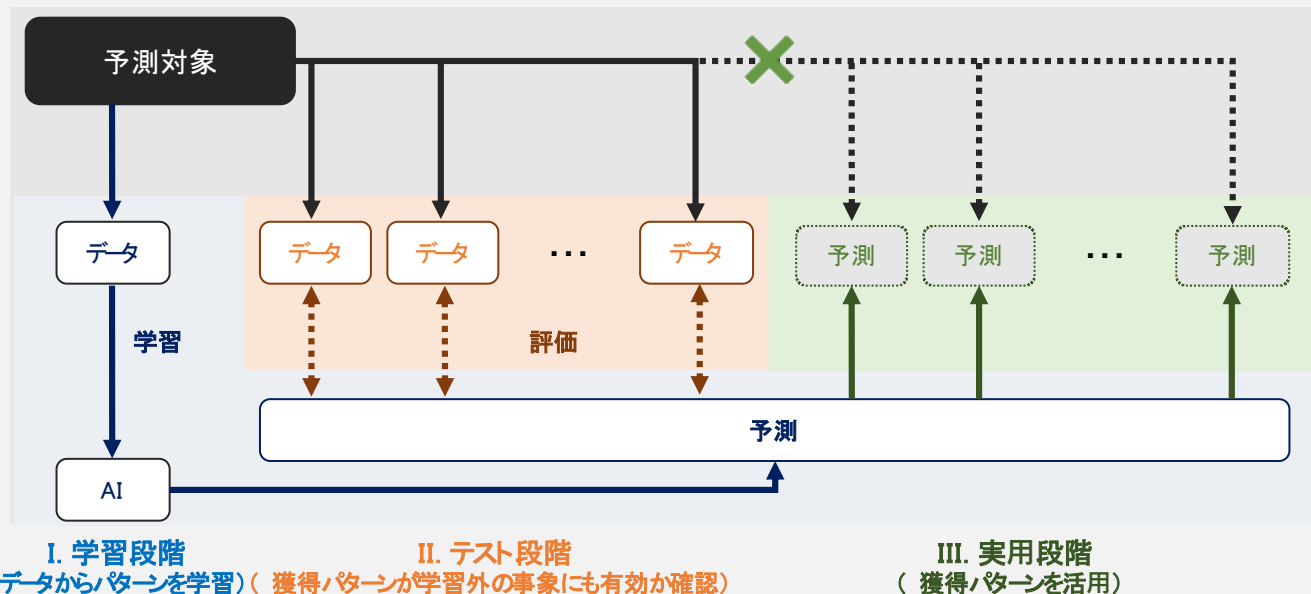
III. 実用段階  
(獲得パターンを活用)

## 経験則的な良い行動の発見を再現する計算手順

- 教師あり学習 : 人の作ったデータから計算手順を逆算する
- 教師なし学習 : データから未知のパターン・特徴を発見する
- 強化学習 : 試行錯誤の中でうまく振る舞い方を発見する

## 人の作ったデータから計算手順を逆算する

入力データと正解データの対応パターンを抽出して、未知の入力に適用



## 例1) 画像分類

- 猫と犬の画像を大量に学習して、それらを区別できるような関数を求める

## 例2) 株価予測

- 過去一年間の株価変動から、一時間後の株価を予測する数式を求める

## 例3) 画像のノイズ除去

- ノイズ付き/無しの画像から、画像のノイズを除去するアルゴリズムを求める

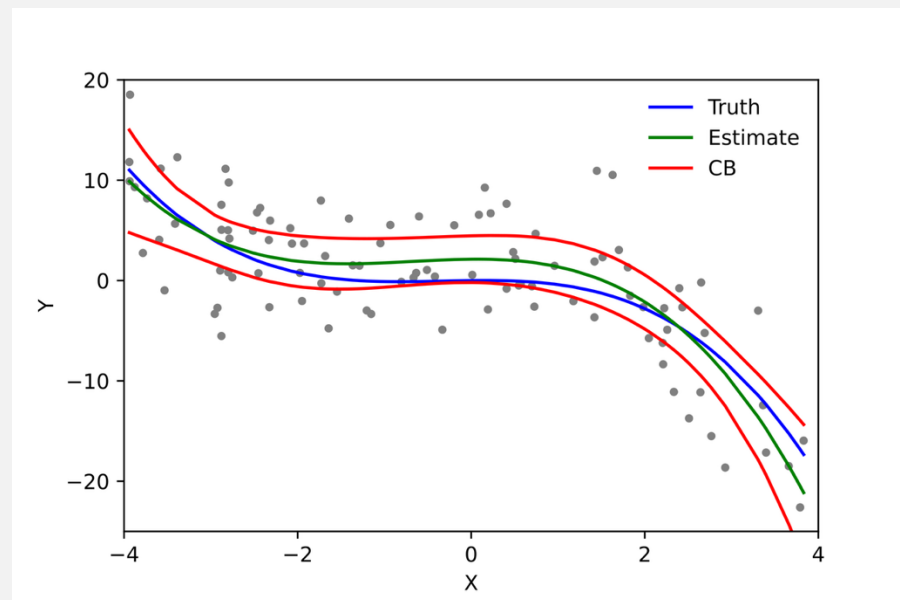
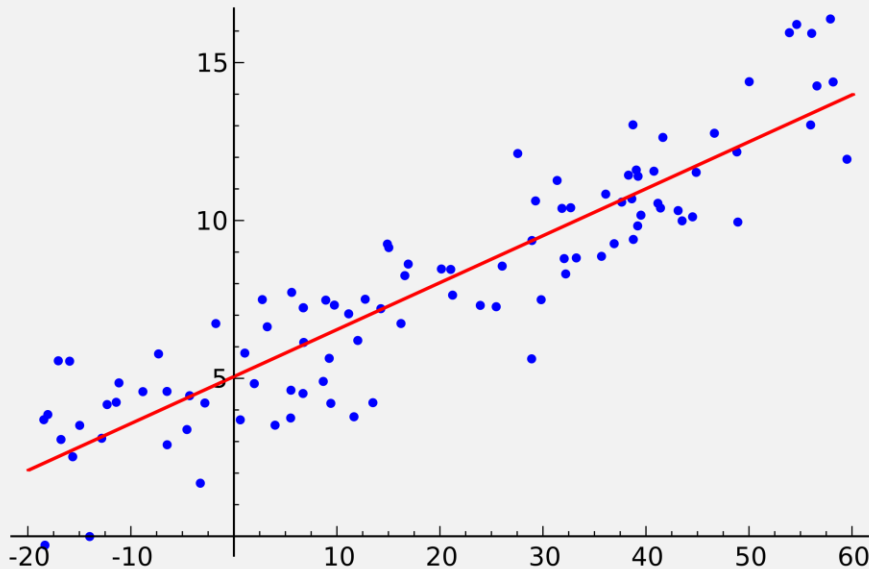
データを近似する関数を作り，データの無い箇所を予測する

例1) 線形回帰：データの特徴を直線で近似する

- $y = Ax + B$

例2) 多項式回帰：データの特徴を多項式で近似する

- $y = a + bx + cx^2 + \dots + kx^n$



## AIの予測と正解データの差が小さくなるように学習する

### モデル：模型，AIの動作を模した数式のこと

- 調整次第で様々な関数の動作を再現できる数式
- 具体的には，内部に調整可能な変数を大量にもつ関数が用いられる

$$\begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{n1} & \cdots & w_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$$

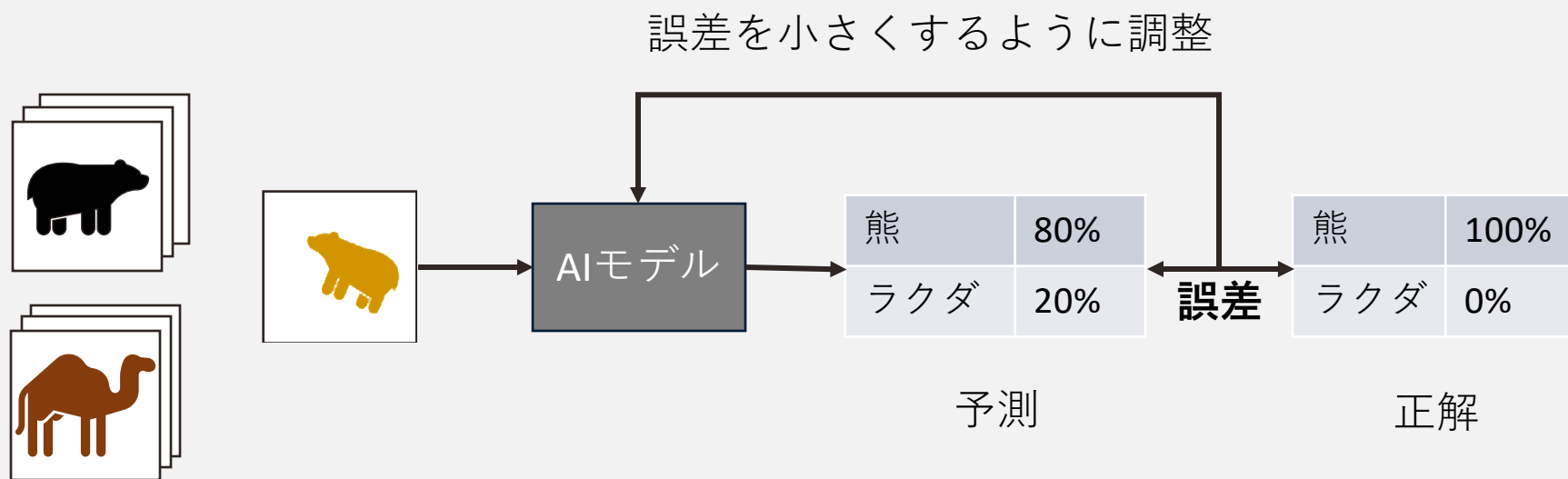
### 損失関数：モデルの内部変数を調整するために用いられる指標

- 現在のモデルの応答と正解データの距離を表す式
  - 例1) 最小二乗誤差 : 出力値と正解値の距離（絶対値の二乗）の平均
  - 例2) クロスエントロピー誤差 : 画像分類などで，予測確率と正解のずれ量を表す値

## ラベル付きの大量の画像を用意して学習

1. 正解のわかる画像を大量に用意
2. 画像を入力してAIの予測を確認
3. 予測が正解に近づくように（＝誤差が小さくなるように）AIモデルの内部変数を調整

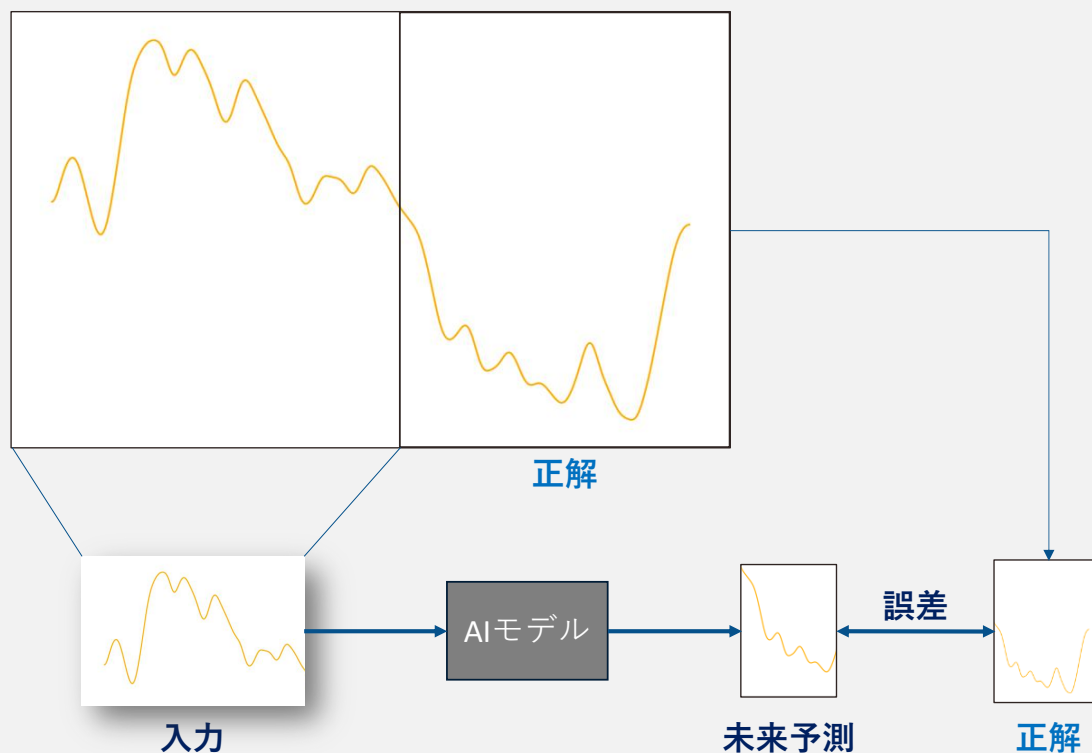
→ 学習データに含まれているパターンを持つ未知の画像も識別可能になる  
（例：学習画像の熊に似た色・形の動物は、熊の確率が高いと判定）



## 株価推移の記録から、変化の仕方のパターンを学習

1. 過去の株価情報を用意して、前半部をAI入力し、その先を予測させる
2. AIの予測と後半部の誤差を計算して、それを縮小するように調整する

→過去の株価情報と類似した変化をするならば、  
現在の株価推移を入れると高精度で未来の株価を予測できる



粗い画像を入れたとき綺麗な画像に近い画像を出力するように学習

1. 同じ被写体の画像ペアを大量に用意
2. 粗い画像を入力したときのAIの出力を確認
3. 出力画像と高解像度画像の誤差が小さくなるようにAIモデルを調整

→ 学習した高解像化のパターンで対応できる「粗い画像」ならば  
AIモデルを通すことで綺麗な画像に変換することができる

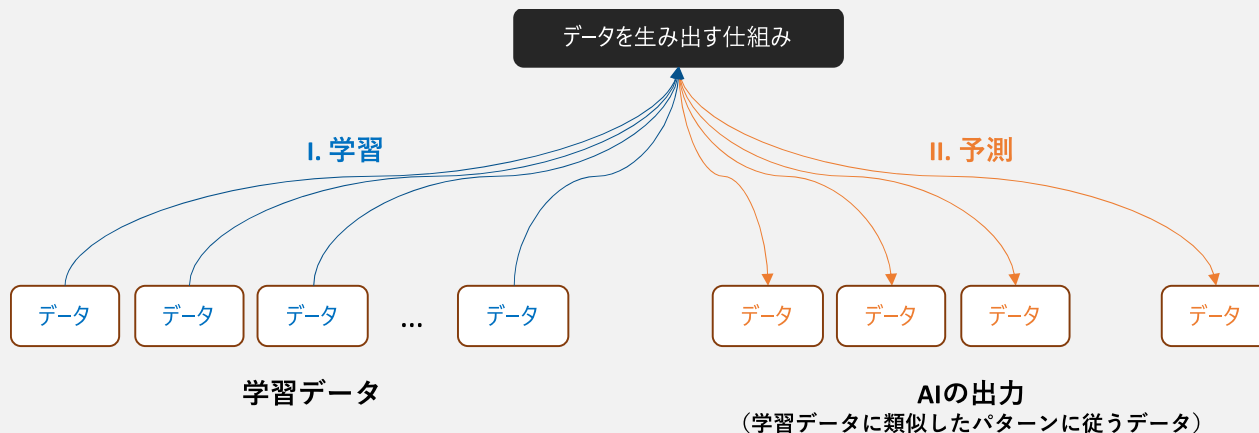


## 経験則的な良い行動の発見を再現する計算手順

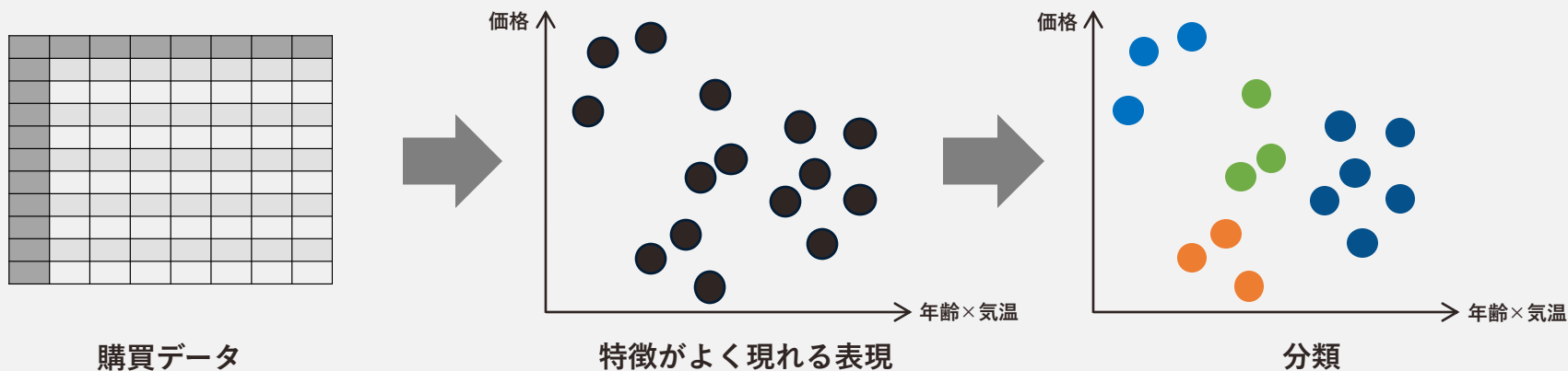
- 教師あり学習 : 人の作ったデータから計算手順を逆算する
- 教師なし学習 : データから未知のパターン・特徴を発見する
- 強化学習 : 試行錯誤の中でうまく振る舞い方を発見する

## データから未知のパターン・特徴を発見する

## 例1) 学習データと同じようなデータを生み出せる仕組みの構築



## 例2) 学習データの構造を抽出



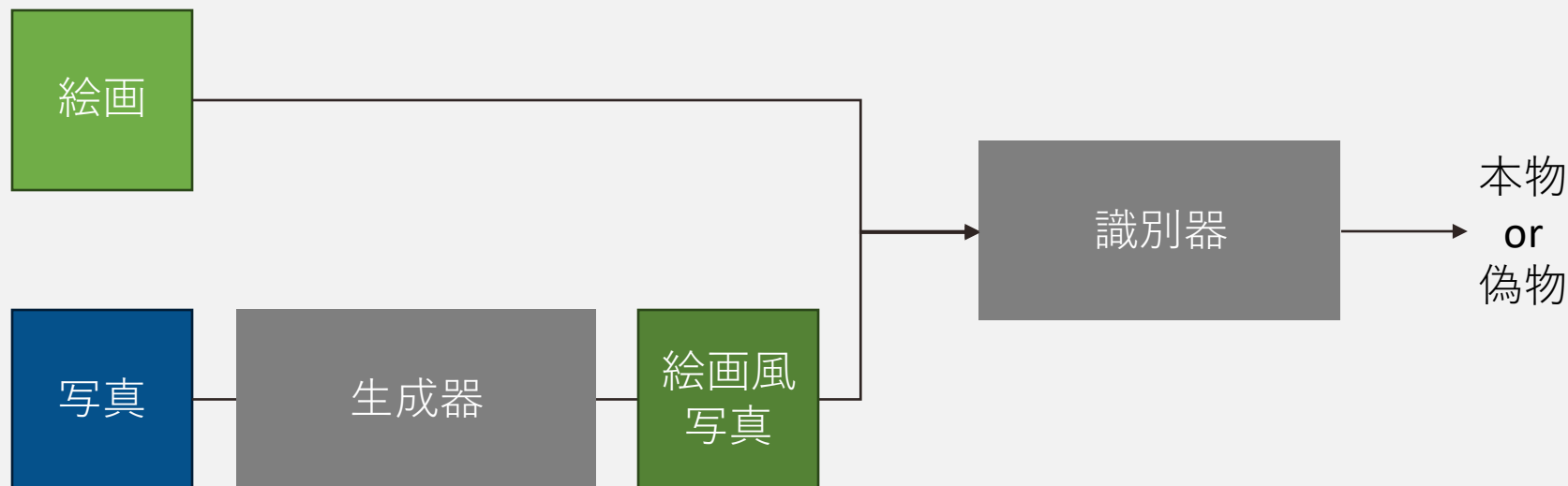
## 敵対的生成ネットワーク（GAN）

生成器（比喻：偽札犯）と識別器（比喻：警察）を競わせるように学習

- ・ 生成器：画像を絵画風に変換して出力
- ・ 識別器：本物の絵画と生成器が写真から作った画像を識別

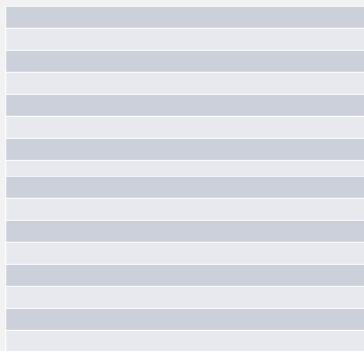
### 基本的な考え方

- ・ 識別器が見破れたら、識別器を騙せるように生成器を修正
- ・ 識別器が騙されたら、次から騙されないように識別器を修正

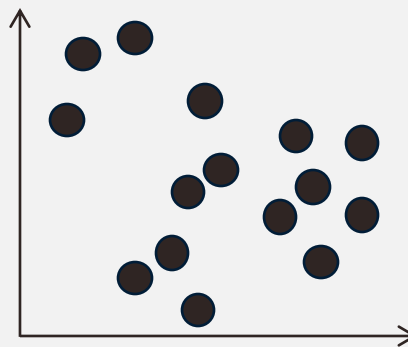


顧客の声や市民の声など、膨大なテキストデータの把握や活用を補助

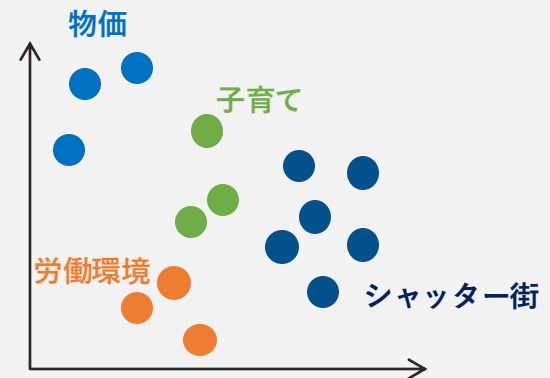
1. 膨大な意見を似た特性を持つ複数のグループに分類
2. 各グループの意見を言語処理AIで要約
3. 似たグループが近くに来るように2次元平面にプロット
4. 人間は、要約されたデータで概形を理解しながら、個別の意見も閲覧できる



収集した意見  
(大量のテキストデータ)



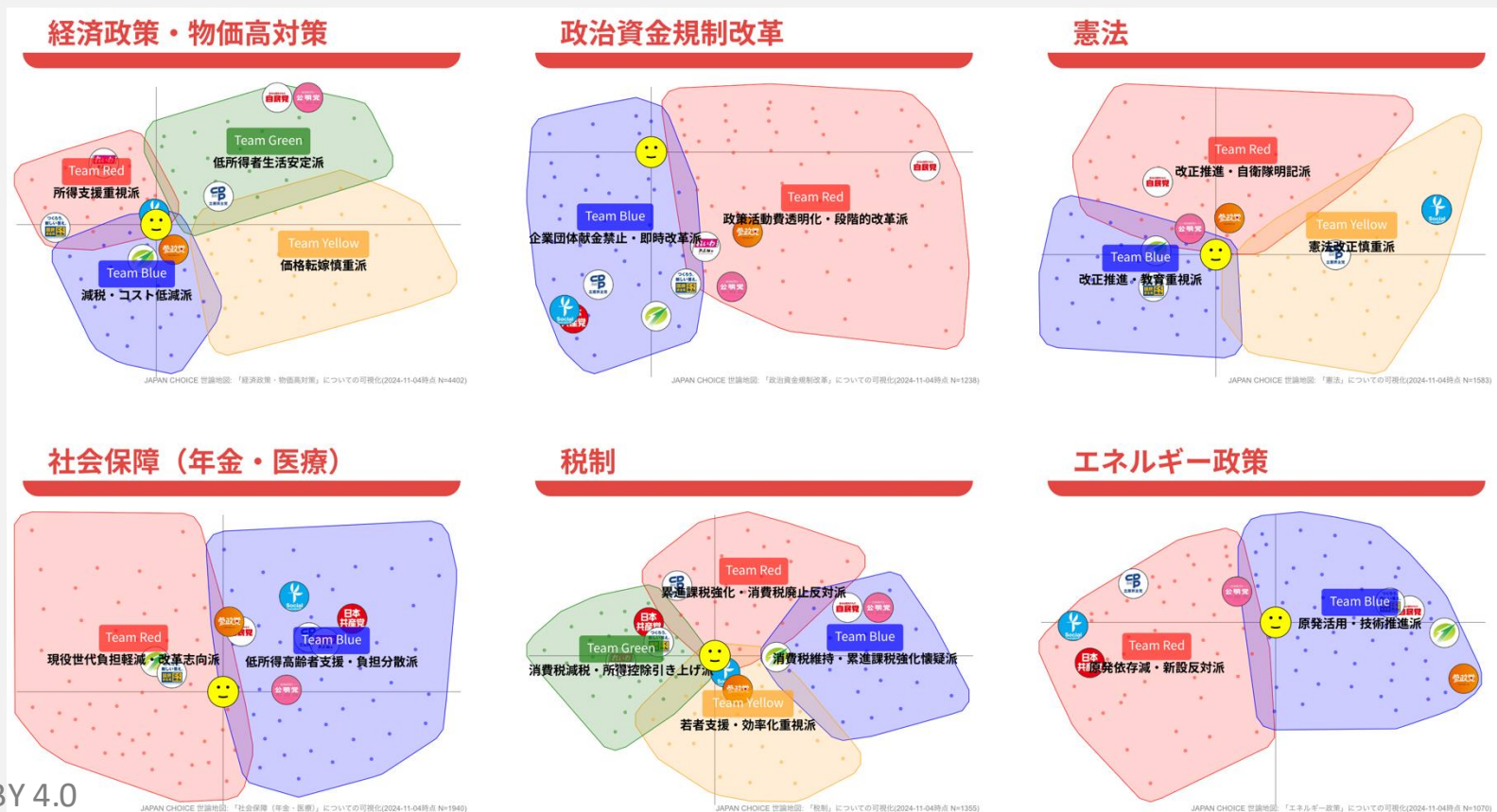
意見の“距離”や“方向”を  
空間上に表現



グループ化と要約

顧客の声や市民の声など，膨大なテキストデータの把握や活用を補助

1. 膨大な意見を似た特性を持つ複数のグループに分類
2. 各グループの意見を言語処理AIで要約
3. 似たグループが近くに来るように2次元平面にプロット
4. 人間は，要約されたデータで概形を理解しながら，個別の意見も閲覧できる



## 経験則的な良い行動の発見を再現する計算手順

- 教師あり学習 : 人の作ったデータから計算手順を逆算する
- 教師なし学習 : データから未知のパターン・特徴を発見する
- 強化学習 : 試行錯誤の中でうまい振る舞い方を発見する

## 試行錯誤の中でうまく振る舞い方を発見する

### 例1) 当たるスロットマシンの探索, 空いている電波チャンネルの探索

1. ランダムな確率に候補（マシンやチャンネル）を選ぶ
2. 結果が良ければその候補を選ぶ確率を次回以降は上げる, 悪ければ下げる
3. この方法で, より候補を見つけられて, 状況変化にも対応できる

## 試行錯誤の中でうまく振る舞い方を発見する

### 例1) 当たるスロットマシンの探索, 空いている電波チャンネルの探索

1. ランダムな確率に候補（マシンやチャンネル）を選ぶ
2. 結果が良ければその候補を選ぶ確率を次回以降は上げる, 悪ければ下げる
3. この方法で, より候補を見つけられて, 状況変化にも対応できる

### 例2) ゲームをクリアする

1. 生存時間や勝敗など, 基準となる報酬を設定する
2. 各場面でランダムな行動をして, その履歴と結果を記録する
3. 結果に応じて, 各場面での行動に点数をつける
4. 点数の高い行動を多く取りながら, より高い報酬が得られる行動を探索する

### 例3) 数学の問題を解く, 人間を満足させる会話をする

1. 正解率や人間の評価（👍/👎）など, 指標を決める
2. 訓練したAIモデルを動かす（対話を進める, 数学の次の操作を推測）
3. 結果に応じて, 各場面での行動に点数をつける
4. より高い点数の行動を取れるように, AIの内部変数を更新する

人間が「なんとなく」行う思考を計算手順として書くのに役立つ技術

統計：人間の直感的な判断を再現するような数式

- データの傾向を要約する（統計量）
- 未来の結果を予測する（回帰分析）
- 仮説の妥当性を確認する（検定）

機械学習：経験則的に良い行動を発見する計算

- 人の作ったデータから計算手順を逆算する（教師あり学習）
- データから未知のパターン・特徴を発見する（教師なし学習）
- 試行錯誤の中でうまく振る舞い方を発見する（強化学習）

大規模言語モデル：言語を用いた思考を再現する計算

- 言葉を自然に連ねることで情報を整理する（翻訳・対話・推論）
- 計算手順の中に曖昧な言語処理を組み込む（ワークフロー）
- 検索や計算機を駆使して現実世界で行動する（エージェント）

## 言葉の自然な連なりを作って情報を整理するAI

### 次単語予測を繰り返して文章を完成させる

例) "The color of the apple" → is (92%), of (3%), which (2%)...  
"The color of the apple is" → red (70%), green (20%), not (5%)...

## 言葉の自然な連なりを作って情報を整理するAI

次単語予測を繰り返して文章を完成させる

例) "The color of the apple" → is (92%), of (3%), which (2%)...  
"The color of the apple is" → red (70%), green (20%), not (5%)...

### 応用例1) 翻訳

対応する英語と日本語の文章を並べたデータを学習

"<EN>The color of apple is red. <JP>りんごの色は赤色です"

"<EN>This is a black pencil<JP>"  
→AIの予測："これは黒い..."

### 応用例2) 対話

ユーザー入力と理想的な応答の繰り返し文を学習

<User>こんにちは<AI>こんにちは！

<User>りんごの色は何色ですか？<AI>赤色です

<User>全て赤色ですか？<AI>いいえ..."

### 応用例3) 推論

ユーザーの入力，良い答えのために思考する文章，応答の繰り返し文を学習

<User>りんごの色は何色ですか？

<Think>ユーザーはりんごの色を尋ねている。りんごの色は品種により異なるが...

<Response>一般的には赤色です。しかし，品種により...

## ワークフロー

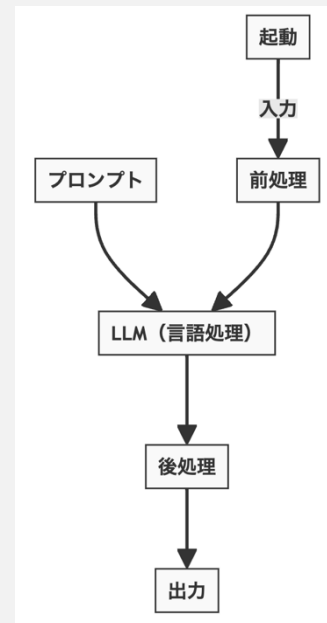
プログラムの中にLLMを組み込む応用手法

例) マニュアルに基づく顧客サポート自動化 (RAG)

1. チャットシステムで顧客が質問を入力
2. 検索システムが社内文書からマニュアルや類似事例を抽出
3. LLMが検索された情報に従って**応答を作成**
4. チャットシステムが顧客に回答を表示

別の例) カレンダーを閲覧して日程調整の答えを提案する

1. メールを受信
2. 受信メールが日程調整に関するものなら、カレンダー情報を取得する
3. 受信メールに書かれた日程とカレンダー情報から**調整可能な日程を推論**する
4. **返信案を作成**してユーザーに通知する

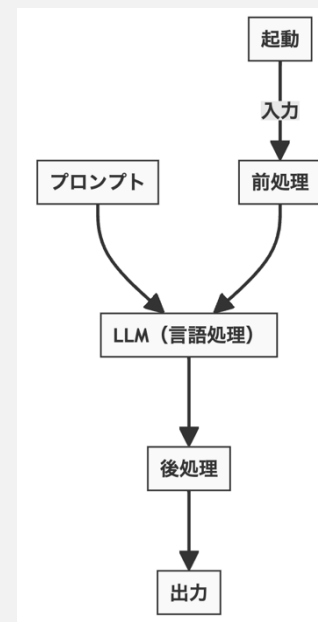


## ワークフロー

プログラムの中にLLMを組み込む応用手法

例) マニュアルに基づく顧客サポート自動化 (RAG)

1. チャットシステムで顧客が質問を入力
2. 検索システムが社内文書からマニュアルや類似事例を抽出
3. LLMが検索された情報に従って応答を作成
4. チャットシステムが顧客に回答を表示

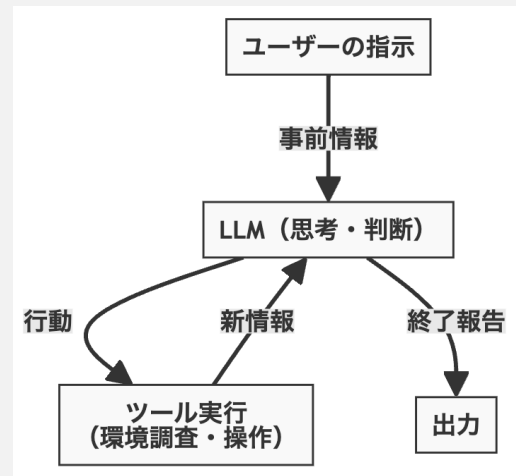


## エージェント

LLMに思考とプログラム実行を交互に行わせて自律的に行動させる手法

例) 検索エージェント (Deep Researchなど)

1. ユーザーが質問を入力
2. LLMが**検索・思考**を繰り返して適切な情報を探索
3. LLMが情報を元に回答を作成



## コーディングエージェント

### LLMがツールを扱えるようになる仕組み

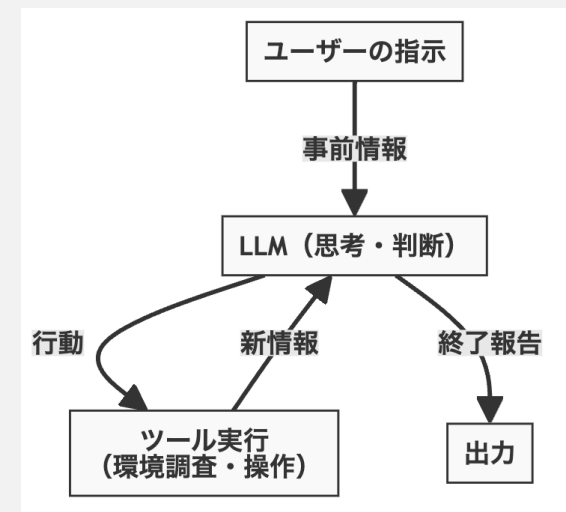
- ・ ツールは決まった形の構造化データとして用途や使い方を書くことにしておく
- ・ LLMは構造化データの形式で情報を埋めることで、目的の情報を得られる

### LLMに渡されるツール例

- ・ ファイルを読み書きするツール
- ・ コマンドを実行するツール
- ・ ブラウザを操作するツール

### 動作の流れ

1. ユーザーからの支持を受け取る
2. 指示を達成するためのToDoリストを作る
3. 各タスクを実行するためにツールを呼び出す
  - ・ ファイルの読み書きやコマンド実行をして目的達成を目指す
  - ・ コマンド実行などの結果から成功/失敗を検証する
  - ・ 成功なら次に進む, 失敗であれば別のアプローチを試す
4. 目標を達成したらレポートを書いて終了する



人間が「なんとなく」行う思考を計算手順として書くのに役立つ技術

**統計：人間の直感的な判断を再現するような数式**

- データの傾向を要約する（統計量）
- 未来の結果を予測する（回帰分析）
- 仮説の妥当性を確認する（検定）

**機械学習：経験則的に良い行動を発見する計算**

- 人の作ったデータから計算手順を逆算する（教師あり学習）
- データから未知のパターン・特徴を発見する（教師なし学習）
- 試行錯誤の中でうまく振る舞い方を発見する（強化学習）

**大規模言語モデル：言語を用いた思考を再現する計算**

- 言葉を自然に連ねることで情報を整理する（翻訳・対話・推論）
- 計算手順の中に曖昧な言語処理を組み込む（ワークフロー）
- 検索や計算機を駆使して現実世界で行動する（エージェント）

# III. 問題を計算機の使える形に整理する

## 5. 情報システムのデザイン

## I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機，インターネット，情報システム

## II. 計算機やデータ，AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

## III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

## IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習（データ構造とアルゴリズム）

## V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

## VI. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

## VII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習（PERT図，決定表）

1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる
3. ITシステムの構造や仕組みを学び、  
**問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる**
4. 計算機のハードウェア構成を知り、目的達成に必要な装置の見積もりや購入ができるようになる
5. 計算機に特有のデータ表現を学び、機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考を体験する

# 2025年度 コンピュータシステム 第5回：情報システムのデザイン

**本当の課題を見極め、役立つITシステムを定義し、実装・検証する**

## 1. 企業がソフトウェアを発注・開発する流れ

- 利用者がやりたいことを整理する
- 発注者が欲しいシステムを定義する
- 開発者が定義通りに動くシステムを作って納入する

## 2. 要件定義の仕方

- 利用者の想像できるシステムが最善の解決策でない可能性に注意する
- 利用者のやりたいことを整理して、それを実現するには何が必要か考える
- 機能を細かく分割して、計算機にできることとすり合わせる

## 3. ソフトウェア開発モデル

- 計画に沿って安全に開発する方法論（ウォーターフォール、V字モデルなど）
- 開発しながら価値を見極める方法論（アジャイル、リーンスタートアップなど）

本当の課題を見極め、役立つITシステムを定義し、実装・検証する

## 1. 企業がソフトウェアを発注・開発する流れ

- 利用者がやりたいことを整理する
- 発注者が欲しいシステムを定義する
- 開発者が定義通りに動くシステムを作って納入する

## 2. 要件定義の仕方

- 利用者の想像できるシステムが最善の解決策でない可能性に注意する
- 利用者のやりたいことを整理して、それを実現するには何が必要か考える
- 機能を細かく分割して、計算機にできることとすり合わせる

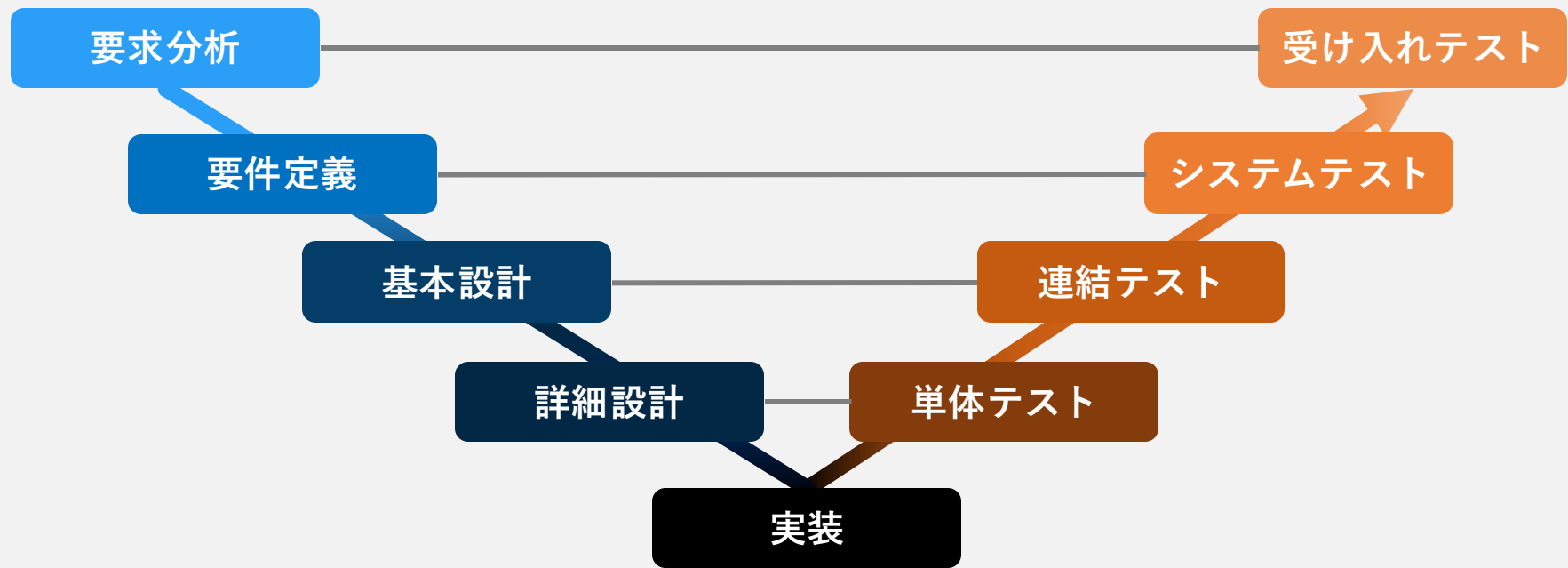
## 3. ソフトウェア開発モデル

- 計画に沿って安全に開発する方法論（ウォーターフォール、V字モデルなど）
- 開発しながら価値を見極める方法論（アジャイル、リーンスタートアップなど）

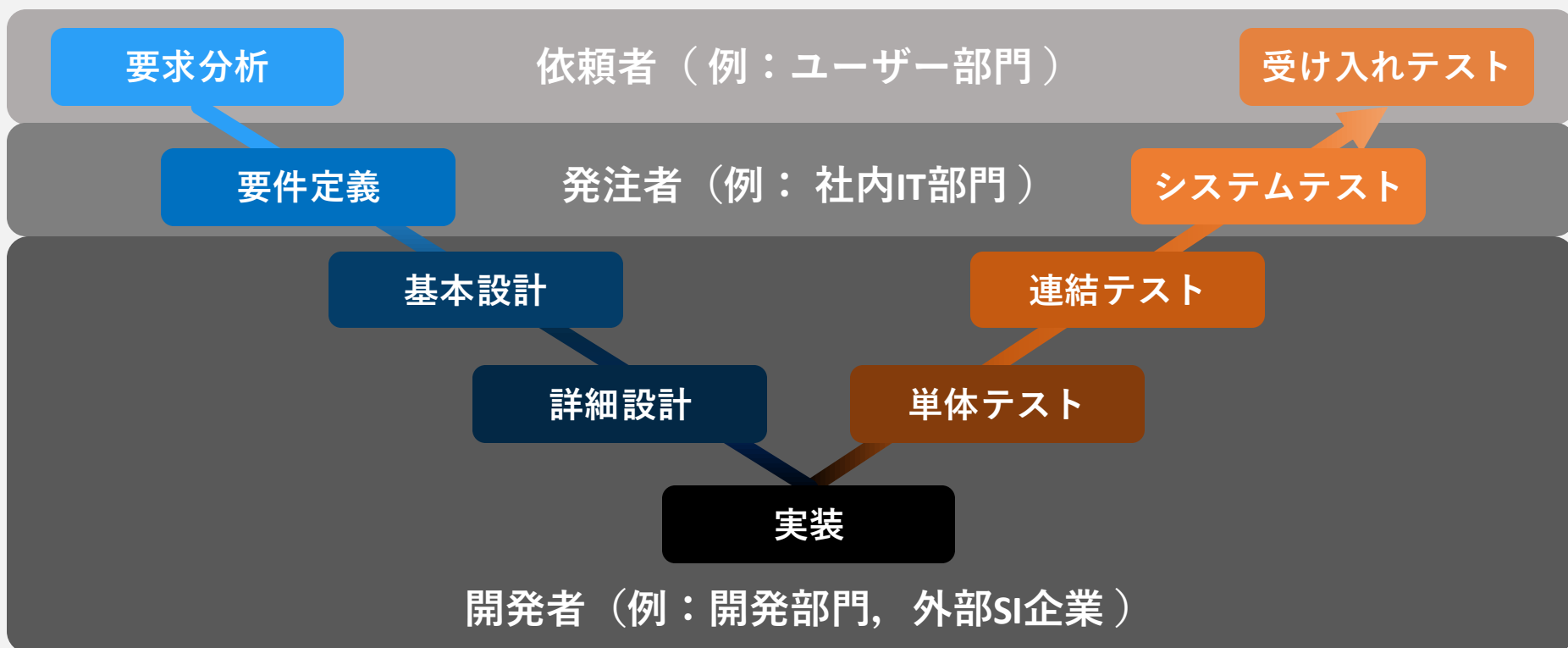
## 情報システム開発に用いられる手法の一つ

「開発→実装→テスト」と段階を踏んで進行していく

- ・ 同じ高さにある段階は「設計」と「確認」の対応関係にある
- ・ プログラミングをするのは「実装」にあたるごく一部分のみ



1. 依頼者は**ソフトウェアを使って価値を生む**責任を負う
2. 発注者は**ITシステムとしての定義**に責任を負う
3. 開発者は**定義通りに動くシステム**を作る責任を負う



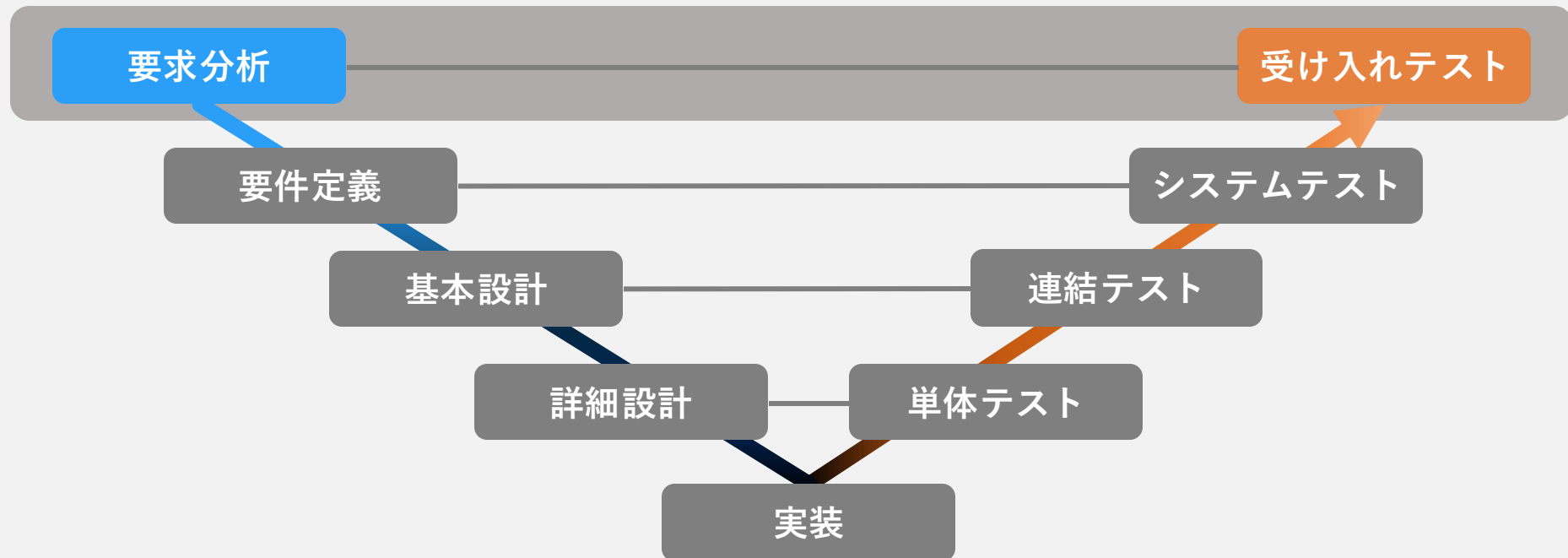
## ソフトウェアを使って価値を生む部分に責任を負う

### 要求分析

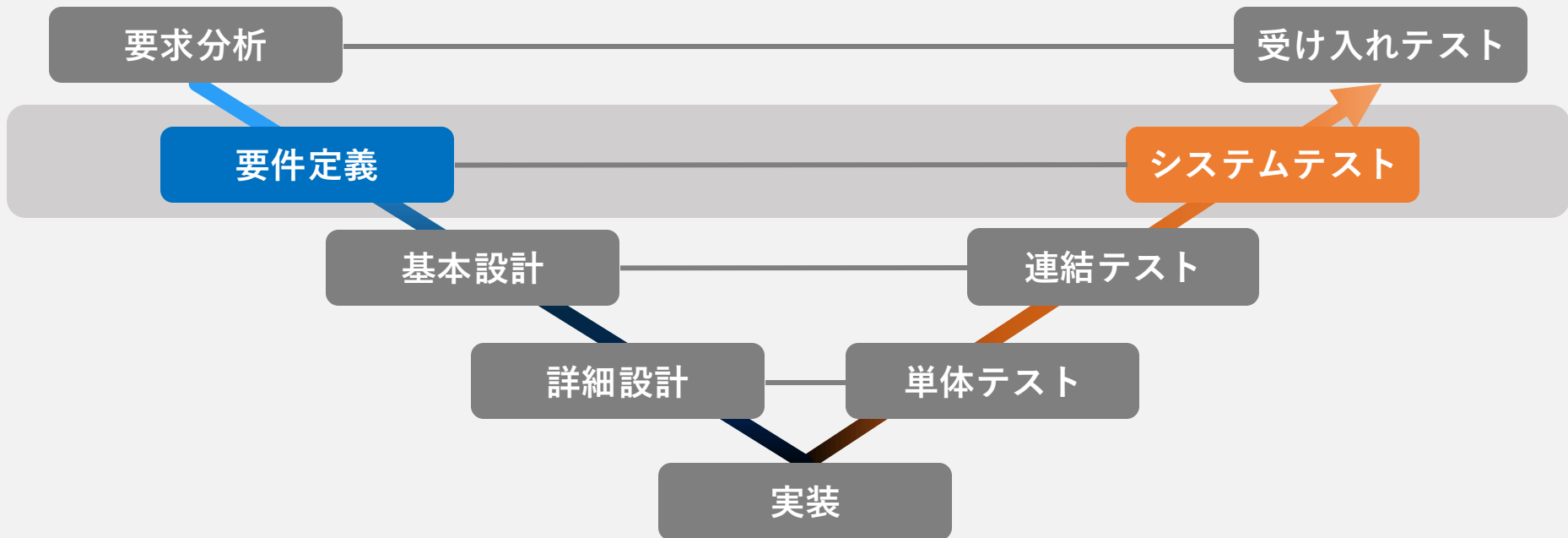
- ・ 業務のために何が必要なのかを明確化
- ・ 潜在的な課題の洗い出し

### 受け入れテスト

- ・ 納品されたソフトウェアが自分たちの要求していた通りの仕事をするか確認

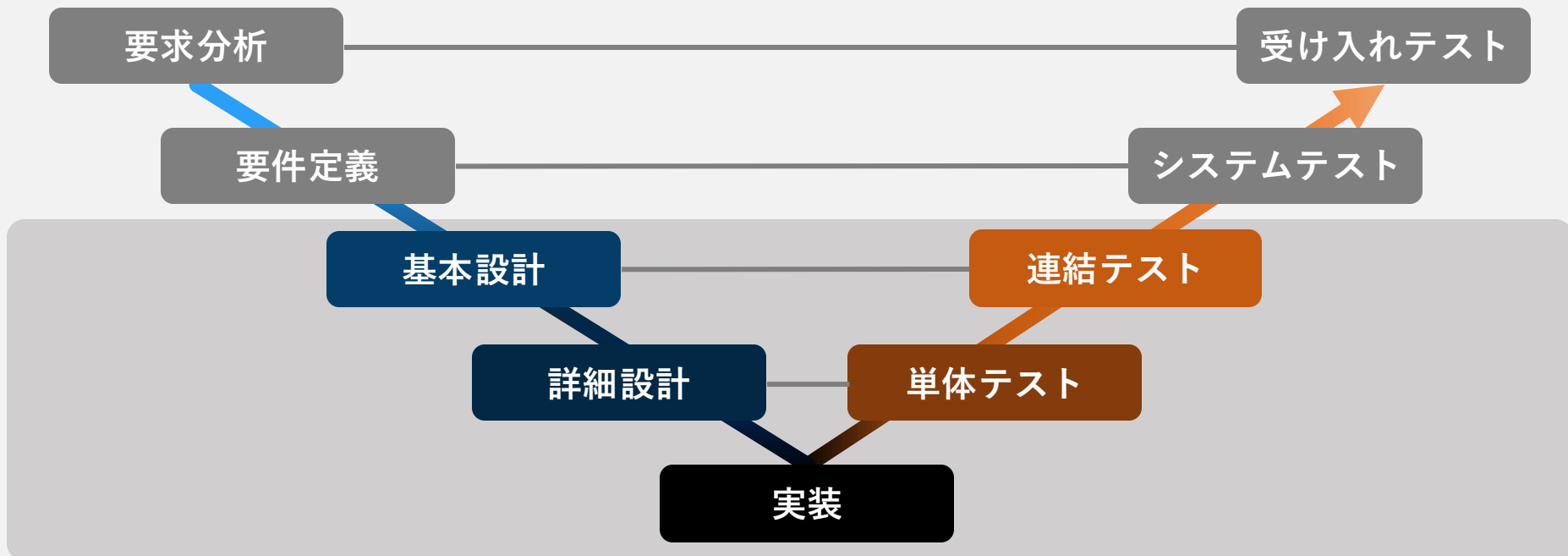


- 要件定義：要求に対してITシステムとして何が実現可能か取捨選択
- システムテスト：何をすれば要求を満たすシステムを作れたことになるか検討
- その他：予算，スケジュール，人員見積もり，コミュニケーション方法の確立



## 定義通りに動くシステムを作る責任を負う

- 基本設計，詳細設計：要件を計算機に実現可能な形に落とし込む
- 実装：プログラミングなど
- 単体テスト，連結テスト：実装が要件を満たしていることを確認する



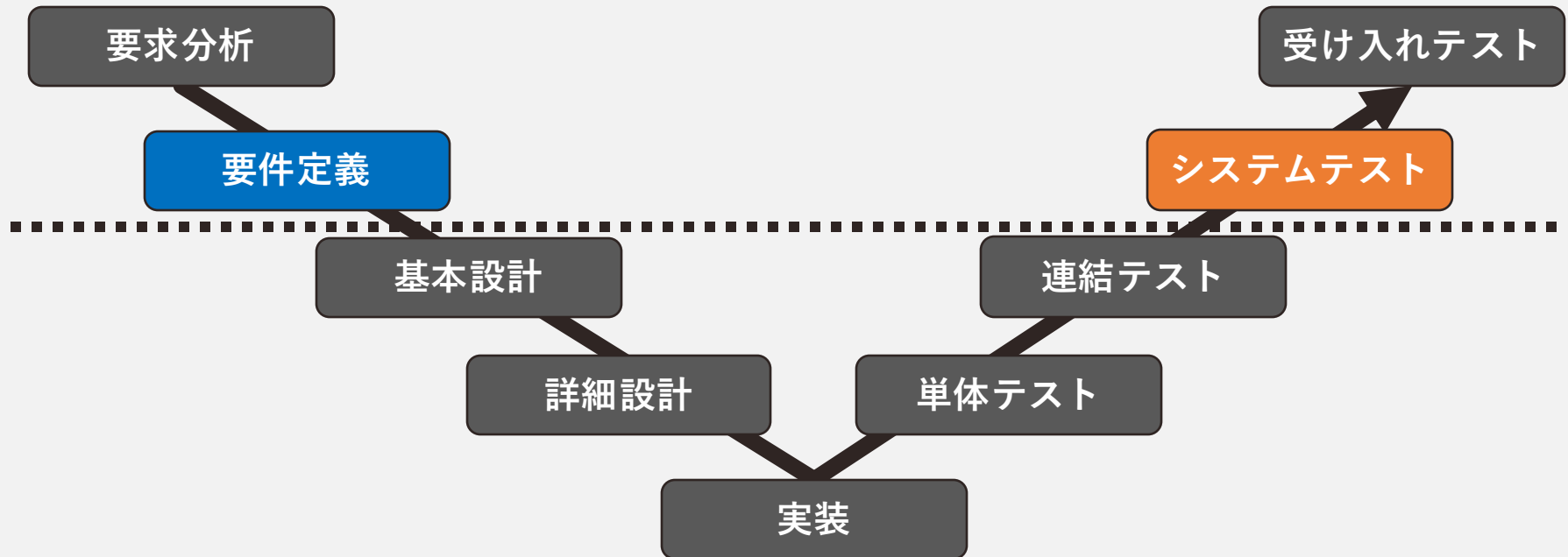
開発企業と発注企業では、仕事の目標が異なる

発注企業の成功：導入されたシステムによる課題解決

→受け入れテストの先にある**利用者への価値提供**がゴール

開発企業の成功：要件定義に沿ったシステムの納入

→システムテストの突破（**発注企業の受け入れ**）がゴール



開発者の仕事は指示されたものを作ることが主であり、それは顧客の成功と同じでない

→正しく要件定義を行わない限り、正しいものは納入されない

本当の課題を見極め、役立つITシステムを定義し、実装・検証する

## 1. 企業がソフトウェアを発注・開発する流れ

- 利用者がやりたいことを整理する
- 発注者が欲しいシステムを定義する
- 開発者が定義通りに動くシステムを作って納入する

## 2. 要件定義の仕方

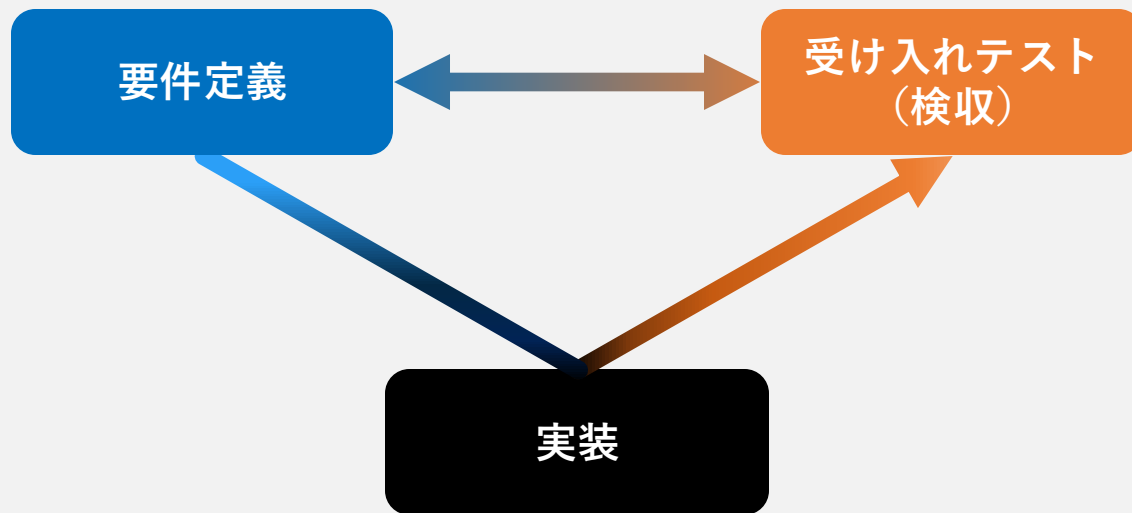
- 利用者の想像できるシステムが最善の解決策でない可能性に注意する
- 利用者のやりたいことを整理して、それを実現するには何が必要か考える
- 機能を細かく分割して、計算機にできることとすり合わせる

## 3. ソフトウェア開発モデル

- 計画に沿って安全に開発する方法論（ウォーターフォール、V字モデルなど）
- 開発しながら価値を見極める方法論（アジャイル、リーンスタートアップなど）

やりたいことを整理して、それを実現するには何が必要か考える

- ・ 小規模な案件や柔軟に顧客にニーズに対応する現代的な開発方式の場合、  
開発企業と発注企業はV字モデルよりも密に連携する
- ・ 実際には、要求分析と要件定義をまとめて「要件定義」とすることが多い



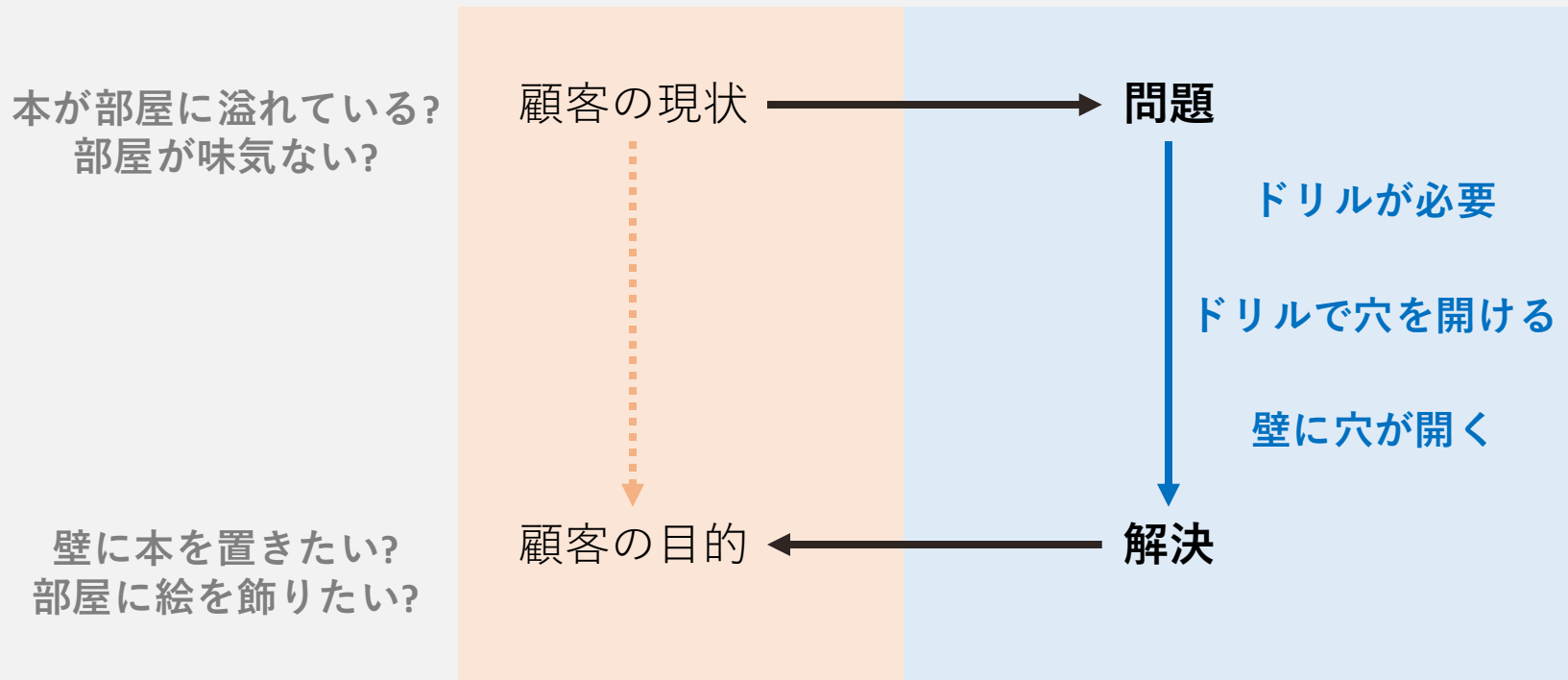
## 顧客にヒアリングして整理することの難しさ

**格言：ドリルを買いに来た人が欲しいのは、ドリルではなく穴である**

製品を提供するときは、顧客の真の目的を考えてサポートすることが重要

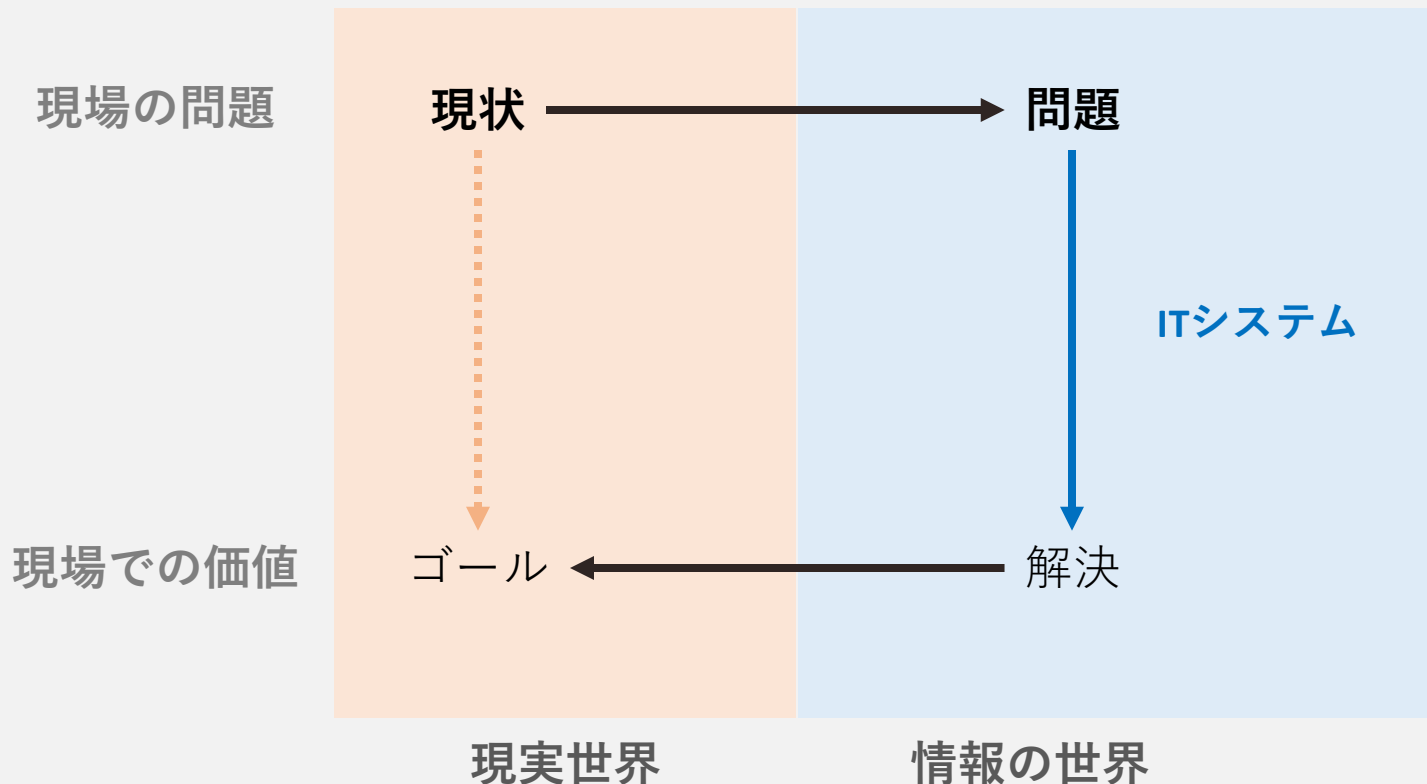
例) 1/4インチのドリルが欲しいと言われたとき、顧客は1/4インチの穴を開けたい。  
その目的は絵を掛けるためかもしれない。そしてその目的は、部屋を美しく飾りたいのかもしれないし、本を置く棚が欲しいのかもしれない

本当の目的を知るためには、顧客にきちんとヒアリングをすることが重要



## ITシステムは現場の問題解決のために作られる

- ITシステムがどのような目的で必要とされているのかを知ることが重要
- 本当の問題を知るために各現場の様々な立場の視点を集める必要がある
  - 要件定義や現場への導入に失敗すると、現実世界と情報世界が紐づかない  
→ ITシステムを作っても現場の問題は解決されない



ヒアリングを受けた人は本当に欲しいものを答えられないことがある

格言：人々に望むものを聞いたら、もっと早い馬が欲しいと答えただろう

- 自動車王ヘンリーフォードの格言
- 自動車を知らなければ、自動車が欲しいとは答えられない

ヒアリングを受けた人は本当に欲しいものを答えられないことがある

格言：人々に望むものを聞いたら、もっと早い馬が欲しいと答えただろう

- ・ 自動車王ヘンリーフォードの格言
- ・ 自動車を知らなければ、自動車が欲しいとは答えられない

情報技術を知らなければ、こういうITシステムが欲しいとは答えられない

例)「納税申請書類を再現したエクセルシートが欲しい」

素朴な発想：納税申請業務デジタル化のために用紙をエクセルで再現できたらいい

→情報が記入しづらい、Officeを買ってない市民が手続きできない

本当の課題：

- ・ 行政情報をデータベース化して、必要に応じてPCから取り出したい
- ・ また、申請に基づいて行政手続きを円滑に行えるようにしたい

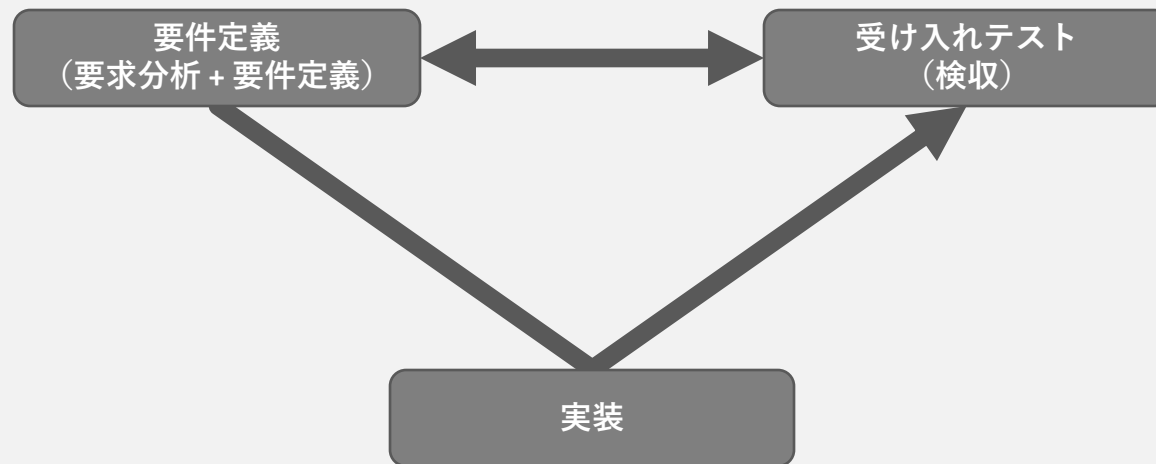
より適切な解決策：

- ・ データベースに何が格納されるのかを定義して、webフォームを作成する

(非エンジニアの社会人にもコンピュータの学習が求められる理由の一つ)

## (1)やりたいことを整理して、(2)それを実現するには何が必要か考える

- ・ 小規模な案件や柔軟に顧客にニーズに対応する現代的な開発方式の場合、  
開発企業と発注企業はV字モデルよりも密に連携する
- ・ 現実的には要求分析と要件定義をまとめて「要件定義」とすることが多い

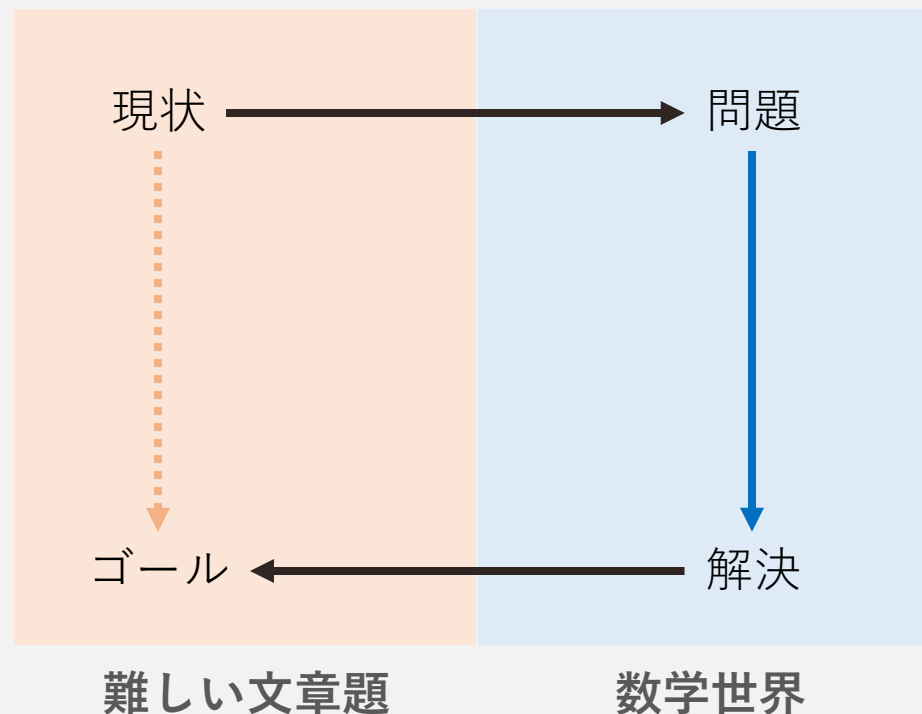


「やりたいことの整理」(要求分析) ができたら  
「実現するには何が必要なのか」を考える段階 (要件定義) に進む

やりたいことを小課題に分割して，計算機にできることと擦り合わせる

## 比喻) 大学二次試験の数学文章問題

- 複雑な状況をそのまま数学問題として解釈して，解法を思いつくのは難しい
- 状況を「**分解**」すれば考えやすくなる
  - 「総和を展開する」「数列的なパターンを探す」「同じ変数をまとめる」
- 数学にできることと「**すり合わせる**」ことで具体的な解決策に近づける
  - 「 $\Sigma$ の展開式」「数列漸化式の公式」「式展開の公式・方法論」

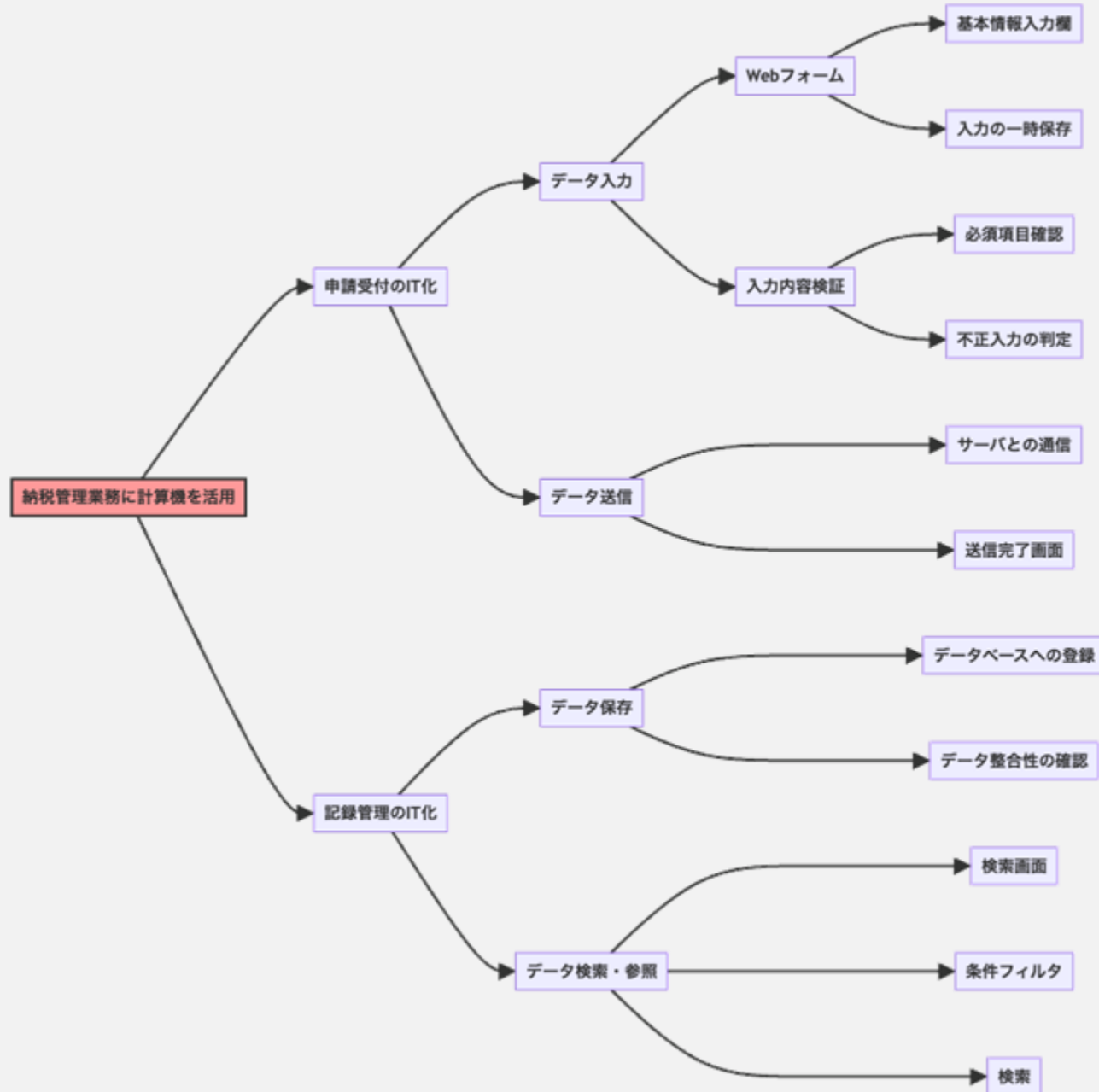


やりたいことを小課題に分割して、計算機にできることと擦り合わせる  
比喻) 大学二次試験の数学文章問題

- 複雑な状況をそのまま数学問題として解釈して、解法を思いつくのは難しい
- 状況を「分解」すれば考えやすくなる
  - 「総和を展開する」「数列的なパターンを探す」「同じ変数をまとめる」
- 数学にできることと「すり合わせる」ことで具体的な解決策に近づける
  - 「 $\Sigma$ の展開式」「数列漸化式の公式」「式展開の公式・方法論」

## 例) 納税申請業務をデジタル化する

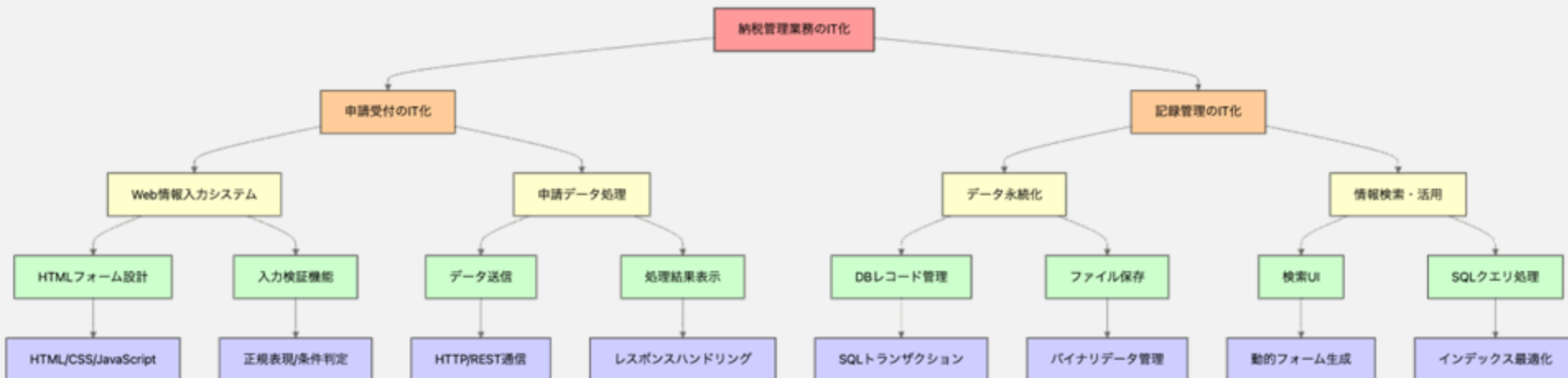
- やりたいことをいきなり「情報世界の問題」と解釈して解法を思いつくのは難しい
- 状況を「分解」すれば考えやすくなる
  - 「住民が納税申請を提出する」「申請内容を記録する」「職員がデータを探す」
- ITにできることと「すり合わせる」ことで具体的な解決策に近づける
  - 「webフォーム」「データベース」「検索」「データの絞り込み」



小さく分解された課題なら、計算世界の解き方を見つけやすい

- 🔍 ブラウザのフォームから入力したデータを保存したい→「webデータベース」
- 🔍 データを入力順に並び替えたい→「ソートアルゴリズム」
- 🔍 入力項目が不完全ならエラー通知したい→「バリデーションフレームワーク」
- ...

→ 計算機にできること（アルゴリズム）とすり合わせながら分解の仕方を調整して、実装の筋道をたてていく



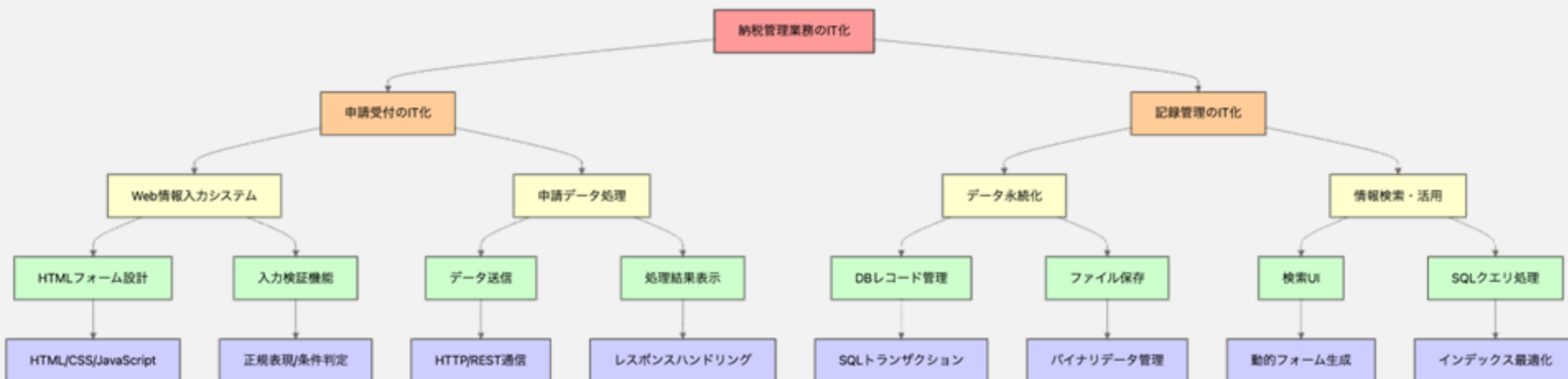
「物事をMECEに分解する力」と「計算機にできることの知識」が必要

MECE：Mutually Exclusive and Collectively Exhaustive, 漏れなくダブリなく

- 元の問題を厳密に維持したまま、解きやすい形に細かく分解する必要がある

計算機が実行できることに還元するための知識

- 目的の機能（図上側）を機能の組み合わせ（図下側）に分解するとき、  
計算機が実現できる形に近づけなければならない



本当の課題を見極め、役立つITシステムを定義し、実装・検証する

## 1. 企業がソフトウェアを発注・開発する流れ

- 利用者がやりたいことを整理する
- 発注者が欲しいシステムを定義する
- 開発者が定義通りに動くシステムを作って納入する

## 2. 要件定義の仕方

- 利用者の想像できるシステムが最善の解決策でない可能性に注意する
- 利用者のやりたいことを整理して、それを実現するには何が必要か考える
- 機能を細かく分割して、計算機にできることとすり合わせる

## 3. ソフトウェア開発モデル

- 計画に沿って安全に開発する方法論（ウォーターフォール、V字モデルなど）
- 開発しながら価値を見極める方法論（アジャイル、リーンスタートアップなど）

## 情報システム開発の型はV字モデルだけでない

計画からの逸脱を抑制しやすいものから、方針変更に対応しやすいものまで様々あり、場面に応じて使い分けられる

例1) 医療・金融システム、大企業の巨大な収益基盤のDX

実現したい価値は比較的明確だが、失敗は致命的

→ 事前計画からの逸脱を最小限に留めたい

→ 事前に定めた計画から逸脱しにくい開発工程

例2) webサービス、スタートアップ企業の製品開発

正解が明確に分かっていないが、ある程度の失敗は許容可能

→ 開発やサービス運営を進めつつ、より良い形を模索したい

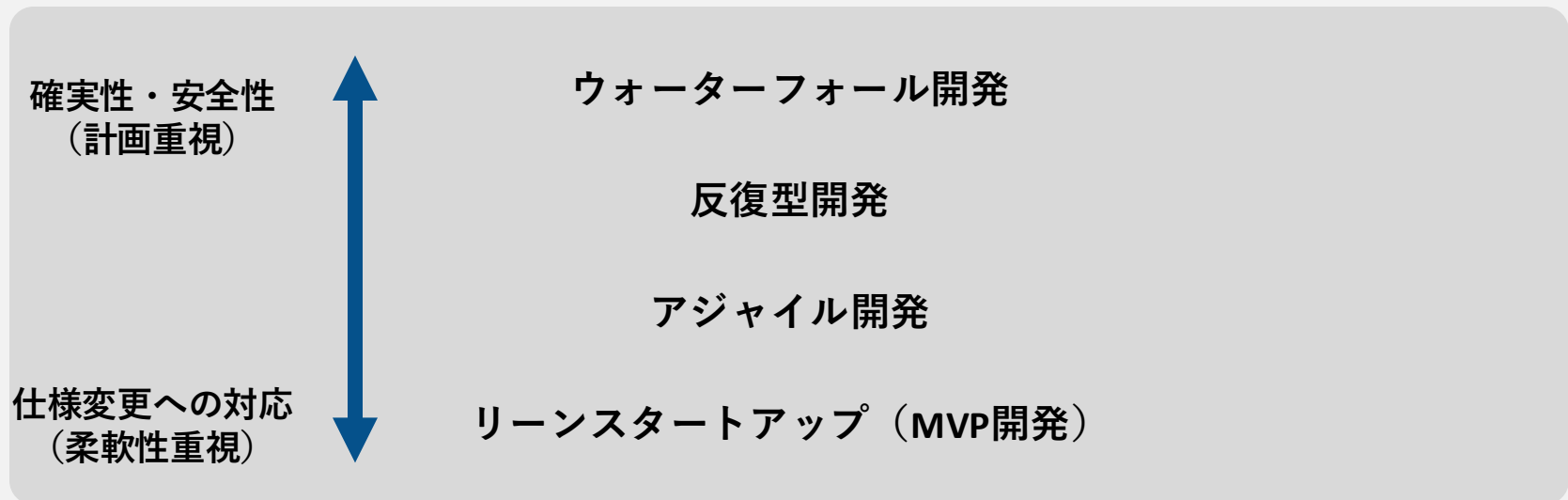
→ 方針変更への柔軟な対応のしやすい開発工程

例1) 医療・金融システム, 大企業の巨大な収益基盤のDX

→ウォーターフォール開発

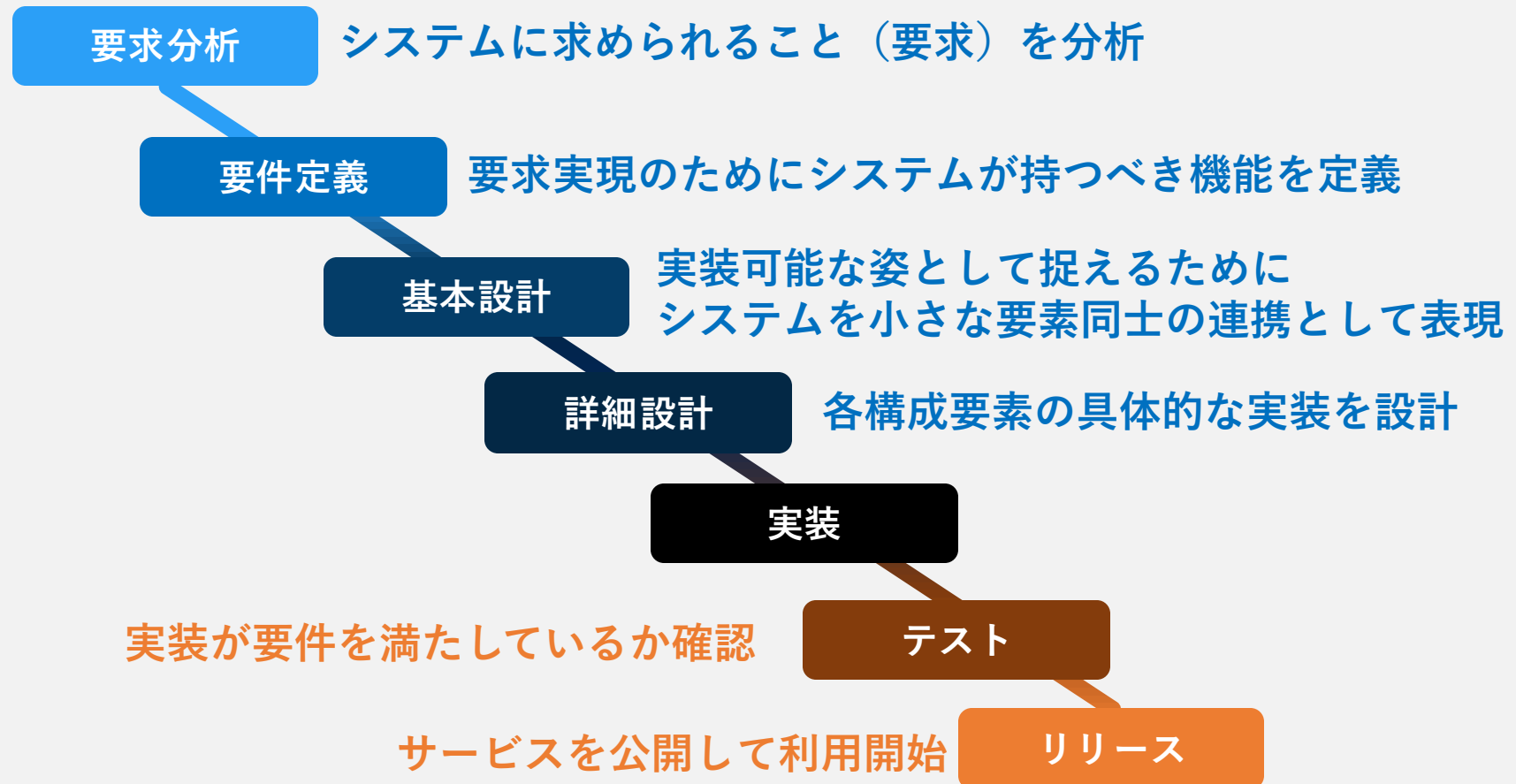
例2) webサービス, スタートアップ企業の製品開発

→アジャイル開発



事前に十分に計画を練り，逸脱や失敗なく完遂を目指すモデル

- ・ 要件定義，設計，実装，テスト，リリースまで一方向的に進める



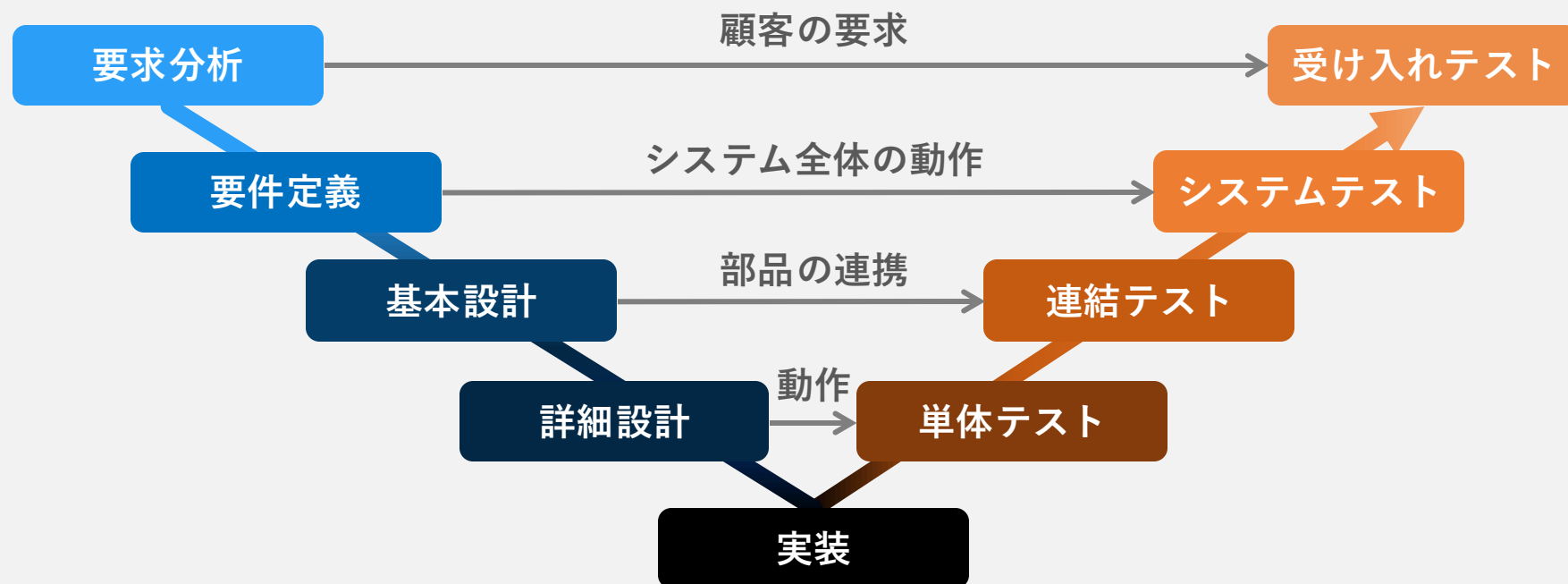
前半に設計・実装を行い，後半に各段階に対応したテストを実施

### 開発工程

- 
1. 顧客が求めているものを分析
  2. システムが持つべき機能を定義
  3. システムを細かな部品同士の連携で表現
  4. 具体的な各部品の仕組みを記述

### テスト工程

- 
9. 顧客の要求を満たすか確認
  8. システム全体の動作を確認
  7. 部品同士の連携を確認
  6. 各部品の動作を確認

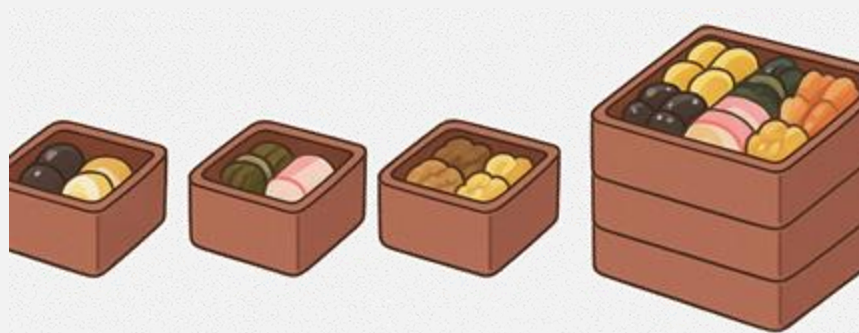


## 開発の過程で学び，要件を固めていく開発体制

最初から厳密に計画を詰めるのではなく，開発・試用を重ねて修正していく

- ・ 要件定義からテストまでの流れを何度も反復する
- ・ 開発者は前段階までに得た経験から学び，計画を調整しながら完成を目指す

### 例1：インクリメンタル開発（漸進的）



**Step1**

黒豆  
伊達巻

**Step2**

昆布巻き  
かまぼこ

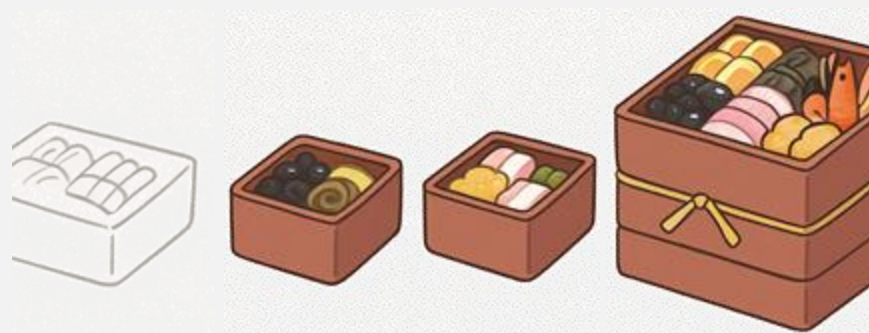
**Step3**

唐揚げ  
栗きんとん

**Step4**

組み合わせて完成

### 例2：イテラティブ開発（反復的）



**Step1**

スケッチ

**Step2**

試作

**Step3**

修正版  
(彩り調整)

**Step4**

修正版  
(分量増加)

## 開発過程での柔軟な要件の変化に対応するための考え方

計画を守る枠組みより状況の変化に対応できる開発体制を重視する価値観

- 計画，設計，実装，テストを小さいサイクルで何度も繰り返す
- リリースを繰り返しながら，他の開発者や顧客と密に対話してより良い形を探る
- スクラム，エクストリームプログラミングなど，色々な方法論が提案されている

早期に市場投入して、利用者の反応を見つつ迅速に改善を行うという考え方

- MVP：Minimum Viable Product, 実用最小限の製品
- 製品を実際に使いながら改善版を開発していく方式
  - 利用者の反応を検証しながら、製品を段階的に改善する



## 情報システム開発の型はV字モデルだけでない

計画からの逸脱を抑制しやすいものから、方針変更に対応しやすいものまで様々あり、場面に応じて使い分けられる

例1) 医療・金融システム、大企業の巨大な収益基盤のDX

実現したい価値は比較的明確だが、失敗は致命的

→ 事前計画からの逸脱を最小限に留めたい

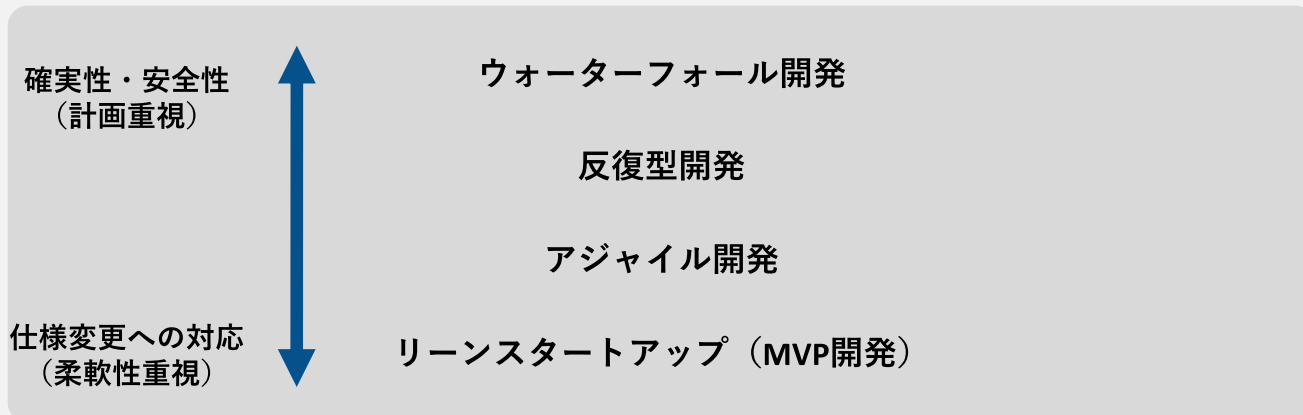
→ 事前に定めた計画から逸脱しにくい開発工程

例2) webサービス、スタートアップ企業の製品開発

正解が明確に分かっていないが、ある程度の失敗は許容可能

→ 開発やサービス運営を進めつつ、より良い形を模索したい

→ 方針変更への柔軟な対応のしやすい開発工程



計画や開発の過程で、現場担当と開発担当の連携がしやすくなる

例1) 現場と実装の事情が計画に織り込まれて、安全性・確実性が向上

- 要件定義や設計の過程で、現場の知識に基づく綿密な計画が可能になる
- リスクを抑制しやすくなる。また、それにより大掛かりな開発も着手可能になる

例2) 開発過程の試行錯誤に参加できて、要求の明確化がしやすい

- 開発会議に現場の知識を持つ人が参加できると、アジャイル開発などの柔軟な開発工程が採用しやすくなる
- 開発や試作を通じた具体的な体験に基づく価値の明確化がしやすくなる

本当の課題を見極め、役立つITシステムを定義し、実装・検証する

## 1. 企業がソフトウェアを発注・開発する流れ

- 利用者がやりたいことを整理する
- 発注者が欲しいシステムを定義する
- 開発者が定義通りに動くシステムを作って納入する

## 2. 要件定義の仕方

- 利用者の想像できるシステムが最善の解決策でない可能性に注意する
- 利用者のやりたいことを整理して、それを実現するには何が必要か考える
- 機能を細かく分割して、計算機にできることとすり合わせる

## 3. ソフトウェア開発モデル

- 計画に沿って安全に開発する方法論（ウォーターフォール、V字モデルなど）
- 開発しながら価値を見極める方法論（アジャイル、リーンスタートアップなど）

# IV. 計算機のおえ方を体感する

6. プログラムの基本

7. データ構造

8. アルゴリズム

9. 演習

## I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機，インターネット，情報システム

## II. 計算機やデータ，AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

## III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

## IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習（データ構造とアルゴリズム）

## V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

## VI. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

## VII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習（PERT図，決定表）

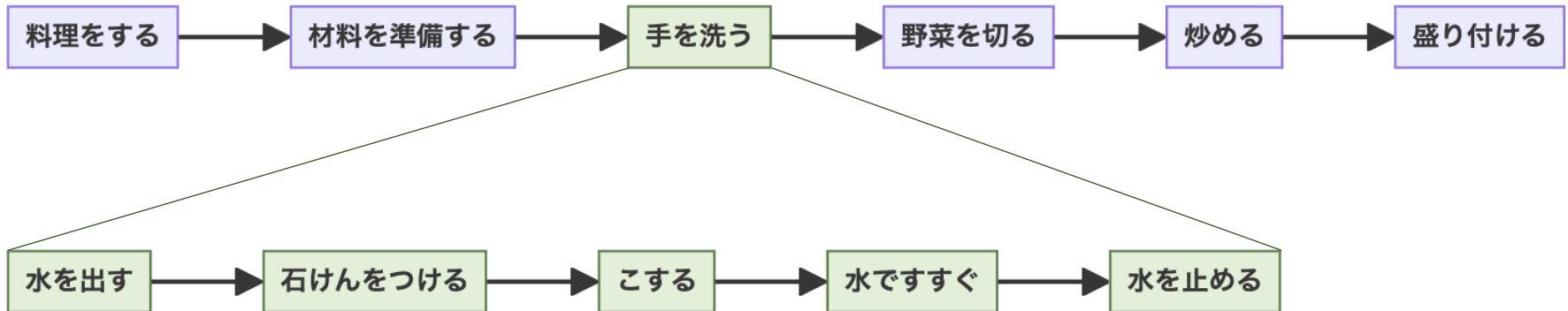
1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、  
目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる
3. ITシステムの構造や仕組みを学び、  
問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる
4. 計算機のハードウェア構成を知り、  
目的達成に必要な装置の見積もりや購入ができるようになる
5. 計算機に特有のデータ表現を学び、  
機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考を体験する

# 2025年度 コンピュータシステム 第6回：プログラムの基本

## 1. アルゴリズムの実行方法の種類

- 手続き型プログラム
- イベント駆動型プログラム

開始後に、計算手順が順番に実行されて、終了するプログラム



常に待機していて、刺激に応じて手続きを行うプログラム

例) ゲーム

- マウスでクリックしながら移動されたら、視点を動かす
- 方向キーが押されたら、キャラクターを動かす
- ジャンプボタンが押されたら、キャラクターをジャンプさせる
- 攻撃ボタンが押されたら、攻撃を発動する
- 防御ボタンが押されたら、防御を発動する
- ...

## アルゴリズムをどう計算機に実装するか？

### 1. プログラミングの記述方法

- 命令を言葉で記述する（プログラミング言語）
- 処理の流れを図示する（ビジュアルプログラミング言語）
- 画面操作でアプリを作る（ノーコード，ローコード）

### 2. プログラミング言語の基本

- 情報を保持する（変数，型）
- 情報を処理する（演算子）
- 繰り返し使う手順を一つの命令にまとめる（関数）

### 3. プログラミングの実際

- 便利な関数を利用する（標準関数，ライブラリ，API）

## 計算機への命令を言葉として記述できる仕組み

C, Java, Python, JavaScriptなど色々な種類がある

### 例) 数字当てゲームのプログラム

```
正解 = 7  
予想 = 入力受付()  
正解済み = 偽
```

```
(正解済み でない) 間は次を繰り返す{  
  もし (予想 == 正解)なら{  
    画面出力("正解!")  
    正解済み = 真  
  }  
  もし(予想 > 正解)なら{  
    画面出力("もっと小さいです!")  
    予想 = 入力受付()  
  }  
  もし(予想 < 正解)なら{  
    画面出力("もっと大きいです!")  
    予想 = 入力受付()  
  }  
}
```

日本語の疑似プログラムの例

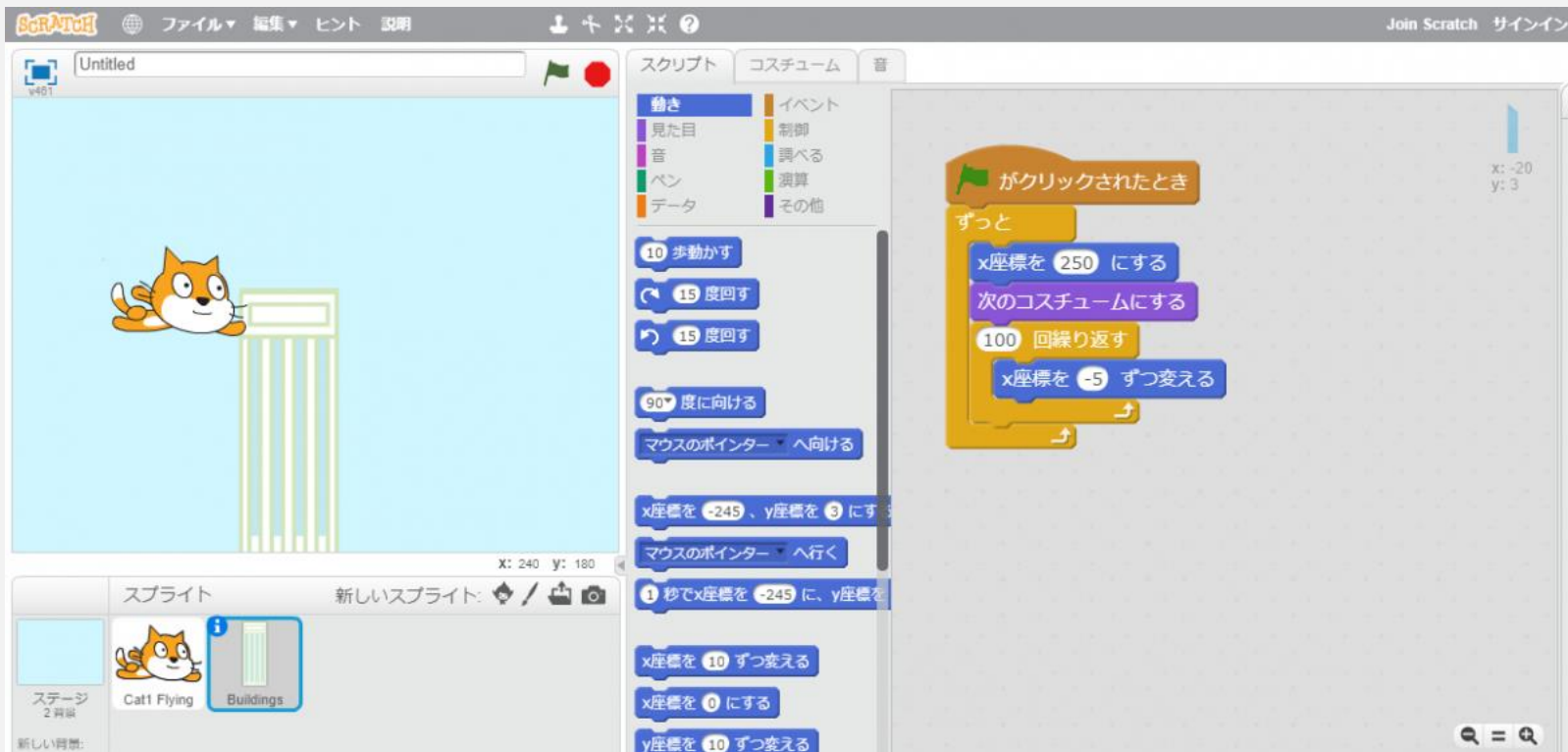
```
correct_answer = 7  
guess = int(input("数字を予想してください: "))  
is_correct = False
```

```
while not is_correct:  
    if guess == correct_answer:  
        print("正解!")  
        is_correct = True  
  
    if guess > correct_answer:  
        print("もっと小さいです!")  
        guess = int(input("数字を予想してください: "))  
  
    if guess < correct_answer:  
        print("もっと大きいです!")  
        guess = int(input("数字を予想してください: "))
```

実際に動作するプログラムの例  
(Python)

## 命令のブロックを並べて処理を記述する方法

### 例) Scratch



出典：「Scratch」（小学校を中心としたプログラミング教育ポータル），  
[https://www.mext.go.jp/miraino\\_manabi/content/285.html](https://www.mext.go.jp/miraino_manabi/content/285.html)（2025/05/22 に利用）

## 装置や数式を矢印でつなげて情報処理の流れを記述する方法

### 例1) ラダー図

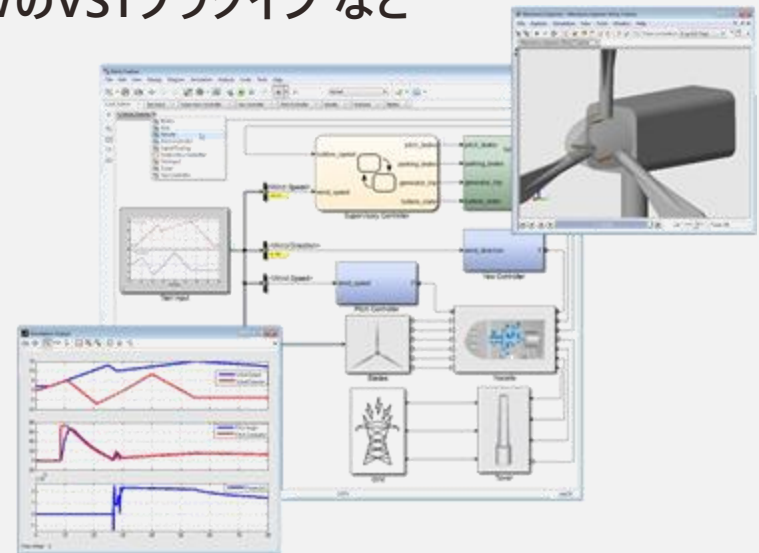
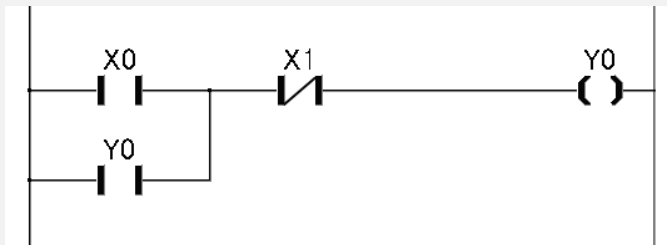
ロボット、自動倉庫、自動ドア、遊園地のアトラクションなど、電気・制御の専門家が機械の動作を制御する場面で利用される

### 例2) Simulink

ハードウェアシステムの設計・シミュレーション（動作確認）に使われるソフトウェア

### 例3) 3DCG, 画像・映像, 音楽を扱うソフトウェア

• Blender, DaVinci Resolve, DAWのVSTプラグイン など



## 画面操作でアプリを作ることのできる仕組み

- ・ ノーコード：画面操作のみでアプリを作る
- ・ ローコード：必要最小限のプログラムのみ記述して、他は画面操作で作る

## 計算や情報整理をノーコード，ローコードで実行できるソフトウェア

### 例1) Excelを用いたデータ分析

- =SUM(A1:A3) のように，セルの値を計算する
- 統計分析をしたり，グラフを作成したりする機能もある

### 例2) マクロを用いた定型文章の記入項目の再利用

- 一箇所に個人情報を入れたら，テンプレートの対応項目に自動挿入される

### 例3) マクロを用いたデータの自動計算

- 毎月各部署から届く売り上げデータを自動で合算して定型レポート化する

### 例) Google Formとスプレッドシートによる簡易的な物品管理

1. Google Formから物品の購入/廃棄に関する情報を送信  
項目例)

- ・ 購入時：記入者，日時，個数，購入金額
- ・ 廃棄時：記入者，日時，個数

2. 記入内容をスプレッドシートの最終行に自動入力  
入力例)

購入or廃棄， 記入者， 日時， 個数， 購入金額

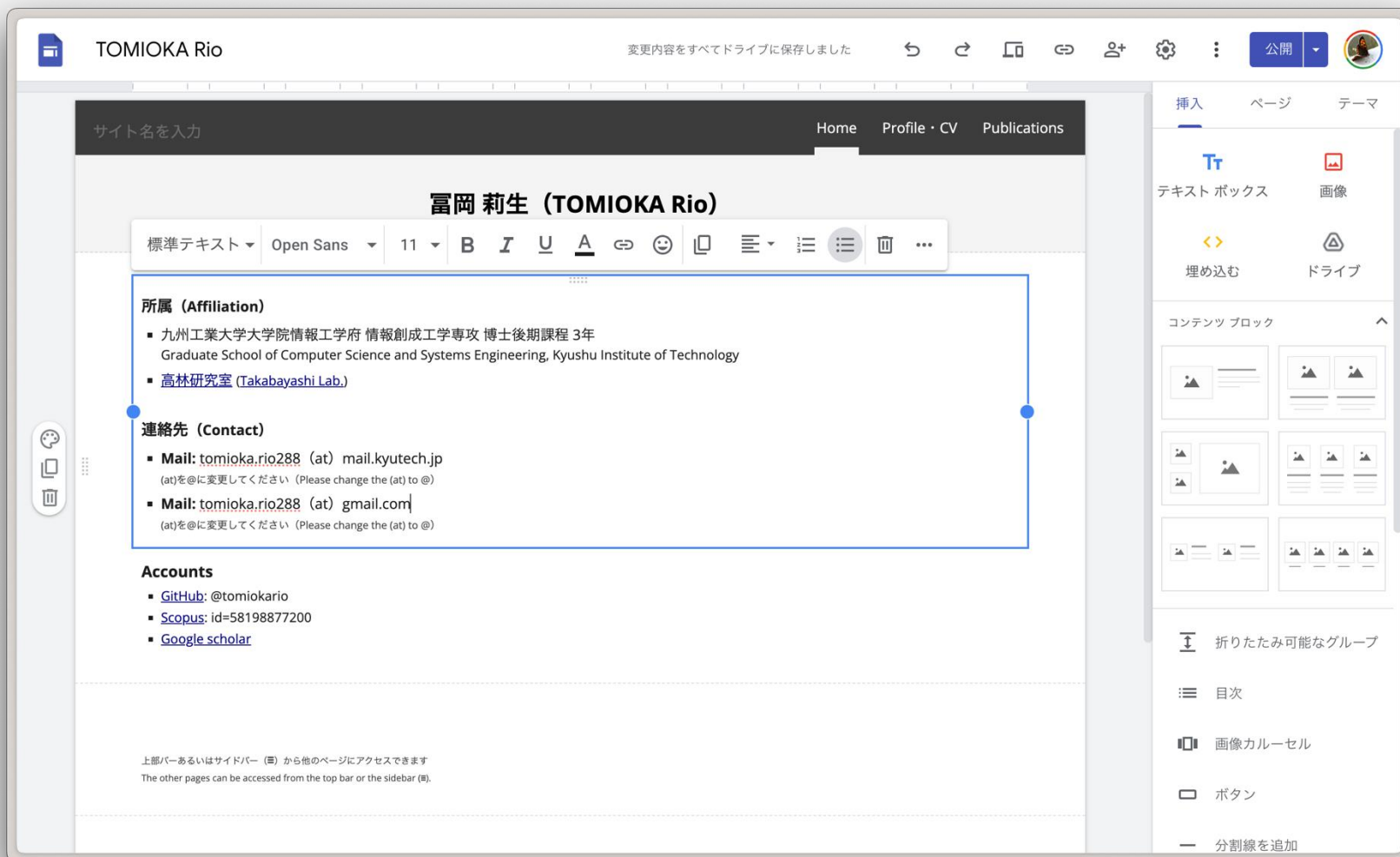
購入           ， John   ， 5/1， 3       ， 300

廃棄           ， 佐藤   ， 5/10， 1       ，

3. スプレッドシートの関数を使って色々な情報を管理・分析  
管理例) 現在の個数

分析例) 月ごとの購入金額， 平均消費量， 人ごとの買い出し回数

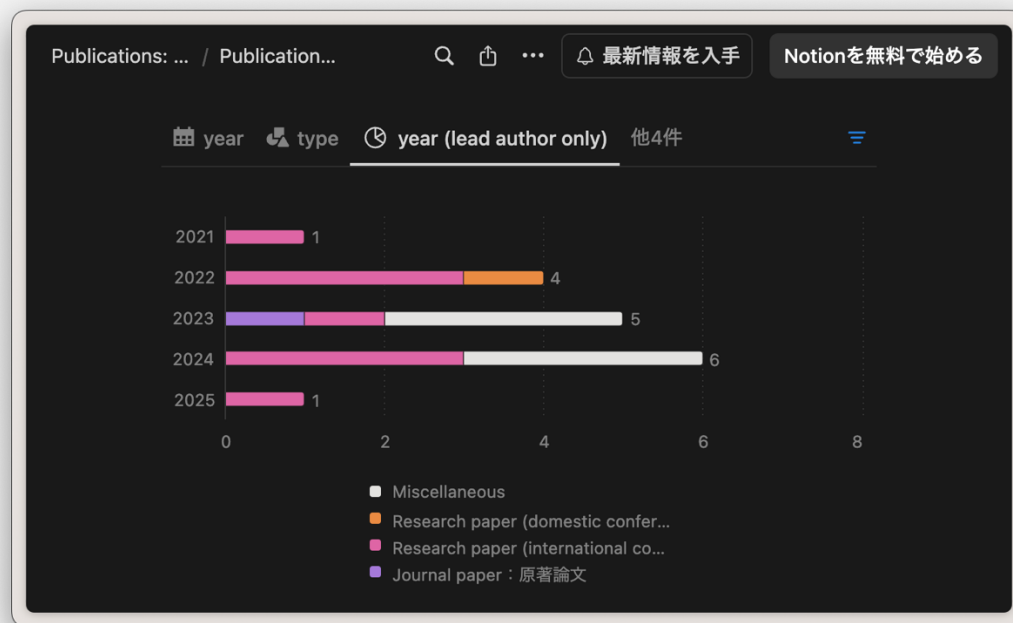
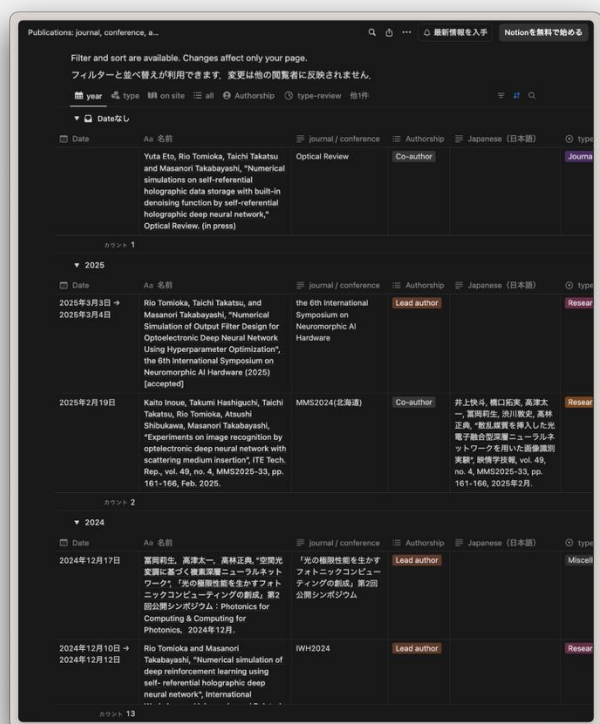
## 画面を操作してwebサイトを作れるサービス



※ <https://sites.google.com/view/tomiokario> で実際に動作している様子を見れます

## オンラインで情報を管理したり，それを公開したりできるツール

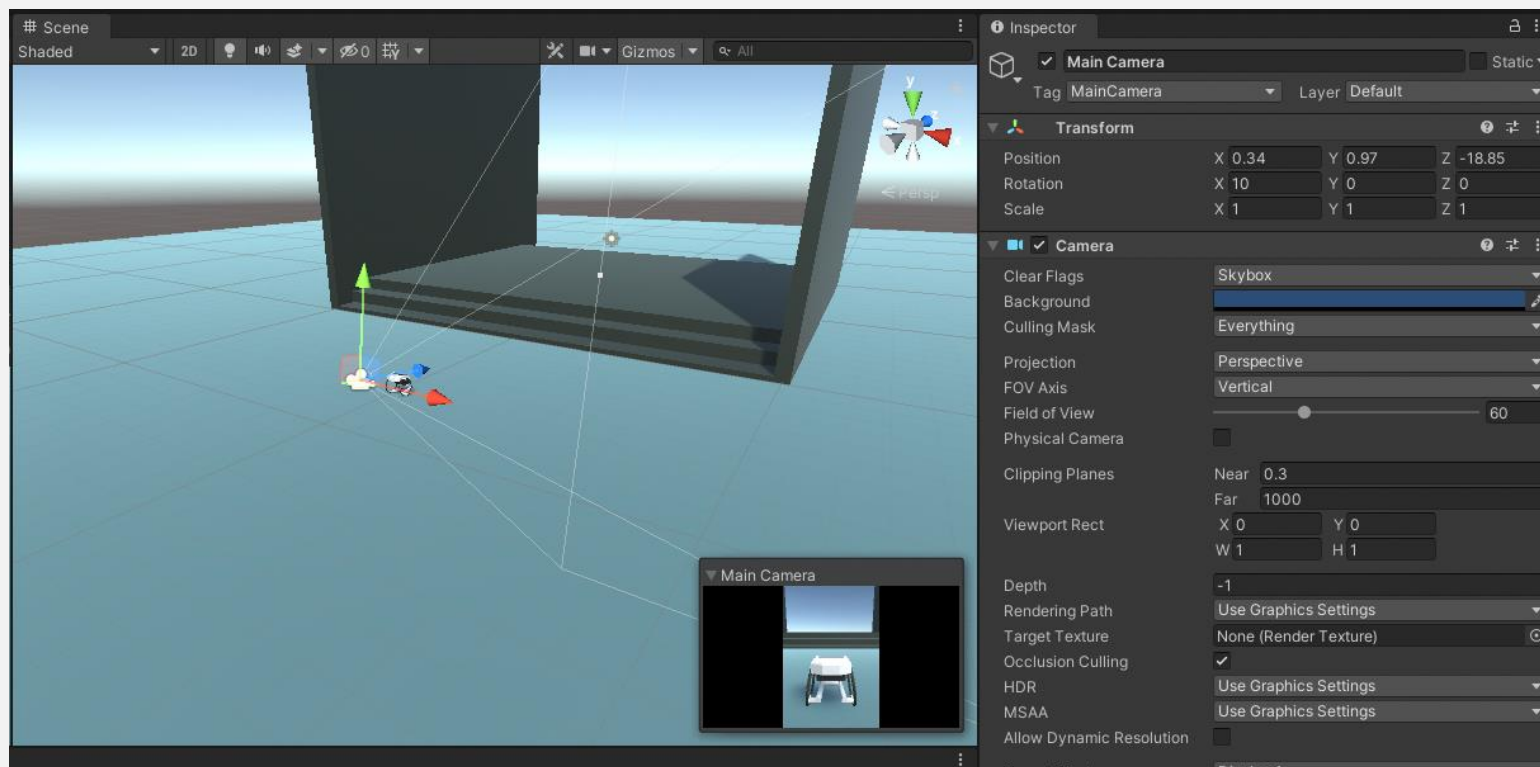
- いくつものメモをページやデータベースとして作成し，管理/公開できるサービス
- 画面操作で並べ替え，絞り込み，検索などができる
- カレンダーやタイルなど様々な形式で表示できる
- 外部データベースの参照やフォーマット作成により内容を自動作成できる



※このページは <https://tomiokario.notion.site/Publication-list-1a6532283c7a802a97edca590300b9f6?pvs=4> で試せます

## 画面操作とプログラミングを組み合わせるゲームを作れるソフトウェア

- 3D空間にオブジェクトを配置して空間を作ることができる
- オブジェクトのパラメータを操作して、特性を設定できる
- 移動や衝突判定などの基本的な動作は「機能」として付与できる
- 用意されていない機能はプログラミングして実装できる



## アルゴリズムをどう計算機に実装するか？

### 1. プログラミングの記述方法

- 命令を言葉で記述する（プログラミング言語）
- 処理の流れを図示する（ビジュアルプログラミング言語）
- 画面操作でアプリを作る（ノーコード，ローコード）

### 2. プログラミング言語の基本

- 情報を保持する（変数，型）
- 情報を処理する（演算子）
- 繰り返し使う手順を一つの命令にまとめる（関数）

### 3. プログラミングの実際

- 便利な関数を利用する（標準関数，ライブラリ，API）

## 情報を保持するための記述

### 例1) 文字列型の変数

1. 文字列  $a = \text{"Hello, "}$
2. 文字列  $b = \text{"John"}$
3. 文字列  $c = a + b$        $\rightarrow$      $c$ は $a$ と $b$ を並べた「"Hello, John"」
4. 文字列  $d = \text{"5"}$
5. 文字列  $e = c + d$        $\rightarrow$      $e$ は $c$ と $d$ を並べた「"Hello, John5"」

## 情報を保持するための記述

### 例1) 文字列型の変数

1. 文字列  $a = \text{"Hello, "}$
2. 文字列  $b = \text{"John"}$
3. 文字列  $c = a + b$        $\rightarrow$      $c$ は $a$ と $b$ を並べた「"Hello, John"」
4. 文字列  $d = \text{"5"}$
5. 文字列  $e = c + d$        $\rightarrow$      $e$ は $c$ と $d$ を並べた「"Hello, John5"」

### 例2) 数値型の変数

1. 数値  $a = 1$
2. 数値  $b = 2$
3. 数値  $c = a + b$        $\rightarrow$      $c$ は「 $1+2$ 」の結果の「 $3$ 」
4. 数値  $d = 4$
5. 数値  $e = c \div d$        $\rightarrow$      $e$ は「 $3 \div 4$ 」の結果の「 $0.75$ 」

## 情報を保持するための記述

### 例1) 文字列型の変数

1. 文字列  $a = \text{"Hello, "}$
2. 文字列  $b = \text{"John"}$
3. 文字列  $c = a + b$        $\rightarrow$      $c$ は $a$ と $b$ を並べた「"Hello, John"」
4. 文字列  $d = \text{"5"}$
5. 文字列  $e = c + d$        $\rightarrow$      $e$ は $c$ と $d$ を並べた「"Hello, John5"」

### 例2) 数値型の変数

1. 数値  $a = 1$
2. 数値  $b = 2$
3. 数値  $c = a + b$        $\rightarrow$      $c$ は「 $1+2$ 」の結果の「 $3$ 」
4. 数値  $d = 4$
5. 数値  $e = c \div d$        $\rightarrow$      $e$ は「 $3 \div 4$ 」の結果の「 $0.75$ 」

### 例3) 論理型（真偽値型）の変数

1. 論理  $a = \text{真}$
2. 論理  $b = a \text{でない}$        $\rightarrow$      $b$ は「真でない」ので「偽」
3. 論理  $c = a \text{ または } b$        $\rightarrow$      $c$ は「真または偽（真 or 偽）」なので「真」
4. 論理  $d = a \text{ かつ } b$        $\rightarrow$      $d$ は「真かつ偽（真 and 偽）」なので「偽」

## 情報を処理する「演算命令」を示すための記述

例1) 代入演算子： $=$

変数に情報を格納する命令（数学的なイコールではない）

## 情報を処理する「演算命令」を示すための記述

### 例1) 代入演算子： $=$

変数に情報を格納する命令（数学的なイコールではない）

### 例2) 四則演算子： $+$ , $-$ , $\times$ , $\div$

同じ演算子で「型」ごとに異なる挙動が定義される場合がある

1. 文字列  $a = \text{"I am"}$   
2. 文字列  $b = \text{"John"}$   
3. 文字列  $c = a + b$   
→  $c$ は「I am John」

1. 数値  $a = 1$   
2. 数値  $b = 2$   
3. 数値  $c = a + b$   
→  $c$ は「3」

1. 文字列  $a = \text{"version"}$   
2. 数値  $b = \text{"3"}$   
3. 文字列  $c = a + b$   
→ エラー

## 情報を処理する「演算命令」を示すための記述

### 例1) 代入演算子：**=**

変数に情報を格納する命令（数学的な**イコール**ではない）

### 例2) 四則演算子：**+**, **-**, **×**, **÷**

同じ演算子で「型」ごとに異なる挙動が定義される場合がある

1. 文字列 a = "I am "  
2. 文字列 b = "John"  
3. 文字列 c = a + b  
→ cは「I am John」

1. 数値 a = 1  
2. 数値 b = 2  
3. 数値 c = a + b  
→ cは「3」

1. 文字列 a = "version "  
2. 数値 b = "3"  
3. 文字列 c = a + b  
→ エラー

### 例3) 比較演算子：**<**, **>**, **≠**, **==** (**イコール**)

変数同士を比較して論理値（真か偽か）を計算する命令

1 == 1 → 真, 1 ≠ 1 → 偽, 1 == 2 → 偽, "hello" == "hello" → 真  
2 > 1 → 真, 1 ≥ 1 → 真, 3 < 1 → 偽, -2 ≤ 0 → 真

## 情報を処理する「演算命令」を示すための記述

### 例1) 代入演算子：`=`

変数に情報を格納する命令（数学的なイコールではない）

### 例2) 四則演算子：`+`，`-`，`×`，`÷`

同じ演算子で「型」ごとに異なる挙動が定義される場合がある

1. 文字列 `a = "I am "`  
2. 文字列 `b = "John"`  
3. 文字列 `c = a + b`  
→ `c`は「I am John」

1. 数値 `a = 1`  
2. 数値 `b = 2`  
3. 数値 `c = a + b`  
→ `c`は「3」

1. 文字列 `a = "version "`  
2. 数値 `b = "3"`  
3. 文字列 `c = a + b`  
→ エラー

### 例3) 比較演算子：`<`，`>`，`≠`，`==`（イコール）

変数同士を比較して論理値（真か偽か）を計算する命令

`1 == 1` → 真，`1 ≠ 1` → 偽，`1 == 2` → 偽，`"hello" == "hello"` → 真  
`2 > 1` → 真，`1 ≥ 1` → 真，`3 < 1` → 偽，`-2 ≤ 0` → 真

### 例4) 論理演算子：`not`（でない），`and`（かつ），`or`（または）

論理（真偽）値の計算をするための命令（使用例は次ページで説明）

| A | Aでない |
|---|------|
| 真 | 偽    |
| 偽 | 真    |

| A | B | AかつB |
|---|---|------|
| 真 | 真 | 真    |
| 真 | 偽 | 偽    |
| 偽 | 真 | 偽    |
| 偽 | 偽 | 偽    |

| A | B | AまたはB |
|---|---|-------|
| 真 | 真 | 真     |
| 真 | 偽 | 真     |
| 偽 | 真 | 真     |
| 偽 | 偽 | 偽     |

「もし（条件）なら {処理} 」という形で書く

- 条件が真のとき、処理内容が実行される
- 条件が偽のとき、処理内容は実行されない

数値 月 = 5

文字列 天気 = “晴れ”

文字列 危険 = “なし”

もし（月==12 or 月==1 or 月==2）なら {  
文字列 季節 = “冬”

}

もし（月==3 or 月==4 or 月==5）なら {  
文字列 季節 = “春”

}

もし（月==6 or 月==7 or 月==8）なら {  
文字列 季節 = “夏”

}

もし（月==9 or 月==10 or 月==11）なら {  
文字列 季節 = “秋”

}

もし（季節==冬 and 天気== 晴れ）なら {  
危険 = “火事”

}

例1) 季節判定プログラム

数値 価格 = 100

文字列 商品カテゴリ = 食品

もし（商品カテゴリ == “食品”）なら {  
数値 税率 = 0.08

}

もし（商品カテゴリ == “その他”）なら {  
数値 税率 = 0.10

}

数値 消費税 = 価格 × 税率

数値 合計金額 = 価格 + 消費税

例2) 消費税計算プログラム

「（数値） 回繰り返す {処理} 」という形で書く

- ・ 指定した回数だけ、処理が実行される

「（条件）の間は繰り返す {処理} 」という形で書く

- ・ 条件が真である間は、処理が繰り返し実行される
- ・ 条件が偽になったら、処理は実行されなくなる

数値 腕立て回数 = 0

```
(10) 回繰り返す {  
    腕立て回数 = 腕立て回数 + 1  
}
```

数値 腹筋回数 = 0

```
(15) 回繰り返す {  
    腹筋回数 = 腹筋回数 + 1  
}
```

例1) 筋トレの手順

数値 貯金 = 0  
数値 目標 = 10000  
数値 毎週 = 1000

```
(貯金 < 目標) の間は繰り返す {  
    貯金 = 貯金 + 毎週  
}
```

例2) 貯金の手順

## 繰り返し使う手順を一つの命令にまとめる記述

例えば、「（名前）は（変数）を用いる {処理} 」のような形で書く

（支払い総額）は（数値 価格, 数値 税率）を用いる {  
数値 消費税額 = 価格 × 税率  
数値 総額 = 価格 + 消費税額  
}

数値 弁当の価格 = 150  
数値 弁当の税率 = 0.10  
数値 弁当の総額 = 支払い総額（弁当の価格, 弁当の税率）

数値 白菜の価格 = 250  
数値 白菜の税率 = 0.08  
数値 白菜の総額 = 支払い総額（白菜の価格, 白菜の税率）

例1) 税率計算の関数

（設定の初期化）は（ ）を用いる {  
文字列 言語 = 「日本語」  
文字列 テーマ = 「ライトモード」  
}

例2) リセットボタンの動作

## アルゴリズムをどう計算機に実装するか？

### 1. プログラミングの記述方法

- 命令を言葉で記述する（プログラミング言語）
- 処理の流れを図示する（ビジュアルプログラミング言語）
- 画面操作でアプリを作る（ノーコード，ローコード）

### 2. プログラミング言語の基本

- 情報を保持する（変数，型）
- 情報を処理する（演算子）
- 繰り返し使う手順を一つの命令にまとめる（関数）

### 3. プログラミングの実際

- 便利な関数を利用する（標準関数，ライブラリ，API）

## プログラミング言語が標準で提供する関数

- 入力受付，文字表示，ファイル操作，三角関数など基本的な処理は用意されていることが多い
- 自力で実装しようとする前に，行いたい処理が用意されているか調べると簡単にプログラミングを書けることがある

- 使用例) 数当てゲーム

```
数値 正解 = 7
数値 予想 = 入力受付()
論理 正解済み = 偽
(正解済み でない) の間は繰り返す{
    もし (予想 == 正解) が真なら{
        画面出力("正解!")
        正解済み = 真
    }
    もし(予想 > 正解)なら{
        画面出力("もっと小さいです!")
        予想 = 入力受付()
    }
    もし(予想 < 正解)なら{
        画面出力("もっと大きいです!")
        予想 = 入力受付()
    }
}
```

入力受付()はキー入力を受け取る関数

画面出力()は()の中の文字列を画面に表示する関数

よく使われるプログラムが詰め合わせで配布されているもの

- プログラミング言語の標準関数に用意されていない計算でも、ライブラリを読み込めば簡単に使えるようになることがある
- 先人が効果的に実装してくれた関数などを自分のプログラムの中で簡単に呼び出して使うことができる
- データベースの操作，基本的な統計・機械学習，一般的な数学演算など，よく使われる処理であれば，大抵はライブラリとして配布されている

例) Pandas

- Pythonというプログラミング言語でデータ分析をする人のためのライブラリ
- 表形式のデータ（例：Excel製データ）を扱うための処理が多数格納されている



## 外部のサービス/ソフトウェアをプログラムに組み込める仕組み

### Application Programming Interface

アプリケーションをプログラムするための接続機構

(Interface : 界面, 異なる仕組み同士の接合部分)

有名な多くのサービス/ソフトウェアは開発者向けにAPIを提供している

例) Instagram, Google Maps, Office365, 楽天トラベル, 気象庁, zoom  
Google Calendar, ChatGPT, Google Drive, 国会図書館, など

### 使用例) Google翻訳 API

テキストと翻訳先の言語を指定すると, 翻訳されたテキストが得られるAPI

1. 文字列 `jp_text` = “こんにちは!”
2. 文字列 `en_text` = `translate_client.translate(jp_text, target_language="jp")`

→`output`にはGoogle翻訳の結果 (“Hello!”) が格納される